



UNIVERSIDAD MAYOR DE SAN SIMÓN
FACULTAD DE CIENCIAS Y TECNOLOGÍA

DIRECCIÓN DE POSGRADO



DIPLOMADO CIENCIA DE DATOS

TERCERA VERSIÓN

MODELO PREDICTIVO PARA LA IDENTIFICACIÓN DEL RIESGO DE REPROBACIÓN EN EL EXAMEN DE INGRESO FCyT-UMSS

**PROYECTO PRESENTADO PARA OBTENER EL GRADO DE LICENCIATURA EN
INGENIERÍA INFORMÁTICA
MODALIDAD DOBLE TITULACIÓN**

POSTULANTE : CLEYSI MANÁ CHAMACA OLIVAR
TUTORA : Ing. MSc. ROXANA GUILLEN SALVADOR

Cochabamba – Bolivia
2026

MODELO PREDICTIVO PARA LA IDENTIFICACIÓN DEL RIESGO DE REPROBACIÓN EN EL EXAMEN DE INGRESO FCyT–UMSS

Por

Cleysi Maná Chamaca Olivar

El presente documento, Trabajo de Grado es presentado a la Dirección de Posgrado de la Facultad de Ciencias y Tecnología en cumplimiento parcial de los requisitos para la obtención del grado académico de Licenciatura en Ingeniería Informática, modalidad Doble Titulación, habiendo cursado el Diplomado “Ciencia de Datos” propuesta por el Centro de Estadística Aplicada (CESA) en su tercera versión.

ASESOR/TUTOR

M.Sc. Ing. Guillen Salvador Roxana

COMITÉ DE EVALUACIÓN

Ing. M.Sc. Ronald Edgar Patiño Tito. (Presidente)

Ing. M.Sc. Guillen Salvador Roxana. (Coordinador)

Ing. M.Sc. (Tribunal)



CENTRO DE ESTADÍSTICA APLICADA (CESA)
DEPARTAMENTO DE MATEMÁTICAS
FCyT - UMSS

Aclaración

Este documento describe el trabajo realizado como parte del programa de estudios de Diplomado “Ciencia de Datos” en el Centro de Estadística Aplicada CESA y la Dirección de Posgrado de la Facultad de Ciencias y Tecnología. Todos los puntos de vista y opiniones expresadas en el mismo son responsabilidad exclusiva del autor y no representan necesariamente las de la institución.

Resumen

El examen de ingreso a la Facultad de Ciencias y Tecnología de la Universidad Mayor de San Simón (FCyT–UMSS) es un proceso fundamental para el acceso a la educación universitaria, caracterizado por niveles históricos de reprobación más que de aprobación. El presente proyecto analiza el desempeño académico de los postulantes y desarrollar modelos predictivos que permitan estimar la probabilidad de reprobación, empleando técnicas de análisis de datos y aprendizaje automático.

En la metodología se basó en el análisis exploratorio y estadístico de un conjunto de datos históricos compuesto por 98 009 registros. Se realizó la limpieza, transformación y selección de variables demográficas y académicas relevantes, tales como edad, sexo, tipo de colegio y resultado final, con el fin de identificar patrones asociados al resultado del examen. Posteriormente, se desarrollaron modelos predictivos utilizando los algoritmos XGBoost, Random Forest y Regresión Logística, considerando el desbalance entre las clases de aprobados y reprobados. El desempeño de los modelos fue evaluado mediante métricas como accuracy, recall, precision, F1-score y el área bajo la curva ROC (AUC).

Los resultados evidencian una tasa de reprobación superior al 75 % y diferencias significativas asociadas al tipo de colegio, donde los colegios particulares urbanos presentan mayores tasas de aprobación que los colegios fiscales y comunitarios rurales. Se determinó que existen variables que no tienen relevancia en el resultado del examen, como ser la edad, sexo y si el postulante trabaja o no.

Además de los modelos de clasificación, el proyecto incluye un análisis gráfico de los datos, utilizando Power BI. Este software permite elaborar dashboards para la comprensión del contexto de los datos analizados.

En cuanto al desempeño de los modelos, XGBoost obtuvo los mejores resultados, alcanzando un accuracy de 65 %, F1-score de 0.49, recall de 68%, precisión de 38% y un AUC de 0.73, lo que demuestra una capacidad moderada para identificar postulantes con mayor probabilidad de reprobación. Los resultados fueron moderados debido a la falta de variables numéricas con peso para el modelo predictivo, mencionando que la ejecución del modelo no presenta fallas y la evidencia de esto es que no existe sobreajuste.

Además, la implementación de una interfaz interactiva para la aplicación práctica del modelo, teniendo una base para futuras investigaciones si así se desea.

A pesar de las limitaciones de las variables disponibles, el estudio muestra que la analítica de datos y los modelos predictivos puede ser una herramienta de ayuda para el análisis del proceso de admisión universitaria.

Palabras clave

Análisis exploratorio de datos, Aprendizaje automático, XGBoost, Modelos predictivos y Examen de ingreso universitario.

Dedicado a mi padre, por enseñarme a poner esfuerzo, disciplina y responsabilidad en cada meta que me propongo. A mi madre, cuyo esfuerzo constante y ejemplo de superación fueron una fuente de motivación para seguir adelante y culminar este proyecto. A mis hermanos, por su apoyo incondicional, confianza y palabras de aliento a lo largo de todo el proceso.

Asimismo, dedico este trabajo a mi novio, por motivarme a no rendirme cuando más lo necesitaba; y a mi sobrino, cuya alegría y entusiasmo fueron un impulso constante para perseverar con optimismo y compromiso hasta el final.

Agradecimientos

Al la Ing. Roxana Guillen Salvador MSc. por su valiosa y desinteresada colaboración.

Al Ing. Danny Luis Huanca Sevilla MSc. por todo el apoyo y orientación brindado.

A todo el equipo de profesionales del diplomado de ciencia de datos por la transmisión de sus conocimientos y por brindarme la información generada en las clases y utilizada en el presente proyecto.

A mi novio

Tabla de contenidos

UNIVERSIDAD MAYOR DE SAN SIMÓN.....	1
1. Introducción.....	1
1.1. Antecedentes	2
1.2. Justificación	3
1.3. Planteamiento del problema.....	3
1.4. Objetivo general.....	4
1.4.1. Objetivos específicos.....	4
2. Marco teórico	5
2.1. Análisis Exploratorio de Datos	5
2.1.1. Concepto de Análisis Exploratorio de Datos (EDA)	5
2.1.2. Características del análisis exploratorio de datos	5
2.2. Concepto de Aprendizaje Automático	6
2.2.1. Tipos de Aprendizaje Automático	6
2.2.1.1. Aprendizaje Supervisado.....	6
2.2.1.2. Aprendizaje no supervisado.....	6
2.2.1.3. Aprendizaje por Refuerzo	7
2.3. Modelos predictivos de clasificación.....	7
2.3.1. Concepto de modelo predictivo	7
2.3.2. Importancia en la Educación.....	7
2.3.3. Proceso general de construcción de un modelo predictivo	8
2.3.4. Algoritmos más utilizados en aprendizaje supervisado	9
2.3.4.1. Técnicas de regresión.....	9
2.3.4.2. Técnicas de aprendizaje computacional	10
2.4. Evaluación de la técnica de Aprendizaje Supervisado.....	12
2.5. Rendimiento Académico Estudiantil	14
2.5.1. Concepto de rendimiento académico.....	14
2.5.2. Factores que influyen en el rendimiento académico	14
2.6. Examen de ingreso en la FCyT	14

2.7.	Tecnologías y herramientas usadas.....	14
2.7.1.	Google colab	15
2.7.2.	Python	15
2.7.3.	Librerías.....	15
2.7.4.	Power BI.....	15
3.	Marco metodológico.....	17
3.1.	Área de estudio	17
3.2.	Flujograma metodológico	17
3.3.	Fuentes de información.....	19
3.3.1.	Fuente de Información Secundaria	19
3.4.	Análisis exploratorio de datos.....	19
3.4.1.	Análisis del problema.....	19
3.4.2.	Recolección de datos	20
3.4.3.	Análisis exploratorio de datos	22
3.4.4.	Creación de nuevas variables.....	24
3.5.	Desarrollo de modelos predictivos.....	25
3.5.1.	Selección de variables para el modelado.....	25
3.5.2.	Procesamiento y limpieza de los datos	25
3.5.2.1.	Limpieza de datos y tratamiento de valores atípicos.....	26
3.5.2.2.	Asignación de variables dependiente e independientes.....	27
3.5.2.3.	División del conjunto en entrenamiento y prueba.....	27
3.5.2.4.	Codificación de variables categóricas y escalado de variables numéricas	28
3.5.3.	Desarrollo de modelos predictivos de clasificación	28
3.5.3.1.	Desarrollo del modelo con XGBoost	28
3.5.3.2.	Desarrollo del modelo con Random forest	31
3.5.3.3.	Desarrollo del modelo con Regresión Logística Binaria	33
3.6.	Evaluación del desempeño de los modelos predictivos	35
3.6.1.	Evaluación de modelos predictivos de clasificación	35
3.6.1.1.	Métricas de evaluación utilizadas para la evaluación de los modelos	35
3.6.1.2.	Evaluación del modelo con XGBoost.....	36

3.6.1.3.	Evaluación del modelo con Random forest.....	37
3.6.1.4.	Evaluación del modelo con Regresión logística binaria	38
3.7.	Desarrollo de la interfaz interactiva	39
4.	Análisis de Resultados y Discusión.....	47
4.1.	Resultados generales del análisis exploratorio de datos	47
4.2.	Resultados del desarrollo de modelos predictivos	54
4.3.	Resultados de la evaluación del desempeño de los modelos predictivos.....	54
4.3.1.	Resultados del modelo XGBoost	54
4.3.2.	Resultados del modelo Random forest.....	57
4.3.3.	Resultados del modelo Regresión logística binaria	59
4.4.	Resultados de la Comparación y Selección de modelos desarrollados.....	62
4.5.	Discusión de Resultados	65
4.6.	Resultados del desarrollo de la interfaz interactiva.....	67
4.6.1.	Interfaz del modelo predictivo	67
4.6.2.	Perfil estadístico del postulante	69
4.6.3.	Evaluación masiva de postulantes.....	70
4.6.4.	Perfil de postulantes en la evaluación masiva.....	71
5.	Conclusiones	72
6.	Recomendaciones.....	74
	Referencia Bibliográfica.....	75
	Anexos.....	79
Anexo 1.	Dataset de los datos de los postulantes	79
Anexo 2.	Creación de las variables	80
Anexo 3.	Filtrado y eliminación de datos para el modelo Predictivo.....	81
Anexo 4.	Asignación de variables categóricas y numéricas.....	81
Anexo 5.	Código de implementación de XGBoost	82
Anexo 6.	Código de implementación de Random Forest.....	83
Anexo 7.	Código de Implementación de Regresión logística binaria	84
Anexo 8.	Dashboards implementado Panorama General	85
Anexo 9.	Dashboards implementado Perfil de los estudiantes.....	86

Anexo 10. Dashboards implementado Origen y Colegio de los postulantes.....	87
Anexo 11. Dashboards implementado Aprobados y reprobados	88
Anexo 12. Repositorio de Github.....	89
Anexo 13. CD con el contenido del proyecto.....	90

Lista de figuras

Figura 3-1: Ubicación del área	17
Figura 3-2: Flujograma metodológico.....	18
Figura 3-3: Cantidad de filas y columnas en los datos	22
Figura 3-4: Información de las variables del dataset.....	23
Tabla 3-3: Tabla estadística de los datos.....	23
Figura 3-5: Columnas seleccionas para el modelo	25
Figura 3-6: Tabla de la edad y la cantidad que existe	26
Figura 3-7: Captura de código de la asignación de X e Y	27
Figura 3-8: Captura de código de la división de entrenamiento(train) y prueba(test).....	28
Figura 3-9: Captura de código de la escalación y one hot encoding	28
Figura 3-10: Configuración de XGBClassifier.....	29
Figura 3-11: Función RandomUnderSampler y el pipeline	29
Figura 3-11: Entrenamiento y predicciones	30
Figura 3-12: Configuración del modelo Random Forest.....	31
Figura 3-13: Pipeline de preprocesamiento, balanceo y modelado con Random Forest.....	32
Figura 3-14: Entrenamiento y predicciones	32
Figura 3-15: Configuración del modelo de Regresión Logística	33
Figura 3-16: Pipeline de preprocesamiento, balanceo y modelado con Regresión Logística	34
Figura 3-17: Entrenamiento del modelo de Regresión Logística.	34
Figura 3-18: Captura de la evaluación del modelo XGBoost.....	36
Figura 3-19: Captura de la evaluación del modelo Random forest	37
Figura 3-20: Captura de la evaluación del modelo Regresión Logística Binaria.	38
Figura 3-21: Fragmento del código de comparación gráfica de modelos	39
Figura 3-22: Fragmento del código de inicialización de la interfaz gráfica desarrollada con PyQt6.....	40
Figura 3-23: Estructura de archivos del sistema implementado.....	41
Figura 3-24: Fragmento del código de inicialización de la interfaz gráfica desarrollada con PyQt6.....	42
Figura 3-25: Fragmento del código para la carga del modelo predictivo y del dataset en el backend.	43
Figura 3-26: Fragmento del código para la ejecución del modelo en el backend.	43
Figura 3-27: Fragmento del código para la ejecución del modelo en el backend.	44

Figura 3-28: Fragmento del código para la predicción de un solo postulante.....	45
Figura 3-29: Fragmento del código para la predicción de varios postulantes	46
Figura 4-1: Dashboard, un Panorama general	47
Figura 4-2: Grafico de líneas de la tasa de aprobación por año	48
Figura 4-3: Grafico de líneas de la tasa de aprobación por año	49
Figura 4-4: Dashboard de perfil general de los postulantes	50
Figura 4-5: Dashboard del origen y colegio de los postulantes.....	51
Figura 4-6: Dashboard de aprobados y reprobados por origen y colegio.....	53
Figura 4-7: Resultados del conjunto de entrenamiento de XGBOOST.....	55
Figura 4-8: Resultados del conjunto de prueba de XGBOOST.....	55
Figura 4-9: Grafica de ROC del modelo XGBoost	56
Figura 4-10: Resultados del conjunto de entrenamiento de Random Forest.....	57
Figura 4-11: Resultados del conjunto de prueba de Random Forest.....	58
Figura 4-12: Grafica de curva de ROC del modelo Random Forest	59
Figura 4-13: Resultados del conjunto de entrenamiento de Regresión Logística Binaria.....	60
Figura 4-14: Resultados del conjunto de entrenamiento de Regresión Logística Binaria.....	61
Figura 4-15: Grafica de curva de ROC del modelo de Regresión logística binaria	61
Figura 4-16: Comparación de métricas de desempeño de los modelos.....	62
Figura 4-17: Interfaz del módulo de predicción individual del prototipo desarrollado.....	68
Figura 4-18: Interfaz del módulo de predicción individual del prototipo desarrollado.....	69
Figura 4-19: Interfaz del módulo de evaluación masiva de postulantes mediante carga de archivos	70
Figura 4-20: Interfaz del módulo de evaluación masiva de postulantes mediante carga de archivos	71

Lista de tablas

Tabla 3-1: Variables demográficas y académicas del conjunto de datos analizado	20
Tabla 3-2: Descripción de Variables datos de estudiantes	21
Tabla 3-4: Descripción de variables creadas	24
Tabla 3-5: Tratamiento de inconsistencias y valores atípicos en las variables.....	27
Tabla 3-6: Tabla resumen de variables creadas.....	35
Tabla 4-1: Tabla de comparación de las métricas de los modelos evaluados.	63
Tabla 4-2: Modelo seleccionado.	65
Tabla 4-3: Resultados del modelo predictivo de éxito/fracaso académico en Contreras (2020).....	65
Tabla 4-4: Resultados comparativos de los modelos aplicados	66

1. Introducción

En el ámbito de la educación superior, los procesos de admisión universitaria representan un desafío significativo para las instituciones educativas, ya que se busca el acceso equitativo y seleccionar a los postulantes con la preparación académica necesaria para enfrentar la formación universitaria. A nivel internacional, diversos estudios han demostrado que el análisis de datos académicos y el uso de técnicas de análisis estadístico permiten identificar patrones asociados al rendimiento académico y al riesgo de fracaso estudiantil, contribuyendo a la mejora de los procesos de admisión y a la toma de decisiones institucionales fundamentadas (Romero & Ventura, 2020; Baker & Inventado, 2014).

En el contexto latinoamericano, las universidades enfrentan una alta variedad en el perfil de los postulantes, influenciada por factores socioeconómicos, educativos y geográficos. Investigaciones realizadas en la región evidencian que variables como el tipo de unidad educativa del postulante, la condición laboral, el contexto socioeconómico y el acceso a recursos educativos influyen de manera significativa en el desempeño de los estudiantes en los exámenes de ingreso y en sus primeros años de formación universitaria (Baker & Inventado, 2014).

En Bolivia, los procesos de admisión universitaria reflejan similares problemáticas, caracterizadas por una marcada diversidad en la formación previa de los postulantes. Las universidades públicas reciben cada año a estudiantes provenientes de distintas regiones, contextos educativos y realidades socioeconómicas, lo que genera diferencias en el nivel de preparación académica con el que enfrentan los exámenes de ingreso. En este escenario, el análisis estadístico de los resultados de admisión se convierte en una herramienta clave para comprender el rendimiento académico de los postulantes y detectar posibles brechas en la formación previa (Ministerio de Educación, 2019).

En la Universidad Mayor de San Simón (UMSS), particularmente en la Facultad de Ciencias y Tecnología (FCyT), el examen de ingreso es una parte fundamental para regular el acceso a las distintas carreras. Cada año, la FCyT recibe un número de postulantes que oscila aproximadamente entre 290 y 2590 estudiantes, provenientes de diferentes provincias, regiones y contextos (FCyT, 2020-2022). Debido a la gran cantidad y variedad de postulantes se comprende que influyen factores como el tipo de unidad educativa, el lugar de procedencia, la condición laboral, el género y otras características demográficas y académicas para su estudio.

En este contexto, el presente proyecto desarrolla un análisis estadístico descriptivo del comportamiento académico de los postulantes a la FCyT-UMSS, complementado con la construcción de modelos predictivos de aprendizaje automático. Estos modelos permiten estimar la probabilidad de aprobación o reprobación en el examen de ingreso a partir de variables demográficas y educativas disponibles.

1.1. Antecedentes

Diversos estudios han abordado la predicción del rendimiento académico con técnicas de análisis estadísticos. En el trabajo titulado *“Modelo predictivo para la selección de postulantes destacados a una institución de educación superior”* (Burgueño, 2017) por ejemplo, se construyó un modelo predictivo para seleccionar postulantes destacados mediante minería de datos y la metodología Knowledge Discovery in Databases (KDD). El autor señala que *“los modelos de predicción alcanzan hasta un 80,4% de precisión, con una ratio de identificación de los postulantes destacados de hasta un 100% a un 75%”*. Este hallazgo evidencia la utilidad de los algoritmos de clasificación en la educación superior.

De forma similar, otros autores destacan la influencia de factores sociodemográficos en la predicción del desempeño. Según (Burgueño, 2017), *“las notas del colegio, el sexo femenino, la edad de 19 años o menos, el puntaje de lenguaje y el puntaje NEM...”* resultan relevantes en los modelos aplicados. Esto demuestra que no solo las variables académicas tradicionales son significativas, sino también los contextos sociales y personales de los estudiantes.

En el estudio titulado *“La puntuación del examen Ser Bachiller como predictor del rendimiento académico universitario”*, (López Altamirano y otros, 2020) analizaron la capacidad predictiva de este examen en Ecuador. Según los autores, *“el examen Ser Bachiller constituye en un instrumento de predicción del rendimiento académico universitario debido a que evalúa conocimientos elementales dentro del Dominio Matemático, Verbal, Social y Aptitud Abstracta”*. Este examen evalúa niveles de logros educativos en Matemáticas, Lengua y Literatura, Ciencias Naturales y Estudios Sociales, y está diseñado para quienes aspiran a ingresar a la educación superior. Los resultados mostraron que los estudiantes con puntajes bajos en este examen tienden a presentar dificultades en su desempeño académico inicial, evidenciando la necesidad de contar con mecanismos de predicción que permitan identificar a quienes podrían requerir apoyo adicional.

En la investigación *“Predicción del rendimiento académico mediante modelos de regresión logística y el análisis discriminante”* (Torres López y otros, 2024), se analizaron diversas variables que afectan el rendimiento académico de los estudiantes en los primeros semestres de ingeniería. Los autores señalan que *“el mejor modelo identificado es el de regresión logística y análisis discriminante, que combina en forma adecuada la significancia global, nivel de predictibilidad, posibilidad de manejo de las variables del modelo y la eficacia de políticas educativas específicas”*. Entre las variables más relevantes se encuentran la edad del estudiante, el promedio ponderado del período anterior y el ingreso familiar agregado. Este estudio evidencia que los modelos predictivos no solo permiten anticipar el desempeño académico de los estudiantes que probablemente destacarán, sino también detectar a aquellos con mayores riesgos de bajo rendimiento, facilitando la implementación de estrategias de acompañamiento y reducción de la deserción.

En la ciudad de Lima, Perú se desarrolla un estudio en el que aplica técnicas de minería de datos para predecir el rendimiento académico en estudiantes universitarios, resaltando que la Minería de Datos Educativa permite identificar factores que influyen en el aprendizaje y obtener información temprana sobre el desempeño estudiantil. El autor demuestra que algoritmos como Árboles de Decisión y Naive

Bayes pueden emplearse eficazmente para anticipar resultados académicos, lo que evidencia la utilidad de los modelos predictivos en contextos educativos (Menacho Chiok, 2020).

1.2. Justificación

Desde el ámbito académico, la construcción de un modelo predictivo del rendimiento en el examen de ingreso de la FCyT constituye un aporte significativo al análisis estadístico aplicado a la educación. El estudio implica el uso de metodologías de análisis exploratorio, procesamiento de datos y algoritmos de machine learning, lo que contribuye al desarrollo de conocimiento científico en el área de la Minería de Datos Educativa. Como se indica, *“la minería de datos educativos permite descubrir patrones relevantes que ayudan a comprender procesos de enseñanza-aprendizaje”* (Romero & Ventura, 2020).

En el ámbito institucional, los resultados obtenidos permiten a la FCyT disponer de una herramienta analítica que facilita la identificación de factores asociados al desempeño de los postulantes. Esta información técnica puede servir como insumo para la comprensión del perfil académico de los aspirantes y para el análisis interno de los procesos de admisión, ya que *“el uso de analítica académica constituye un recurso fundamental para la toma de decisiones institucionales en educación superior”* (García Peñalvo & Cruz Benito, 2019).

Finalmente, la importancia del proyecto radica en su aplicabilidad práctica. Los modelos predictivos y las visualizaciones generadas permiten identificar tendencias y patrones de rendimiento, aportando una base para el estudio de los factores que influyen en el examen de ingreso. De este modo, el proyecto trasciende un ejercicio meramente académico y se convierte en un recurso informativo útil dentro del análisis institucional de la universidad.

1.3. Planteamiento del problema

La Facultad de Ciencias y Tecnología (FCyT) de la Universidad Mayor de San Simón (UMSS) recibe cada gestión a un número considerable de postulantes que deben rendir un examen de ingreso como principal mecanismo de selección. Este proceso constituye la vía formal para acceder a las diversas carreras de la facultad; sin embargo, cada año se observa que una parte significativa de los postulantes no alcanza la calificación mínima requerida para ser admitidos. Esta situación evidencia que existe un grupo de estudiantes con mayor probabilidad de reprobación, pero que actualmente no puede ser identificado de manera previa al examen.

El análisis que realiza la institución sobre los resultados del examen de ingreso es predominantemente descriptivo. Si bien este enfoque permite conocer cifras básicas como porcentajes de aprobación o reprobación, no ofrece información suficiente para comprender los patrones de rendimiento ni los factores que influyen en el desempeño de los postulantes. Variables como la edad, el tipo de colegio de procedencia, el lugar de origen, la condición laboral o el historial académico previo, podrían relacionarse con la probabilidad de obtener una calificación favorable, pero sin el uso de técnicas analíticas avanzadas estos patrones permanecen ocultos.

Si bien la institución dispone de información histórica sobre los postulantes y sus resultados, no se cuenta con un mecanismo analítico específico que permita estimar de forma automática la probabilidad de reprobación antes del examen. Esto genera un espacio de oportunidad para complementar los análisis existentes mediante el uso de técnicas predictivas basadas en datos. En un contexto donde se dispone de registros amplios y sistematizados, la aplicación de modelos de clasificación puede contribuir a transformar esta información en estimaciones útiles para el estudio del examen de ingreso.

Ante esta situación, surge la siguiente interrogante central: ¿Cómo se puede mejorar la identificación de estudiantes propensos a reprobación en el examen de ingreso de la FCyT-UMSS? Resolver esta pregunta implica la necesidad de aplicar técnicas de análisis estadístico y modelos de clasificación que permitan transformar los datos disponibles en información útil para el análisis institucional. Mediante el uso de algoritmos de machine learning y la comparación de diversos modelos, es posible estimar la probabilidad de que un postulante repruebe, permitiendo así detectar perfiles de postulantes que quieren ingresar a la facultad.

1.4. Objetivo general

Desarrollar un modelo predictivo para la identificación del riesgo de reprobación en el examen de ingreso fcyt-umss

1.4.1. Objetivos específicos

- Realizar un análisis exploratorio de datos, sobre los datos históricos del examen de ingreso de la FCyT-UMSS, con el fin de identificar patrones, tendencias y características relevantes del desempeño de los postulantes.
- Desarrollar modelos predictivos de clasificación como ser XGBoost, Random Forest y Regresión Logística Binaria, utilizando datos históricos de postulantes mediante la aplicación de técnicas de procesamiento de datos.
- Evaluar el desempeño del modelo predictivo mediante métricas estadísticas como precisión, recall, matriz de confusión, f1-score y exactitud, determinando su capacidad para identificar postulantes propensos a reprobación.
- Desarrollar una interfaz interactiva, que permitan presentar las predicciones del modelo, para determinar si un postulante o muchos, están en riesgo o no de reprobación el examen de ingreso a través de sus datos.

2. Marco teórico

2.1. Análisis Exploratorio de Datos

2.1.1. Concepto de Análisis Exploratorio de Datos (EDA)

En el pasado, el análisis de datos dependía del trabajo humano, pero el aumento exponencial en la cantidad de información hizo necesario el desarrollo de sistemas automatizados capaces de aprender y adaptarse a cambios constantes. El aprendizaje automático, o machine learning, es una disciplina que utiliza algoritmos estadísticos para que las computadoras aprendan a partir de ejemplos y construyan modelos que puedan realizar predicciones. Estos modelos reciben datos de entrada, conocidos como características o "features", y generan una salida o resultado, llamado valor objetivo o etiqueta. En el aprendizaje supervisado, que es la base del modelado predictivo, el objetivo es entrenar modelos que puedan predecir con precisión resultados para datos nuevos, calculando patrones extraídos de datos históricos (Lamba & Madhusudhan, 2022).

El proceso de creación de un modelo predictivo es iterativo: primero se transforma el problema en un formato que la computadora pueda procesar, seleccionando las características relevantes; luego se elige y entrena un modelo con los datos disponibles; Finalmente, se evalúa su desempeño y se analizan los errores para mejorar la precisión. Desde la década de 1990, el aprendizaje automático ha sido aplicado en tareas como la clasificación de documentos, que consisten en asignar textos a categorías específicas utilizando técnicas supervisadas o no supervisadas. Además, los modelos predictivos tienen múltiples aplicaciones prácticas, como en el reconocimiento de voz e imágenes, sistemas de recomendación, detección de fraudes y mejoras en motores de búsqueda. En el ámbito educativo, estos modelos son herramientas valiosas para anticipar el rendimiento académico, lo que permite diseñar estrategias de apoyo y mejorar la toma de decisiones (Lamba & Madhusudhan, 2022).

2.1.2. Características del análisis exploratorio de datos

Los modelos predictivos se basan en información estadística e histórica recopilada en un período específico, la cual puede ser procesada mediante minería de datos para identificar factores críticos y realizar estimaciones en contextos no analizados previamente (Santesteban López & López Malo, 2008). Estos modelos utilizan un conjunto de variables que, tras su análisis y de acuerdo a su prioridad, permiten establecer cuáles son más influyentes en la situación analizada (Eckert & Suénaga, 2015). Cabe destacar que esta herramienta contribuye a comprender el desarrollo del evento y evaluar la capacidad de predicción de los resultados de acuerdo con la cantidad de información disponible en el momento de la estimación (Herrero y otros, 2013).

2.2. Concepto de Aprendizaje Automático

El aprendizaje automático (Machine Learning) es una rama de la inteligencia artificial que se enfoca en el desarrollo de algoritmos capaces de aprender automáticamente a partir de los datos, sin requerir instrucciones explícitas para cada tarea. Estos algoritmos analizan grandes volúmenes de información con el objetivo de identificar patrones, relaciones y regularidades que permitan realizar predicciones o tomar decisiones sobre nuevos datos. El aprendizaje automático se caracteriza por su capacidad de mejorar el desempeño de los modelos conforme aumenta la cantidad de datos disponibles, lo que lo convierte en una herramienta fundamental para aplicaciones de análisis predictivo y clasificación en diversos ámbitos, incluido el educativo (Sadiku y otros, 2025).

2.2.1. Tipos de Aprendizaje Automático

El aprendizaje automático se clasifica principalmente en tres tipos: aprendizaje supervisado, que predice resultados utilizando datos etiquetados, el aprendizaje no supervisado, que detecta patrones en datos sin etiquetar y el aprendizaje aprender mediante recompensas.

2.2.1.1. Aprendizaje Supervisado

El aprendizaje supervisado es un enfoque de Machine Learning en el que los algoritmos se entrenan utilizando datos previamente etiquetados, es decir, donde se conoce de antemano la respuesta o resultado esperado para cada conjunto de características. De esta forma, el modelo aprende a relacionar las entradas con los resultados y puede realizar predicciones sobre datos nuevos basados en esos patrones (Sandoval, 2018).

Dentro del aprendizaje supervisado se distinguen principalmente dos tipos de tareas: clasificación y regresión.

- **Clasificación:** El objetivo del algoritmo es asignar cada dato a una categoría específica. Para ello, analiza patrones en los datos de entrenamiento y, a partir de ellos, puede ubicar nuevos datos en el grupo correspondiente, lo que permite identificar a qué clase pertenece cada observación. (Sandoval, 2018)
- **Regresión:** En este caso, el algoritmo busca predecir un valor numérico continuo en lugar de una categoría. A partir de las relaciones aprendidas entre las variables de entrada y los resultados, genera estimaciones cuantitativas precisas para nuevas observaciones (Sandoval, 2018).

2.2.1.2. Aprendizaje no supervisado

En el aprendizaje no supervisado, los algoritmos trabajan únicamente con las características de los datos, sin disponer de etiquetas o resultados conocidos. El objetivo principal es identificar patrones o estructuras ocultas dentro del conjunto de datos. De esta manera, el algoritmo agrupa las observaciones

que presentan similitudes en sus atributos, asumiendo que los datos con características semejantes podrían pertenecer a la misma categoría o clúster. Este enfoque es útil para tareas como la segmentación de clientes, la detección de anomalías o la organización de información compleja cuando no se dispone de información de salida previamente definida (Sandoval, 2018).

2.2.1.3. Aprendizaje por Refuerzo

El aprendizaje por refuerzo (AR) es una técnica de Machine Learning cuyo objetivo es desarrollar un agente capaz de resolver problemas específicos mediante la interacción directa con un entorno, que puede ser real o simulado (Sutton & Barto, 2018). A diferencia de otros enfoques de aprendizaje automático, en los que los algoritmos se entrenan previamente con conjuntos de datos etiquetados de manera offline (Goodfellow y otros, 2016), en el AR el sistema aprende a partir de las experiencias obtenidas mientras actúa sobre el entorno. Cada acción recibe una retroalimentación en forma de recompensas o penalizaciones, diseñada por el usuario, que guía al agente para mejorar su desempeño y tomar decisiones más efectivas en el tiempo.

2.3. Modelos predictivos de clasificación

2.3.1. Concepto de modelo predictivo

Un modelo predictivo de clasificación es un algoritmo de aprendizaje supervisado que tiene como finalidad asignar una observación o instancia a una categoría previamente definida. Para ello, el modelo analiza patrones presentes en datos históricos etiquetados y aprende las relaciones entre las características (variables predictoras) y la clase objetivo. Una vez entrenado, el modelo puede estimar la probabilidad de que un nuevo caso pertenezca a una u otra categoría, permitiendo así anticipar comportamientos o resultados futuros (Kotsiantis, 2007).

En el ámbito educativo, los modelos de clasificación son ampliamente utilizados para predecir eventos como el riesgo de deserción, el rendimiento académico o la aprobación de asignaturas, ya que permiten transformar grandes conjuntos de datos estudiantiles en información útil para apoyar la toma de decisiones institucionales (Romero & Ventura, 2020).

2.3.2. Importancia en la Educación

El uso de modelos predictivos en el ámbito educativo ha tomado relevancia en la última década debido al incremento de datos generados por sistemas de gestión académica, evaluaciones estandarizadas y plataformas digitales de aprendizaje. La disciplina conocida como Educational Data Mining (EDM) o Minería de Datos Educativa ha demostrado que estos modelos permiten identificar patrones relevantes del proceso de aprendizaje y del rendimiento estudiantil, facilitando la intervención temprana y la personalización educativa (Baker & Inventado, 2014).

Diversos estudios evidencian que los modelos de clasificación ayudan a:

- Detectar estudiantes en riesgo académico (Kabakchieva, 2013).
- Predecir el rendimiento en cursos o exámenes de admisión (López Altamirano y otros, 2020)
- Comprender la influencia de factores sociodemográficos en los resultados educativos (Menacho Chiok, 2020).
- Optimizar programas de nivelación o tutorías preventivas (García Peñalvo & Cruz Benito, 2019).

En el caso específico de los exámenes de ingreso universitario, estos modelos proporcionan a las instituciones una herramienta objetiva para analizar perfiles de postulantes, anticipar dificultades y promover estrategias de preparación más eficientes.

2.3.3. Proceso general de construcción de un modelo predictivo

La elaboración de un modelo de clasificación sigue un ciclo sistemático que permite garantizar la calidad del modelo y la validez de las predicciones. Este proceso coincide con metodologías reconocidas como CRISP-DM (Cross Industry Standard Process for Data Mining) y KDD (Knowledge Discovery in Databases), este proceso incluye las siguientes etapas (Wirth & Hipp, 2000):

a. Selección y comprensión de los datos: Se recopilan los datos disponibles y se evalúa su estructura, calidad y relevancia para el problema de estudio.

b. Limpieza y preparación de los datos: Incluye tratamiento de valores faltantes, codificación de variables categóricas, normalización, detección de outliers y construcción de variables derivadas. Esta fase suele representar hasta el 70% del tiempo total del análisis (Kotsiantis, 2007).

c. Selección del modelo y entrenamiento: Se elige uno o varios algoritmos de clasificación y se entrenan con los datos etiquetados, optimizando hiperparámetros para mejorar el rendimiento (Moré, 2019).

d. Evaluación del modelo: Se utilizan métricas como exactitud, precisión, recall, F1-score y matriz de confusión para validar su desempeño con datos no utilizados en el entrenamiento (Moré, 2019) .

e. Implementación e interpretación: El modelo final se integra como herramienta de apoyo a decisiones. En el ámbito educativo, esta etapa implica interpretar qué variables influyen más en la predicción y cómo los resultados pueden usarse para mejorar el proceso de admisión o el acompañamiento estudiantil (Moré, 2019).

2.3.4. Algoritmos más utilizados en aprendizaje supervisado

De forma general, los enfoques y técnicas aplicadas en el análisis predictivo se agrupan en dos categorías: técnicas de regresión y técnicas de aprendizaje computacional. (Timón, 2017)

2.3.4.1. Técnicas de regresión

Los modelos de regresión son fundamentales dentro del análisis predictivo, dado que permiten construir representaciones matemáticas que explican las interacciones entre variables. La elección del modelo depende del contexto y de las características de los datos analizados. (Timón, 2017)

a. Regresión Logística Binaria

La regresión logística binaria es un método de aprendizaje supervisado utilizado para predecir variables dicotómicas, es decir, resultados que solo pueden tomar dos valores, como aprobar/reprobar. Su objetivo es estimar la probabilidad de que una observación pertenezca a una determinada clase en función de un conjunto de variables independientes (Hosmer y otros, 2013).

El modelo transforma una combinación lineal de las variables mediante la función logística: $p = 1/(1 + e^{(-z)})$ donde p es la probabilidad de la clase positiva y $z = \beta_0 + \beta_1 X_1 + \dots + \beta_n X_n$. La predicción final se obtiene comparando esa probabilidad con un umbral (generalmente 0.5).

Una de sus principales ventajas es su interpretabilidad, ya que permite identificar qué variables incrementan o reducen la probabilidad del resultado analizado. Por esta razón, es uno de los métodos más utilizados en estudios educativos para analizar riesgos académicos y predecir el rendimiento de los estudiantes (Kabakchieva, 2013).

Aunque la regresión logística es sencilla y eficiente, puede presentar limitaciones cuando las relaciones entre variables no son lineales o existen interacciones complejas. En esos casos, modelos como Random Forest pueden ofrecer un desempeño superior (Breiman, 2001).

b. Modelo de Regresión Lineal

La regresión lineal es un método que sirve para estudiar cómo una variable depende de una o varias. Para ello se construye una ecuación que busca predecir los valores de la variable de interés a partir de las variables que la explican. El modelo se ajusta usando el método de mínimos cuadrados, que consiste en reducir al máximo la diferencia entre los valores reales y los estimados.

Después del ajuste, se revisa qué tan importantes son las variables incluidas en el modelo, comprobando si sus coeficientes realmente aportan información. En el caso más simple, cuando solo hay una variable independiente, el resultado es una línea recta que se acerca lo más posible a los datos: algunos puntos

quedan por encima y otros por debajo, pero en conjunto permite hacer predicciones razonables. (Timón, 2017)

c. Árboles de Clasificación y Regresión

El Análisis Discriminante Óptimo Jerárquico (HODA) es una extensión del análisis discriminante óptimo que busca el modelo más preciso para predecir variables categóricas a partir de datos que combinan variables numéricas y categóricas. Su resultado es un árbol de decisión que utiliza categorías y puntos de corte en variables continuas, evaluando además la capacidad de generalización del modelo.

De manera similar, los árboles de clasificación y regresión (CART) son técnicas basadas en reglas que dividen los datos en grupos cada vez más homogéneos. Dependiendo de si la variable dependiente es categórica o numérica, generan un árbol de clasificación o de regresión. Cada división se aplica de forma recursiva hasta que ya no se logran mejoras, y el proceso finaliza en nodos terminales que representan las predicciones del modelo.

El proceso consiste en aplicar reglas que dividen el conjunto de datos en grupos cada vez más homogéneos:

- Se seleccionan divisiones en las variables que permitan separar mejor las observaciones.
- Cada división crea nuevos nodos, y el procedimiento se repite de manera recursiva.
- El algoritmo se detiene cuando ya no se logran mejoras o se cumplen ciertos criterios de parada definidos previamente.

Al final, cada observación termina en un nodo terminal, que está definido por un conjunto específico de reglas y representa el resultado final de la clasificación o predicción.

2.3.4.2. Técnicas de aprendizaje computacional

A continuación, se proporciona una breve descripción de algunos de estos métodos utilizados comúnmente para el análisis predictivo.

a. Random Forest

Random Forest es un algoritmo de aprendizaje supervisado basado en la construcción de múltiples árboles de decisión que trabajan de manera conjunta para mejorar la precisión y estabilidad de las predicciones. Este método utiliza dos estrategias: bagging (entrenar cada árbol con muestras aleatorias del conjunto de datos) y la selección aleatoria de variables en cada división, lo cual reduce el sobreajuste y aumenta la capacidad de generalización del modelo (Breiman, 2001).

La predicción final se obtiene mediante votación mayoritaria entre todos los árboles, lo que convierte al Random Forest en un modelo robusto frente al ruido, capaz de manejar grandes cantidades de datos y útil para identificar la importancia de las variables. En el ámbito educativo, se emplea para predecir el rendimiento académico y detectar estudiantes en riesgo debido a su alta precisión y facilidad de interpretación (Kabakchieva, 2013)

b. Redes Neuronales

Las redes neuronales son herramientas avanzadas de modelado no lineal capaces de representar relaciones complejas entre variables. Se emplean en una gran variedad de áreas, como finanzas, neurociencia, psicología cognitiva, medicina, ingeniería y física, y pueden utilizarse para tareas de predicción, clasificación o control.

Estas redes resultan especialmente útiles cuando no se conoce de forma precisa cómo se relacionan las entradas con las salidas. Su característica principal es que aprenden a partir de ejemplos, ajustando sus parámetros durante un proceso de entrenamiento. Existen tres enfoques principales de entrenamiento: supervisado, no supervisado y por refuerzo, siendo el aprendizaje supervisado el más común en la práctica.

c. Clasificador Naive Bayes

Los clasificadores bayesianos son un tipo específico de red bayesiana, diseñados para realizar clasificación supervisada, especialmente cuando se trabaja con datos de tipo discreto (Chaparro y otros, 2013).

El clasificador Naive Bayes funciona aplicando la Regla de Bayes para calcular la probabilidad de que un archivo pertenezca a una determinada categoría. Para ello, combina tres criterios:

- La probabilidad de que el archivo presente ciertas características si ya sabemos a qué clase pertenece.
- La probabilidad previa de que esa clase aparezca dentro del conjunto de entrenamiento.
- Con esta información, estima la probabilidad final de que el documento se clasifique en esa categoría (Anguiano Hernández, 2009).

d. K-vecinos más cercanos

El método del vecino más cercano es una técnica que no requiere suposiciones previas sobre los datos. La idea es que, cuando aparece un dato nuevo cuyo valor queremos predecir, se busca el dato más parecido que puede ser su vecino, dentro del conjunto ya conocido y se toma como referencia (Cover & Hart, 1967). Este enfoque puede mejorarse usando no solo un vecino, sino varios de los más próximos. A esto se le llama k-vecinos más cercanos (k-NN). Al considerar varios ejemplos en lugar de uno solo, el

modelo se vuelve más resistente al ruido en los datos y genera predicciones más estables (Moreno Seco, 2004).

e. XGBoost (Extreme Gradient Boosting)

XGBoost (Extreme Gradient Boosting) es un algoritmo de aprendizaje supervisado basado en árboles de decisión y desarrollado por Chen y Guestrin (2016). Pertenece a la familia de métodos de boosting, en los cuales los árboles se construyen de manera secuencial para corregir los errores de los modelos anteriores. Este proceso se fundamenta en la optimización por gradiente, lo que permite mejorar progresivamente el desempeño del modelo. (Chen & Guestrin, 2016)

A diferencia de otros algoritmos basados en árboles, como Random Forest, en XGBoost el usuario controla explícitamente la complejidad de los árboles mediante restricciones como profundidad máxima, tasas de aprendizaje y parámetros de regularización. Esta estructura, junto con el procesamiento en paralelo, el manejo de valores perdidos y la poda de árboles, contribuye a evitar el sobreajuste y a mejorar la eficiencia computacional. (Chen & Guestrin, 2016)

El algoritmo funciona construyendo primero un árbol inicial y calculando los residuales (errores). Luego se generan nuevos árboles que ajustan esos residuales de manera iterativa, reduciendo gradualmente el error total del modelo. Este proceso continúa hasta que el error deja de mejorar significativamente, siguiendo los principios del método de gradiente descendente. (Chen & Guestrin, 2016)

Entre sus principales ventajas se encuentran su alta precisión, su capacidad para manejar grandes volúmenes de datos con múltiples variables, el soporte para datos faltantes y su notable velocidad de entrenamiento. No obstante, el algoritmo puede requerir un alto consumo computacional y una cuidadosa selección de hiperparámetros para evitar sobreajuste. (Chen & Guestrin, 2016)

2.4. Evaluación de la técnica de Aprendizaje Supervisado

Al evaluar un modelo, se parte de un trabajo realizado con un conjunto específico de datos, tomando en cuenta que ya se ha aplicado un algoritmo de machine learning y generado un modelo inicial. En este punto, es necesario determinar si dicho modelo cumple adecuadamente con el objetivo planteado y si su desempeño es satisfactorio para la tarea que se quiere resolver (Molina Félix & Sangüesa i Solé, 2010).

Para esto, se emplean distintas métricas de evaluación, entre ellas se consideran las siguientes:

a. Accuracy: Es una métrica utilizada para evaluar modelos de clasificación, ya que proporciona una medida directa de la corrección general. Calcula la proporción de predicciones correctas respecto al número total de predicciones. Aunque la precisión es valiosa en muchos casos, puede no ser suficiente cuando los datos están desequilibrados (Moré, 2019).

b. Precisión: La precisión evalúa qué tan acertadas son las predicciones que el sistema ha clasificado como pertenecientes a una determinada clase, comparándolas con la clasificación realizada por un experto. La precisión se obtiene dividiendo la cantidad de verdaderos positivos entre la suma de verdaderos positivos y falsos positivos (Moré, 2019).

c. Recall: El recall indica el porcentaje de respuestas del conjunto de evaluación que el sistema ha clasificado correctamente. Se calcula dividiendo los verdaderos positivos entre la suma de verdaderos positivos y falsos negativos (es decir, los casos en los que el sistema falló al no detectar una opinión positiva).

De esta manera, aunque el sistema identifique correctamente muchas opiniones auténticas, si falla en clasificar alguna opinión como falso, la cobertura disminuye y el sistema no logra aprobar la evaluación completa (Moré, 2019).

d. F1-Score: Combina la precisión y el recall en una única métrica, proporcionando una medida equilibrada del rendimiento del modelo. Es la media armónica entre precisión y recall, dando igual importancia a ambas. El F1 score es útil cuando se desea mantener un equilibrio entre precisión y recall, considerando tanto los falsos positivos como los falsos negativos. Resulta especialmente valioso en situaciones donde precisión y recall tienen la misma relevancia, como en recuperación de información o clasificación de documentos (Moré, 2019).

e. Matriz de Confusión: La matriz de confusión es una herramienta fundamental para evaluar modelos de clasificación, ya que permite comparar las predicciones del modelo con los valores reales de la clase objetivo. Está organizada en una tabla donde se muestran cuatro resultados posibles: verdaderos positivos (VP), verdaderos negativos (VN), falsos positivos (FP) y falsos negativos (FN). A partir de estos valores se calculan métricas como precisión (precision), recall (sensibilidad), F1-score y exactitud (accuracy), que ayudan a determinar qué tan bien está clasificando el modelo (Moré, 2019).

Esta matriz es especialmente útil cuando las clases están desequilibradas, ya que permite identificar si el modelo está favoreciendo una categoría sobre la otra, algo común en estudios educativos donde los casos de reprobación suelen superar a los de aprobación. Por ello, se considera una herramienta esencial para evaluar modelos que predicen rendimiento académico o riesgo de fracaso estudiantil (Moré, 2019).

f. Curva de ROC (AUC-ROC): Receiver Operating Characteristic (ROC) es una herramienta gráfica utilizada para evaluar el desempeño de modelos de clasificación binaria, permite analizar la capacidad del modelo para diferenciar entre dos clases. Esta curva se construye representando la tasa de verdaderos positivos (recall) frente a la tasa de falsos positivos para distintos umbrales de decisión, lo que posibilita observar el comportamiento del modelo ante diferentes criterios de clasificación. A partir de esta curva se calcula el área bajo la curva (AUC-ROC), cuyo valor oscila entre 0 y 1, donde valores cercanos a 1 indican una mayor capacidad discriminatoria del modelo, mientras que valores próximos a 0,5 reflejan

un desempeño similar al azar. En este estudio, el uso del AUC-ROC resulta especialmente relevante debido al desbalance de clases existente, ya que permite evaluar el poder de discriminación del modelo de manera independiente del umbral de clasificación utilizado (Hand & Till, 2001).

2.5. Rendimiento Académico Estudiantil

El rendimiento académico estudiantil comprende su definición conceptual y el análisis de factores que influyen en el rendimiento académico.

2.5.1. Concepto de rendimiento académico

El rendimiento académico de los estudiantes es uno de los indicadores de mayor peso al abordar la calidad de la enseñanza superior; se relaciona con conceptos tales como excelencia académica y eficacia escolar. (Avendaño y otros, 2016)

También puede considerarse como el desarrollo, aplicación y puesta en práctica del conocimiento por parte de los estudiantes en contextos educativos, lo cual puede reflejarse a través de su forma de expresarse, sus actitudes y la vivencia de valores (García Ortiz y otros, 2014).

2.5.2. Factores que influyen en el rendimiento académico

Los principales factores que influyen en el rendimiento de los estudiantes se dividen en personales y sociales. En lo personal intervienen aspectos como el género del estudiante, su edad, los resultados en solicitudes de ingreso. En lo social destacan el estado civil, el lugar de procedencia, el nivel socioeconómico, las condiciones de empleo y el financiamiento de los estudios (Barreno Freire y otros, 2022).

2.6. Examen de ingreso en la FCyT

La prueba de conocimientos denominada, Prueba de Suficiencia Académica (PSA) se toma una vez se haya cumplido los requisitos de inscripción al mismo, posteriormente tomado el examen, este se califica sobre un total de 100 puntos, estableciendo como puntaje mínimo de aprobación el 51%. Los contenidos evaluados abarcarán las áreas de Aritmética-Álgebra, Geometría-Trigonometría, Física, Química y Biología, distribuyendo de manera equilibrada la cantidad de preguntas en cada una de estas materias (Universidad Mayor de San Simón- Facultad de Ciencias y Tecnología, 2020-2022).

2.7. Tecnologías y herramientas usadas

Para desarrollar el proyecto se utilizará las siguientes herramientas y sus características. Estas herramientas son seleccionadas por la facilidad de obtención y manejo para la exploración de datos y entrenamiento de modelos.

2.7.1. Google colab

Google Colaboratory (también conocido como Colab) es un servicio en la nube basado en Jupyter Notebooks para difundir la educación y la investigación del aprendizaje automático. Proporciona un tiempo de ejecución totalmente configurado para el aprendizaje profundo y el acceso gratuito a una GPU robusta (Carneiro y otros, 2018).

2.7.2. Python

Python es un lenguaje de programación interpretado de propósito general que se utiliza en una gran variedad de aplicaciones, desde el desarrollo web hasta el análisis de datos. Según Guido van Rossum, el creador de Python, tiene una sintaxis simple y sencilla; el tipado dinámico, su gestor de memoria, la gran cantidad de librerías disponibles y su potencia del lenguaje, entre otros, hacen que desarrollar una aplicación en Python sea accesible, rápido y entendido para los desarrolladores (Rodríguez Villanueva y otros, 2023).

2.7.3. Librerías

Entre las librerías más conocidas podemos mencionar a Pandas, Seaborn y Matplotlib.

La librería Pandas, es una librería que brinda herramientas para la manipulación de datos, a través de sus submódulos con funciones básicas que permiten leer, modificar y filtrar archivos tipo xlsx, csv, json y otros formatos de textos (Castillo Requíz y otros, 2023)

La librería de Matplotlib se usa en Python, está inspirada en MATLAB y se centra en el análisis, visualización y modelamiento de grandes conjuntos de datos, creando visualizaciones de datos estáticas, animadas e interactivas, como gráficos de líneas, dispersión y barras (Castillo Requíz y otros, 2023).

La librería Seaborn sirve para generar gráficos estadísticos de alto nivel que se integra directamente con la librería de Pandas. De esta forma Seaborn amplía la biblioteca Matplotlib para la creación de gráficos en Python (Castillo Requíz y otros, 2023).

Después de una limpieza de datos y los siguientes pasos del análisis exploratorio, se empieza a trabajar con las librerías Pandas, Seaborn y Matplotlib.

2.7.4. Power BI

Power BI es una herramienta de análisis y visualización de datos que permite conectar distintos orígenes de información, crear paneles interactivos e informes dinámicos, y compartir resultados fácilmente. Esta solución incluye Power BI Desktop, una aplicación para Windows donde se pueden diseñar modelos de datos, construir visualizaciones e informes; Power BI Service, un servicio en línea bajo el modelo SaaS (Software como Servicio); y las aplicaciones móviles de Power BI, disponibles para teléfonos y tabletas.

De esta manera, los usuarios pueden desarrollar y compartir informes, o incluso integrarlos en sitios web y aplicaciones. Además, Power BI no solo permite analizar lo que ocurrió en el pasado o lo que sucede en el presente, sino también realizar pronósticos sobre el futuro mediante técnicas de series temporales, como el suavizado exponencial, lo que facilita anticipar la demanda y planificar decisiones estratégicas.

Por otro lado, la herramienta Power Query permite crear consultas, limpiar y transformar los datos antes de cargarlos en Power BI Desktop, obteniendo así modelos refinados y listos para el análisis (Microsoft Power Platform, 2020).

3. Marco metodológico

3.1. Área de estudio

El área de estudio se encuentra en Cochabamba-Cercado, centro de Cochabamba en la calle Jordán esquina Oquendo, zona central de la ciudad de Cochabamba (ver figura 3-1), fue fundada el 5 de noviembre de 1832, mediante decreto del Mariscal Andrés de Santa Cruz, las coordenadas son aproximadamente:

17°23'47"S 66°09'21"O.



Figura 3-1: Ubicación del área

Fuente: Google Maps, 2025

3.2. Flujograma metodológico

La metodología seguida para la realización del proyecto se puede visualizar en el flujograma, el cual describe de manera secuencial las etapas de análisis exploratorio de datos, desarrollo y evaluación de modelos predictivos, así como la fase de visualización de resultados. (ver Figura 3-2).

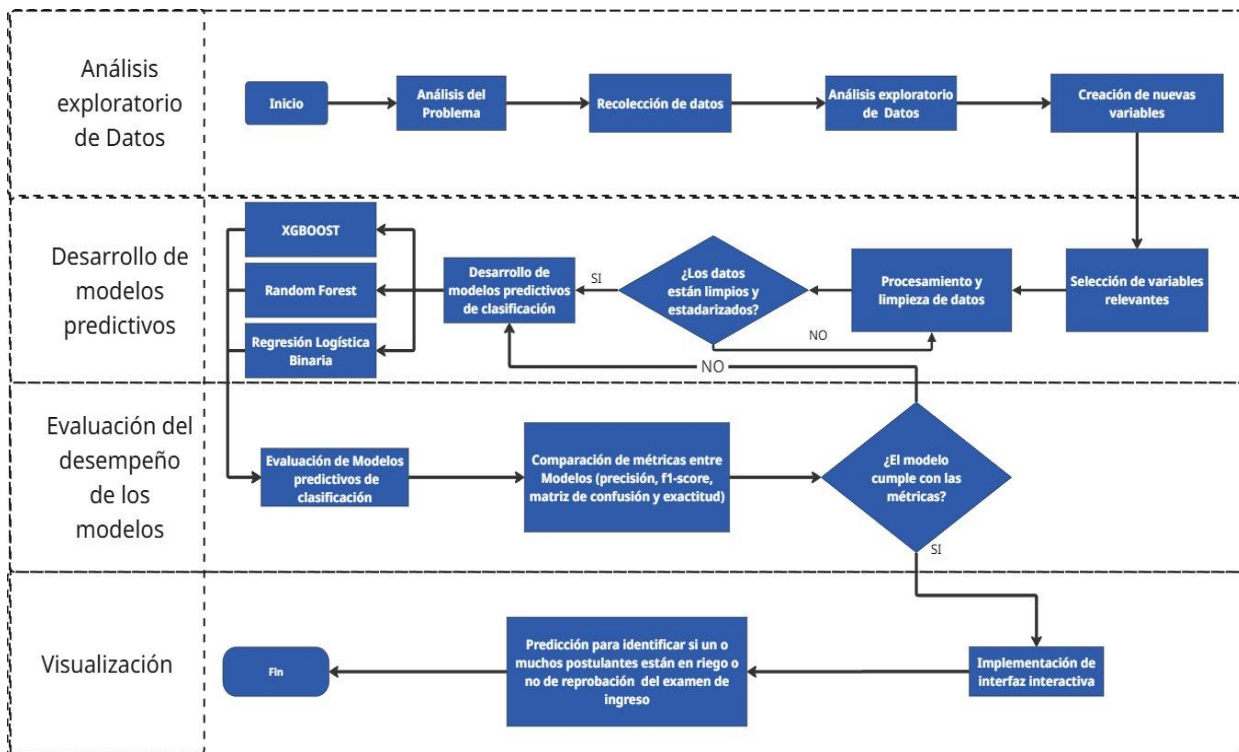


Figura 3-2: Flujograma metodológico

Fuente: *Elaboración propia, 2025*

La Figura 3-2 muestra el flujograma metodológico. El proceso inicia con el análisis del problema y la recolección de los datos históricos correspondientes al examen de ingreso de la FCyT-UMSS. Posteriormente, se realiza el análisis exploratorio de los datos, que tiene que ver con la creación de nuevas variables, con el fin de comprender la estructura del conjunto de datos e identificar patrones, tendencias y características relevantes del desempeño de los postulantes.

Seguidamente, se lleva a cabo la selección de variables relevantes que serán utilizadas en el desarrollo de los modelos predictivos de clasificación. Continuando, se procede al procesamiento y limpieza de los datos, etapa en la que se tratan valores faltantes y se estandarizan las variables. Durante el proceso de desarrollo de modelos, se verifica si los datos se encuentran correctamente limpios y estandarizados; en caso contrario, el flujo retorna a la etapa de procesamiento y limpieza de datos hasta cumplir con dichas condiciones. Finalmente, en esta fase se desarrollan modelos predictivos de clasificación, entre ellos XGBoost, Random Forest y regresión logística binaria.

Una vez desarrollados los modelos predictivos, se realiza la evaluación de su desempeño mediante métricas de evolución. Para ello, se calculan indicadores como la precisión, el recall, f1-score, la exactitud y la matriz de confusión, los cuales permiten medir la capacidad de los modelos para identificar postulantes con riesgo de reprobación del examen de ingreso. Posteriormente, se comparan las métricas obtenidas entre los distintos modelos con el propósito de seleccionar aquel que presenta el mejor

rendimiento. Si el modelo no cumple con las métricas establecidas, el proceso retorna a la etapa de desarrollo de modelos predictivos.

Para concluir, cuando el modelo seleccionado cumple con las métricas de evaluación, se procede con la implementación de la interfaz interactiva, para la predicción de postulantes que podrían estar en riesgo de reprobación el examen de ingreso.

3.3. Fuentes de información

3.3.1. Fuente de Información Secundaria

La fuente de información pertenece a las fuentes secundarias, ya que los datos fueron recabados en la UMSS durante los exámenes de ingreso a la FCyT. Estos datos incluyen información sobre características demográficas de los estudiantes, como edad, sexo, lugar de nacimiento y tipo de colegio, así como sus respuestas y calificaciones en las distintas opciones de examen. Al ser datos recolectados con fines administrativos y académicos, se consideran fiables y completos para el análisis que se desea realizar.

Al tratarse de información histórica previamente registrada, estos datos permiten realizar análisis exploratorios detallados y construir modelos predictivos sin necesidad de recopilar nueva información. Esto facilita el estudio de tendencias, la identificación de patrones relacionados con el rendimiento académico y el desarrollo de herramientas que apoyen la detección temprana de estudiantes con mayor probabilidad de reprobación el examen de ingreso.

Es importante resaltar que, aunque se trata de información secundaria, se aplicaron criterios de limpieza y depuración de datos para asegurar la consistencia y calidad de la información utilizada en el proyecto. De esta manera, se garantiza que los resultados obtenidos a partir de los modelos sean confiables y representativos del comportamiento de los estudiantes durante los exámenes de ingreso.

3.4. Análisis exploratorio de datos

3.4.1. Análisis del problema

La Universidad Mayor de San Simón (UMSS), a través de la Facultad de Ciencias y Tecnología (FCyT), tiene un proceso de admisión que se lleva a cabo mediante un examen de ingreso, cuyos resultados determinan el acceso a la formación universitaria. Con el fin de comprender mejor el comportamiento académico de los aspirantes, la FCyT dispuso los registros históricos de postulantes y sus resultados correspondientes a las gestiones 2006 a 2025.

El análisis de estos datos permite explorar diversos aspectos del proceso de admisión, tales como las características demográficas y académicas de los postulantes, los niveles de aprobación y las variables que podrían estar asociadas al éxito o fracaso en el examen. Si bien los registros permiten realizar evaluaciones

descriptivas, también se considera incorporar modelos predictivos que estimen de manera anticipada la probabilidad de aprobación o reprobación, como complemento al análisis tradicional y ampliar la comprensión del rendimiento de los postulantes.

En este contexto, surge la necesidad de aplicar técnicas de aprendizaje supervisado que permitan construir un modelo predictivo capaz de identificar perfiles de postulantes con mayor riesgo de reprobación. La implementación de este tipo de modelos no pretende sustituir los métodos actuales de evaluación, sino complementarlos mediante el uso de datos históricos, facilitando una comprensión más profunda de los factores asociados al rendimiento académico.

3.4.2. Recolección de datos

La recolección de los datos fue realizada por la institución. Los datos académicos y demográficos utilizados son el registro y resultado de exámenes de ingreso de postulantes a la FCyT, empleados para su respectivo procesamiento, análisis y construcción de modelo.

Los registros son presentados en formato de tablas en archivos excel. El archivo contiene datos demográficos, académicos y calificaciones obtenidas (ver Tabla 3-1). La primera fila representa el encabezado de las columnas, que representan las variables sobre las características de los postulantes y sus resultados en el examen rendido.

Para mantener la confidencialidad, los datos que pudieran identificar a los postulantes no fueron entregados en el Dataset proporcionado.

Tabla 3-1: Variables demográficas y académicas del conjunto de datos analizado

	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1	DESCRIPCIÓN	C. INGRESO	NOMBRE_PLAN	FECHA_NAC	SEXO	ESTADO_CIVIL	CIUDAD_NAC	PAZ	CIUDAD_NAC	CIUDAD_NAC	CIUDAD_NAC	CIUDAD_NAC	NOMBRE_COLEGIO	CIUDAD_COLEGIO	PO_COLEGIO	CIUDAD_COLEGIO	CIUDAD_COLEGIO
2	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1987-09-25 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	FRANCISCO VIGNAUD	COCHABAMBA	FISCAL URBANO	ABA	
3	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1985-09-06 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	MEJILLONES	COCHABAMBA	FISCAL URBANO	21	REP
4	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1987-05-24 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	INCOS	BENI	FISCAL URBANO	31	REP
5	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1984-12-22 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	FE Y ALEGRIA	COCHABAMBA	FISCAL URBANO	11	REP
6	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1985-02-19 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	NORDICO BOLIVIANO	COCHABAMBA	FISCAL URBANO	21	REP
7	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1988-09-28 00:00:00	FEMENINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	FRANCISCO GONZALES PRADA	COCHABAMBA	FISCAL URBANO	31	REP
8	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1988-09-18 00:00:00	FEMENINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	MARIA AUXILIADORA	COCHABAMBA	PARTICULAR	51	APR
9	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1987-09-29 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	8 DE MAYO RENE BARRIENTOS	COCHABAMBA	FISCAL URBANO	11	REP
10	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1988-04-12 00:00:00	FEMENINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	ANTONIO QUIJARRO	POTOSI	FISCAL URBANO	51	APR
11	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1985-09-13 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	KHOCHAMPAMPA	COCHABAMBA	PARTICULAR	26	REP
12	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1986-11-14 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	SAN MARCOS	COCHABAMBA	FISCAL URBANO	26	REP
13	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1986-01-17 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	COCHABAMBA	COCHABAMBA	FISCAL URBANO	21	REP
14	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1988-07-21 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	PUERTO DE MEJILLONES/TECNOL	COCHABAMBA	FISCAL URBANO	26	REP
15	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1985-12-09 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	EUCALIPTUS	ORURO	FISCAL URBANO	26	REP
16	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1986-10-09 00:00:00	FEMENINO	SOLTERO(A)	BO	BOLIVIA	LA PAZ	LOS ANDES	NO	2005	COPACABANA	LA PAZ	FISCAL URBANO	61	REP
17	EXAMEN DE INGRESO	1	LICENCIATURA EN INGENIERIA	1986-09-25 00:00:00	MASCULINO	SOLTERO(A)	BO	BOLIVIA	COCHABAMBA	COCHABAMBA	NO	0	ALCIDES ARGUEDAS	COCHABAMBA	FISCAL URBANO	21	REP

Fuente: *Elaboración FCyT, 2025*

La Tabla 3-1 muestra una vista parcial del conjunto de datos utilizados en este proyecto, correspondiente a los registros históricos del examen de ingreso de la FCyT-UMSS. Debido a la gran cantidad de variables que conforman el dataset completo, en la imagen se presentan únicamente algunas de ellas con fines ilustrativos, para mostrar la estructura general de la información disponible (ver Anexo 1).

Para comprender de manera adecuada los datos académicos y demográficos de los postulantes, se elaboró una descripción de los encabezados de los archivos utilizada en la Tabla 3-2. En ella se detalla el tipo de dato (numérico o cadena de caracteres), la clasificación de la variable (categórica o cuantitativa) y una descripción que resumen la información contenida en cada columna.

Tabla 3-2: Descripción de Variables datos de estudiantes

Campo	Tipo	Variable	Descripción
ANIO	Numérico	Categórica	Representa el año al que pertenecen las calificaciones registradas
PERIODO	Numérico	Categórica	Representa al semestre del año que pertenece las calificaciones registradas
DESCRIPCIÓN	Cadena de Caracteres	Categórica	Representa la forma de ingreso a la Universidad
OPC_INGRESO	Numérico	Categórica	Representa al Numero de Opción del examen que se presentó el postulante
COD_PLAN	Numérico	Identificador	Representa el código de la Carrera a la que el postulante quiere ingresar
NOMBRE_PLAN	Cadena de Caracteres	Categórica	Representa el nombre completo de la carrera a la cual el postulante quiere ingresar
FECHA_NAC	Fecha	Categórico	Representa el día, mes y año de nacimiento del postulante
SEXO	Cadena de Caracteres	Categórico	Representa el género del postulante, puede ser Masculino M o Femenino F
ESTADO_CIVIL	Cadena de Caracteres	Categórico	Representa el estado civil del postulante, puede ser Soltero(a) o Casado(a)
COD_NACIONALIDAD	Cadena de Caracteres	Identificador	Código que representa el país de nacionalidad del postulante
NACIONALIDAD	Cadena de Caracteres	Categórico	Representa el lugar de origen del postulante
ID_CIUDAD	Numérico	Identificador	Código que representa la ciudad del postulante
CIUDAD_NACIMIENTO	Cadena de Caracteres	Categórico	Representa la ciudad de origen del postulante
ID_PROVINCIA	Numérico	Identificador	Código que representa la provincia del postulante
PROVINCIA_NACIMIENTO	Cadena de Caracteres	Categórico	Representa la provincia de origen del postulante
COD_MUNICIPIO	Numérico	Identificador	Código que representa el municipio del postulante
MUNICIPIO	Cadena de Caracteres	Categórico	Representa el municipio de origen del postulante

TRABAJA	Cadena de Caracteres	Categorico	Indica si el postulante trabaja o no con SI o No como respuesta
ANIO_BACHILLERATO	Número	Categorico	Representa la gestión en la que el postulante se graduó del colegio
NOMBRE_COLEGIO	Cadena de Caracteres	Categorico	Representa el nombre del colegio del cual el postulante se graduó
PROVINCIA_COLEGIO	Cadena de Caracteres	Categorico	Representa la provincia del colegio del cual el postulante vino
CIUDAD_COLEGIO	Cadena de Caracteres	Categorico	Representa la ciudad del colegio del cual el postulante vino
COD_TIPO_COL	Número	Identificador	Código que representa el tipo de colegio del cual el postulante vino
TIPO_COLEGIO	Cadena de Caracteres	Categorico	Representa los tipos de colegio del cual los postulantes pueden venir
PROM_FINAL	Número	Cuantitativo	Representa la calificación que obtuvo el postulante en el examen de admisión a la FCyT
RESULTADO_FINAL	Número	Categorico	Es el estado en el cual se encuentra el postulante posterior al examen de admisión

Fuente: *Elaboración propia, 2025*

Para comprender la información proporcionada, se usó Google Colab como entorno de trabajo, junto con las librerías Pandas, NumPy, Matplotlib y Seaborn. Estas herramientas permiten realizar una exploración inicial del conjunto de datos, facilitando la lectura, limpieza de la información, así como la identificación de valores nulos e inconsistencias presentes en las variables.

3.4.3. Análisis exploratorio de datos

Se realizó una exploración inicial del conjunto de datos con el objetivo de revisar su estructura general y evaluar la calidad de la información disponible. Se analizaron aspectos como el número de registros, el tipo de variables (numéricas y categóricas) y la presencia de valores faltantes o inconsistencias en los datos. A continuación se presenta un resumen de la cantidad de registros (ver Figura 3-3).

```
df_postulantes.shape
(98009, 26)
```

Figura 3-3: Cantidad de filas y columnas en los datos

Fuente: *Elaboración propia, 2025*

Mediante la función `shape` de python se muestra una tupla con dos valores dentro, el primero representa las filas y el segundo las columnas en total.

También se hizo una revisión de los datos para ver si el tipo de variable concuerda con el que debería mediante la función `info()` de python (ver Figura 3-4).

```

RangeIndex: 98009 entries, 0 to 98008
Data columns (total 26 columns):
#   Column              Non-Null Count  Dtype   13  ID_PROVINCIA        98009 non-null  int64
---  ---
0   ANIO                 98009 non-null  int64   14  PROVINCIA_NACIMIENTO 98009 non-null  object
1   PERIODO              98009 non-null  int64   15  COD_MUNICIPIO        49968 non-null  float64
2   DESCRIPCIÓN          98009 non-null  object   16  MUNICIPIO            49968 non-null  object
3   OPC_INGRESO          98009 non-null  int64   17  TRABAJA              98009 non-null  object
4   COD_PLAN             98009 non-null  int64   18  ANIO_BACHILLERATO    98009 non-null  int64
5   NOMBRE_PLAN          98009 non-null  object   19  NOMBRE_COLEGIO       98009 non-null  object
6   FECHA_NAC            98009 non-null  object   20  PROVINCIA_COLEGIO    98009 non-null  object
7   SEXO                 98009 non-null  object   21  CIUDAD_COLEGIO       98009 non-null  object
8   ESTADO_CIVIL        98009 non-null  object   22  COD_TIPO_COL         98009 non-null  int64
9   COD_NACIONALIDAD    98009 non-null  object   23  TIPO_COLEGIO         98009 non-null  object
10  NACIONALIDAD         98009 non-null  object   24  PROM_FINAL           96639 non-null  float64
11  ID_CIUADAD           98009 non-null  int64   25  RESULTADO_FINAL      98009 non-null  object
12  CIUDAD_NACIMIENTO    98009 non-null  object
dtypes: float64(2), int64(8), object(16)
memory usage: 19.4+ MB

```

Figura 3-4: Información de las variables del dataset

Fuente: *Elaboración propia, 2025*

La Figura 3-4 se muestra un listado general del conjunto de datos utilizado en el proyecto, el cual está compuesto por variables de tipo numérico (int 64 o float 64) y categórico(object). Se observa que la mayoría de las variables no presenta valores faltantes; sin embargo, algunas columnas, como COD_MUNICIPIO, MUNICIPIO y PROM_FINAL, contienen valores nulos, los cuales fueron considerados en la etapa de limpieza y preparación de los datos.

A continuación, se presenta la Tabla 3-3, la cual muestra el resumen estadístico de las variables numéricas obtenido mediante la función `describe()` de Python.

Tabla 3-3: Tabla estadística de los datos

	ANIO	PERIODO	OPC_INGRESO	COD_PLAN	ID_CIUADAD	ID_PROVINCIA	COD_MUNICIPIO	ANIO_BACHILLERATO	COD_TIPO_COL	PROM_FINAL
count	98009.000000	98009.000000	98009.000000	98009.000000	98009.000000	98009.000000	49968.000000	98009.000000	98009.000000	96639.000000
mean	2016.729668	1.257405	1.622902	381919.105929	3.203563	2.461794	1.315482	1970.959269	1.807436	38.913151
std	4.882355	0.437207	0.698014	119913.395600	1.064754	3.043637	0.961886	295.287516	0.394315	15.598067
min	2006.000000	1.000000	1.000000	114071.000000	0.000000	0.000000	0.000000	0.000000	1.000000	1.000000
25%	2013.000000	1.000000	1.000000	319801.000000	3.000000	1.000000	1.000000	2011.000000	2.000000	26.000000
50%	2017.000000	1.000000	1.000000	339701.000000	3.000000	1.000000	1.000000	2016.000000	2.000000	36.000000
75%	2020.000000	2.000000	2.000000	419701.000000	3.000000	2.000000	1.000000	2019.000000	2.000000	47.000000
max	2025.000000	2.000000	3.000000	760101.000000	9.000000	19.000000	8.000000	2024.000000	2.000000	100.000000

Fuente: *Elaboración propia, 2025*

La Figura 3-5 muestra las principales medidas descriptivas calculadas para cada característica numérica del dataset. El indicador `count` refleja la cantidad de valores no nulos por columna, permitiendo identificar variables con información incompleta, como COD_MUNICIPIO, que presenta

aproximadamente la mitad de registros disponibles. Los valores de mean y std muestran el nivel de dispersión y las diferencias de escala entre atributos como EDAD, OPCIÓN DE INGRESO y AÑO DE BACHILLERATO. Asimismo, los estadísticos min y max permitieron verificar que no existieran datos fuera de los dominios esperados y apoyar la selección de características utilizadas posteriormente en los modelos de clustering y clasificación.

3.4.4. Creación de nuevas variables

Como parte del análisis exploratorio de datos y de la etapa de preparación para el modelo, se realiza un proceso de creación de variables, en el cual se genera nuevas variables derivadas a partir de la información original disponible en el conjunto de datos, obteniendo así variables demográficas y académicas que no se estaban explícitamente definidos en las variables originales. La creación de estas variables permitió transformar información implícita en indicadores más interpretables.

En la Tabla 3-4 se presenta un listado de las variables creadas, especificando para cada una de ellas: La variable creada, las variables de origen utilizadas, su descripción conceptual, el tipo de dato resultante y la operación aplicada para su construcción.

Tabla 3-4: Descripción de variables creadas

Variable creada	Variables de origen	Descripción	Tipo de Dato	Operación aplicada
EDAD	FECHA_NAC, ANIO	Edad del postulante al rendir el examen	N Numérico	Diferencia entre el ANIO y FECHA_NAC
ANIOS_POST_BATCH	ANIO_BACHILLERATO, ANIO	Años transcurridos desde el bachillerato	N Numérico	Diferencia entre ANIO y ANIO_BACHILLERATO
TRABAJO_COLEGIO	TRABAJA, TIPO_COLEGIO	Condición de trabajo durante etapa escolar	C Categórico	Combinación de TRABAJA y TIPO_COLEGIO
MIGRA_UNIVERSIDAD	CIUDAD_NACIMIENTO, PROVINCIA_COLEGIO	Identificación de migración para rendir el examen	C Categórico	Condicional si CIUDAD_COLEGIO es distinto a “COCHABAMBA” o PROVINCIA_COLEGIO es distinto a “COCHABAMBA(CERCADO)”, entonces el postulante migra
MAYOR_EDAD	EDAD	Clasificación según mayoría de edad	C Categórico	Condicional si EDAD es mayor o igual a 18, el postulante es mayor de edad
TASA_APR_COLEGIO	RESULTADO_FINAL, NOMBRE_COLEGIO	Porcentaje de aprobación por colegio	N Numérico	Calcula el promedio de NOMBRE_COLEGIO, cuyo estado de RESULTADO_FINAL sea igual a “APR”

Fuente: *Elaboración propia, 2025*

En este proceso se incorporaron variables relacionadas con la edad del postulante, el tiempo transcurrido desde la finalización del bachillerato, la condición laboral durante la etapa escolar, la migración para rendir el examen de ingreso, la mayoría de edad y la tasa de aprobación por unidad educativa. El detalle del procedimiento de creación de las variables, así como el código utilizado para su implementación, se incluye en los Anexos (ver Anexo 2).

3.5. Desarrollo de modelos predictivos

3.5.1. Selección de variables para el modelado

Para seleccionar las variables se tomó en cuenta aquellas que serían más utilizadas en el proceso de modelado predictivo. A partir del conjunto de datos, se definió un subconjunto de variables relevantes considerando criterios teóricos, resultados del análisis exploratorio y la disponibilidad de información.

Las variables seleccionadas incluyen atributos demográficos, académicos y variables derivadas creadas previamente (ver Figura 3-5), conformando el conjunto de datos denominado `df_modelo`, el cual fue utilizado para el entrenamiento y evaluación de los modelos de aprendizaje automático.

```
df_modelo.columns

Index(['PERIODO', 'SEXO', 'EDAD', 'OPC_INGRESO', 'CIUDAD_COLEGIO',
      'PROVINCIA_COLEGIO', 'ANIO_BACHILLERATO', 'MUNICIPIO', 'NACIONALIDAD',
      'ESTADO_CIVIL', 'RESULTADO_FINAL', 'ANIOS_POST_BACH',
      'TASA_APR_COLEGIO', 'TRABAJO_COLEGIO', 'MAYOR_EDAD',
      'MIGRA_UNIVERSIDAD'],
      dtype='object')
```

Figura 3-5: Columnas seleccionas para el modelo

Fuente: *Elaboración propia, 2025*

A través del `columns` en Python se obtiene el listado de todas las variables seleccionadas, con un total de 17 columnas que conforman a la variable `df_modelo`, siendo este el data set seleccionado para los modelos predictivos.

3.5.2. Procesamiento y limpieza de los datos

A continuación, se describen las actividades realizadas para preparar el conjunto de datos antes del entrenamiento de los modelos predictivos, asegurando la calidad de la información y evitando fallos durante el proceso de modelado. Estas tareas incluyeron la selección de variables relevantes, el tratamiento de valores faltantes y la verificación de la consistencia de los tipos de datos. Asimismo, se aplicaron procedimientos de depuración para garantizar que el dataset final cumpliera con los requerimientos de las herramientas de minería y aprendizaje automático.

3.5.2.1. Limpieza de datos y tratamiento de valores atípicos

En primer lugar, se analizó la variable EDAD como criterio de control de calidad, aun cuando esta no fue incluida directamente como variable explicativa en el modelado. Se identificó que los valores de edad variaban entre 0 y 70 años (ver Figura 3-6).

	EDAD	Cantidad		EDAD	Cantidad
0	0	2	21	33	19
1	8	2	22	34	35
2	14	8	23	35	24
3	15	53	24	36	12
4	16	1944	25	37	12
5	17	27937	26	38	21
6	18	33163	27	39	5
7	19	16430	28	40	14
8	20	8276	29	41	4
9	21	4287	30	42	4
10	22	2343	31	43	4
11	23	1334	32	44	1
12	24	758	33	45	3
13	25	441	34	46	1
14	26	263	35	47	1
15	27	202	36	48	1
16	28	124	37	50	2
17	29	107	38	54	1
18	30	79	39	55	2
19	31	52	40	68	2
20	32	35	41	70	1

Figura 3-6: Tabla de la edad y la cantidad que existe

Fuente: *Elaboración propia, 2025*

Se puede observar que no hay valores tan significativos antes de 15 y después de 43 por lo que se estableció un rango válido comprendido entre 15 y 43 años, excluyendo aquellos registros que están fuera del intervalo. Posteriormente, la variable EDAD fue eliminada del conjunto de datos utilizado para el modelado, conservándose únicamente la variable derivada MAYOR_EDAD.

Asimismo, se realizó una revisión de consistencia sobre la variable ANIO_BACHILLERATO, detectándose registros con valores iguales a cero, los cuales no representan un año válido de finalización del bachillerato. Estos registros fueron eliminados del conjunto de datos, manteniéndose únicamente aquellos con valores distintos de cero o nulo.

Adicionalmente, se identificaron valores faltantes en la variable MUNICIPIO, por lo que se procedió a eliminar dichos registros. Finalmente, se filtró la variable RESULTADO_FINAL, considerando únicamente las categorías APR y REP, descartando registros con valores distintos, con el fin de asegurar la coherencia de la variable objetivo utilizada en el proceso de clasificación (ver Tabla 3-5).

Este proceso es necesario, para un mejor aprendizaje del modelo, pues si hay inconsistencias como nulos o datos en cero, se tendrá un mal resultado.

El detalle del código empleado para la limpieza y filtrado de los datos se presenta en los Anexos (ver Anexo 3).

Tabla 3-5: Tratamiento de inconsistencias y valores atípicos en las variables

Variable	Problema Identificado	Criterio aplicado	Acción
EDAD	Valores extremos (0–70)	Rango 15–43	Filtrado y Eliminación
ANIO_BACHILLERATO	Valores nulos	Año bachillerato > 0	Eliminación
MUNICIPIO	Valores nulos	Eliminar nulos	Eliminación
RESULTADO_FINAL	Valores distintos de APR/REP	Solo APR y REP	Filtrado

Fuente: *Elaboración propia, 2025*

3.5.2.2. Asignación de variables dependiente e independientes

Para realizar el modelo predictivo es necesario asignar valores en X e Y, siendo Y la variable objetivo y en X las variables independientes (ver Figura 3-7).

```
# Asignación X e y
X = df_modelo.drop(columns=["RESULTADO_FINAL"])
y = df_modelo["RESULTADO_FINAL"]
```

Figura 3-7: Captura de código de la asignación de X e Y

Fuente: *Elaboración propia, 2025*

Las variables independientes X estuvieron conformadas por el conjunto de variables seleccionadas previamente en df_modelo, excluyendo la variable objetivo. Esta asignación adapta los datos para el entrenamiento de los modelos predictivos, asegurando una correcta diferenciación entre las características explicativas y la variable a predecir.

Se asigna la variable dependiente Y que corresponde a la variable RESULTADO_FINAL, la cual representa el resultado obtenido por el postulante en el examen de ingreso, clasificado en las categorías APR (aprobado) y REP (reprobado).

3.5.2.3. División del conjunto en entrenamiento y prueba

Posteriormente, el conjunto de datos fue dividido en subconjuntos de entrenamiento y prueba, utilizando el 65% de los registros para el entrenamiento y el 35% restante para la evaluación del desempeño de los modelos. La división se realizó considerando la variable objetivo, con el fin de preservar la proporción de clases en ambos subconjuntos (ver Figura 3-8).

Asimismo, se utilizó una semilla aleatoria con valor 42 que va asignado en el random_state, para garantizar la reproducibilidad del proceso.

```
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.35,
    random_state=42,
    stratify=y
)
```

Figura 3-8: Captura de código de la división de entrenamiento(train) y prueba(test)

Fuente: *Elaboración propia, 2025*

3.5.2.4. Codificación de variables categóricas y escalado de variables numéricas

Para la correcta utilización de las variables por los algoritmos de aprendizaje automático, se identificaron las variables categóricas y numéricas del conjunto de datos (ver Figura 3-9).

```
# | Preprocesador
preprocesador = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_features),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_features)
    ]
)
```

Figura 3-9: Captura de código de la escalación y one hot encoding

Fuente: *Elaboración propia, 2025*

Las variables numéricas fueron sometidas a un proceso de estandarización utilizando el método StandardScaler, mientras que las variables categóricas fueron codificadas mediante One Hot Encoding. Ambos procesos fueron integrados dentro de un esquema de preprocesamiento implementado mediante un ColumnTransformer en Python.

El preprocesamiento fue incorporado dentro de un pipeline de modelado, para que las transformaciones se apliquen al conjunto de entrenamiento y prueba con consistencia. Debido al proceso interno del pipeline durante el entrenamiento del modelo, no se genera una tabla final fija con las variables transformadas, sino que estas son procesadas durante la ejecución del pipeline (ver Anexo 4).

3.5.3. Desarrollo de modelos predictivos de clasificación

3.5.3.1. Desarrollo del modelo con XGBoost

El modelo predictivo fue implementado utilizando el algoritmo XGBoost, configurado con un conjunto de hiperparámetros definidos para el presente estudio. Entre los principales parámetros utilizados se encuentran el número de estimadores, la profundidad máxima de los árboles, la tasa de aprendizaje y parámetros de regularización. Estos valores permitieron controlar la complejidad del modelo y reducir el riesgo de sobreajuste (ver Figura 3-10).

```
# Modelo XGBoost
xgb_modelo = XGBClassifier(
    n_estimators=800,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.7,
    colsample_bytree=0.7,
    eval_metric="logloss",
    random_state=42,
    reg_alpha=2,
    reg_lambda=3
)
```

Figura 3-10: Configuración de XGBClassifier

Fuente: *Elaboración propia, 2025*

Para la aplicación del algoritmo XGBoost se crea la variable `xgb_modelo` para el modelo predictivo, donde se utiliza el parámetro `n_estimators` igual a 800 árboles de decisión, la profundidad máxima de los árboles se estableció en `max_depth` igual a 5 niveles, sin estructuras complejas, seguido de la tasa de aprendizaje en el parámetro `learning_rate` de 0.05, reduciendo riesgo de sobreajuste. También, se aplicó un muestreo del 70% de los registros (`subsample = 0.7`) en cada iteración, así como un muestreo del 70% de las variables predictoras (`colsample_bytree = 0.7`), seguido parámetros de regularización en el parámetro `reg_alpha = 2` y `reg_lambda = 3`, ayudando a estabilizar los coeficientes del modelo.

Como métrica de evaluación se empleó la función de pérdida logarítmica (`logloss`) esto en el parámetro `eval_metric`, para evaluar la calidad de las probabilidades estimadas por el modelo y se fijó una semilla aleatoria (`random_state = 42`) para reproducir los resultados obtenidos.

a. Balanceo de clases

Dado el desbalance entre las clases de la variable objetivo, se aplica submuestreo aleatorio (`undersampling`) equilibrando postulantes aprobados y reprobados. Fue integrado dentro de un pipeline, haciendo que el balanceo se aplique únicamente sobre el conjunto de entrenamiento y evitando posibles pérdidas de información (ver Figura 3-11).

```
# Undersampling
undersampler = RandomUnderSampler(
    sampling_strategy=1.0, # Balancea 1:1 APR vs REP
    random_state=42
)

pipeline = ImbPipeline(steps=[
    ("preproceso", preprocesador),
    ("undersample", undersampler),
    ("modelo", xgb_modelo)
])
```

Figura 3-11: Función `RandomUnderSampler` y el pipeline

Fuente: *Elaboración propia, 2025*

Para la aplicación del submuestreo, se utilizó el muestreo 1:1, en el parámetro `sampling_strategy = 1.0`, para que ambas clases tengan la misma representación en el proceso de entrenamiento del modelo, lo que ayuda a reducir el desbalance hacia la clase mayoritaria y a mejorar su capacidad para identificar correctamente los casos de la clase minoritaria. Además, el uso de una semilla aleatoria fija en el parámetro `random_state = 42`, que asegura la reproducibilidad de los resultados obtenidos.

Asimismo, se implementó un pipeline de modelado, el cual integra de manera secuencial las etapas de preprocesamiento de los datos (descrito en la sección 3.4.9), y el balanceo de clases y el entrenamiento del modelo XGBoost. Esta estructura asegura que el proceso de submuestreo se aplique sobre los datos de entrenamiento en cada iteración evitando pérdidas.

b. Entrenamiento y generación de predicciones

Una vez definido el pipeline de preprocesamiento, balanceo y modelado, se procede al entrenamiento del modelo utilizando el conjunto de datos de entrenamiento. Posteriormente, se generan predicciones para los datos de entrenamiento y prueba, así como las probabilidades asociadas a cada predicción (ver Figura 3-11).

```
# Entrenamiento del modelo
pipeline.fit(X_train, y_train)

# Predicciones

y_train_pred = pipeline.predict(X_train)
y_train_prob = pipeline.predict_proba(X_train)[:, 1]

y_test_pred = pipeline.predict(X_test)
y_test_prob = pipeline.predict_proba(X_test)[:, 1]
```

Figura 3-11: Entrenamiento y predicciones

Fuente: *Elaboración propia, 2025*

Para el entrenamiento del modelo se utiliza `pipeline.fit(...)` ejecutando todas las etapas definidas dentro del pipeline sobre el conjunto de datos de entrenamiento con las variables predictoras (`X_train`) y la variable objetivo (`y_train`).

Una vez entrenado el modelo, se genera las predicciones de clase mediante la función `predict()` que genera la predicción final de clase y las probabilidades asociadas para los conjuntos de entrenamiento (`y_train_pred` y `y_train_prob`) y de prueba (`y_test_pred` y `y_test_prob`). Se utilizaron las predicciones del modelo sobre el conjunto de entrenamiento (`y_train_pred`) para evaluar el modelo durante el proceso de aprendizaje, mientras que las probabilidades estimadas (`y_train_prob`) permitieron analizar el nivel de confianza del modelo al asignar la clase positiva (APR) a cada observación.

3.5.3.2. Desarrollo del modelo con Random forest

La implementación del modelo Random Forest utilizó el algoritmo RandomForestClassifier, configurando un conjunto de hiperparámetros definidos para modelo predictivo como el número de estimadores y la profundidad máxima de los árboles. También, se definieron criterios mínimos para la división de nodos y para la cantidad de muestras en nodos hoja (ver Figura 3-12).

```
from sklearn.ensemble import RandomForestClassifier

rf_modelo = RandomForestClassifier(
    n_estimators=800,
    max_depth=5,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)
```

Figura 3-12: Configuración del modelo Random Forest

Fuente: *Elaboración propia, 2025*

Para el modelo predictivo denominado rf_modelo se utiliza, el parámetro n_estimators igual a 800, para un modelo robusto y reducir la varianza del modelo. La profundidad máxima de los árboles en max_depth igual a 5, limitando el crecimiento de cada árbol. También, se definió un número mínimo de muestras requerido para dividir un nodo en min_samples_split igual a 5, que restringe la generación de particiones poco representativas y se estableció un número mínimo de muestras en nodos hoja en min_samples_leaf igual a 2, asegurando que cada hoja contenga una cantidad suficiente de observaciones. Finalmente, se fijó una semilla aleatoria (random_state = 42) que permite reproducir los resultados obtenidos, y se habilitó el uso de procesamiento en paralelo (n_jobs = -1), para aprovechar todos los núcleos disponibles del sistema y reducir el tiempo de entrenamiento del modelo.

a. Integración del preprocesamiento y balanceo de clases

Con el objetivo de asegurar un tratamiento adecuado de las variables y tratar el desbalance presente en la variable objetivo, el modelo Random Forest fue integrado dentro de un pipeline que incluye las etapas de preprocesamiento y balanceo de clases. El preprocesamiento consideró la estandarización de las variables numéricas y la codificación de las variables categóricas, mientras que el balanceo se realizó mediante la técnica de undersampling, con el fin de equilibrar la proporción entre postulantes aprobados y reprobados.

La implementación del pipeline permitió aplicar estas transformaciones de forma secuencial y controlada, garantizando que el balanceo se realice únicamente sobre el conjunto de entrenamiento y evitando posibles fugas de información (ver Figura 3-13).

```
undersampler = RandomUnderSampler(
    sampling_strategy=1.0, # Balancea 1:1
    random_state=42
)
#Preprocesamiento y Pipeline
pipeline_rf = Pipeline(steps=[
    ("preprocesamiento", preprocesador),
    ("undersampling", undersampler),
    ("modelo", rf_modelo)
])
```

Figura 3-13: Pipeline de preprocesamiento, balanceo y modelado con Random Forest

Fuente: *Elaboración propia, 2025*

Al igual que XGBoost es necesario tratar el desbalance existente entre las clases de la variable objetivo, usando el submuestreo aleatorio (Random UnderSampling) que equilibra la proporción entre las clases, en el parámetro `sampling_strategy` igual a 1.0, balanceando 1:1. Reduciendo el desbalance a la clase mayoritaria y mejorar la identificación correcta en ambas clases. Asimismo, se utilizó una semilla aleatoria fija (`random_state = 42`) para reproducir los resultados obtenidos. También se construye un pipeline de procesamiento y modelado, que integra las etapas de preprocesamiento de los datos, el balanceo de clases y el entrenamiento del modelo. La instrucción `pipeline_rf = Pipeline(steps=[ ... ])` se utiliza para construir todas las etapas necesarias para el entrenamiento y la evaluación del modelo. En este pipeline se definen tres fases principales: el preprocesamiento, undersampling y modelo.

b. Entrenamiento del modelo y Predicción

Una vez definido el pipeline completo, se entrena el modelo Random Forest con el conjunto de datos de entrenamiento. Este proceso ajusta el modelo a partir de los datos históricos, manteniendo la coherencia con el modelo XGBoost implementado anteriormente. Se generaron las predicciones correspondientes al conjunto de entrenamiento y de prueba (ver Figura 3-14).

```
#Entrenamiento
pipeline_rf.fit(X_train, y_train)
#Predicción
y_train_pred_rf = pipeline_rf.predict(X_train)
y_train_prob_rf = pipeline_rf.predict_proba(X_train)[: , 1]

y_test_pred_rf = pipeline_rf.predict(X_test)
y_test_prob_rf = pipeline_rf.predict_proba(X_test)[: , 1]
```

Figura 3-14: Entrenamiento y predicciones

Fuente: *Elaboración propia, 2025*

Para el entrenamiento del modelo se utiliza `pipeline_rf.fit(...)` ejecutando todas las etapas definidas dentro del pipeline sobre el conjunto de datos de entrenamiento con las variables predictoras (`X_train`) y la variable objetivo (`y_train`).

Una vez entrenado el modelo, se genera las predicciones de clase mediante la función `predict()` que se utiliza para generar la predicción final de clase y las probabilidades asociadas para los conjuntos de entrenamiento (`y_train_pred_rf` y `y_train_prob_rf`) y de prueba (`y_test_pred_rf` y `y_test_prob_rf`). Se utilizaron las predicciones del modelo sobre el conjunto de entrenamiento (`y_train_pred_rf`) para evaluar el modelo durante el proceso de aprendizaje, mientras que las probabilidades estimadas (`y_train_prob_rf`) permitieron analizar el nivel de confianza del modelo al asignar la clase positiva (APR) a cada observación.

3.5.3.3. Desarrollo del modelo con Regresión Logística Binaria

Para la implementación del tercer modelo predictivo se utilizó el algoritmo de Regresión Logística binaria, considerando su uso como modelo de referencia para la clasificación (ver Figura 3-15).

```
from sklearn.linear_model import LogisticRegression

logreg_modelo = LogisticRegression(
    penalty="l2",
    solver="liblinear",
    max_iter=500,
    random_state=42
)
```

Figura 3-15: Configuración del modelo de Regresión Logística

Fuente: *Elaboración propia, 2025*

Este modelo fue configurado mediante un conjunto de hiperparámetros definidos, para obtener un modelo predictivo con adecuada capacidad de generalización.

Se utilizó regularización de tipo L2 (`penalty = "l2"`), que permite controlar valores grandes y reduciendo el riesgo de sobreajuste. El `solver = liblinear` para resolver el problema de clasificación binaria de manera eficiente fue seleccionado por su eficiencia en conjuntos de datos de tamaño pequeño a mediano.

Asimismo, se estableció un número máximo de iteraciones en `max_iter` igual a 500, permitiendo la convergencia del algoritmo durante el proceso de optimización. Finalmente, se fijó una semilla aleatoria (`random_state = 42`) para reproducir los resultados obtenidos.

a. Integración del preprocesamiento y balanceo de clases

Al igual que en los modelos previamente implementados, la regresión logística fue integrada dentro de un pipeline que incluye las etapas de preprocesamiento y balanceo de clases. La implementación del pipeline permitió asegurar que todas las transformaciones se apliquen únicamente sobre el conjunto de entrenamiento, evitando fugas de información y manteniendo la coherencia metodológica con los modelos XGBoost y Random Forest (ver Figura 3-16).

```

# Undersampling
undersampler = RandomUnderSampler(
    sampling_strategy=1.0, # Balancea 1:1
    random_state=42
)
pipeline_logreg = Pipeline(steps=[
    ("preprocesamiento", preprocesador),
    ("undersampling", undersampler),
    ("modelo", logreg_modelo)
])

```

Figura 3-16: Pipeline de preprocesamiento, balanceo y modelado con Regresión Logística

Fuente: *Elaboración propia, 2025*

En el caso de regresión logística binaria también se aplicó submuestreo(undersampling) aleatorio (RandomUnderSampler), estableciendo una relación 1:1 entre aprobados y reprobados esto en el parámetro `sampling_strategy = 1.0`. que reduce el desbalance a la clase mayoritaria y mejorar la capacidad del modelo para identificar ambas clases. Asimismo, se utilizó `random_state = 42`, para reproducir los resultados obtenidos. También se construye el pipeline, que integra las etapas de preprocesamiento de los datos, el balanceo de clases y el entrenamiento del modelo. La instrucción `pipeline_logreg` igual a `Pipeline(steps= [ ...])` se utiliza para construir un flujo que integra todas las etapas necesarias para el entrenamiento y la evaluación del modelo. En este pipeline se definen tres fases principales: el preprocesamiento, undersampling y modelo.

b. Entrenamiento del modelo y generación de predicciones

Con el pipeline terminado, se procede al entrenamiento del modelo de regresión logística binaria utilizando los datos de entrenamiento. Este proceso ajusta los coeficientes del modelo a partir de los datos disponibles. Posteriormente, se generaron las predicciones (ver Figura 3-17).

```

#Entrenamiento con Regresion logistica binaria
pipeline_logreg.fit(X_train, y_train)
#Predicciones
y_train_pred_lr = pipeline_logreg.predict(X_train)
y_train_prob_lr = pipeline_logreg.predict_proba(X_train)[: , 1]

y_test_pred_lr = pipeline_logreg.predict(X_test)
y_test_prob_lr = pipeline_logreg.predict_proba(X_test)[: , 1]

```

Figura 3-17: Entrenamiento del modelo de Regresión Logística.

Fuente: *Elaboración propia, 2025*

Para el entrenamiento del modelo se utiliza `pipeline_logreg.fit(X_train, y_train)` ejecutando todas las etapas definidas dentro del pipeline sobre el conjunto de datos de entrenamiento con las variables predictoras y la variable objetivo.

Una vez entrenado el modelo, se genera las predicciones de clase mediante la función `predict()` que se utiliza para generar la predicción final y las probabilidades asociadas para los conjuntos de entrenamiento (`y_train_pred_lr` y `y_train_prob_lr`) y de prueba (`y_test_pred_lr` y `y_test_prob_lr`). Se utilizaron las predicciones del modelo sobre el conjunto de entrenamiento (`y_train_pred_lr`) para evaluar el modelo durante el proceso de aprendizaje, mientras que las probabilidades estimadas (`y_train_prob_lr`) permitieron analizar el nivel de confianza del modelo al asignar la clase positiva (APR) a cada observación.

3.6. Evaluación del desempeño de los modelos predictivos

3.6.1. Evaluación de modelos predictivos de clasificación

La evaluación de modelos predictivos de clasificación aborda las métricas de evaluación utilizadas para evaluar modelos y la misma evaluación de los 3 modelos.

3.6.1.1. Métricas de evaluación utilizadas para la evaluación de los modelos

Para poder comparar de manera adecuada los modelos predictivos desarrollados, se utilizaron las mismas métricas de evaluación en todos los algoritmos implementados: XGBoost, Random Forest y Regresión Logística. Esto permite analizar el desempeño de cada modelo bajo los mismos criterios y mantener una evaluación coherente de los resultados obtenidos.

Para complementar la evaluación cuantitativa del desempeño de los modelos predictivos, se utilizó la matriz de confusión, la cual permite analizar de manera detallada la relación entre las clases reales y las clases predichas por cada algoritmo. Esta herramienta facilita la identificación de los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos, proporcionando una visión más precisa del comportamiento de los modelos más allá de una única métrica agregada. Las métricas utilizadas fueron: exactitud (accuracy), precisión (precision), recall, F1-score y área bajo la curva ROC (AUC-ROC), las cuales fueron calculadas a partir de las predicciones generadas sobre el conjunto de datos de prueba,

En la Tabla 3-6 se presenta una descripción resumida de cada métrica y su propósito dentro del proceso de evaluación.

Tabla 3-6: Tabla resumen de variables creadas

Métrica	Propósito en la evaluación
Exactitud (Accuracy)	Evaluar el nivel general de acierto del modelo.
Precision	Medir la confiabilidad de las predicciones de riesgo.
Recall	Evaluar la capacidad de detección de postulantes en riesgo
F1-score	Analizar el equilibrio entre precisión y recall
AUC-ROC	Medir la capacidad discriminatoria del modelo.

Fuente: *Elaboración propia, 2025*

3.6.1.2. Evaluación del modelo con XGBoost

El desempeño del modelo XGBoost fue evaluado utilizando métricas de evaluación, como la exactitud (accuracy), precisión (precision), recall, F1-score y el área bajo la curva ROC (AUC-ROC) (descritas en la Tabla 3-6). Estas métricas ayudan a analizar la capacidad del modelo para identificar a los postulantes con riesgo de reprobación. A continuación, se presenta el fragmento de código utilizado para el cálculo de dichas métricas a partir de las predicciones generadas por el modelo entrenado (ver Figura 3-18).

```
# Evaluación del modelo

# Matriz de confusión
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_auc_score
from sklearn.metrics import accuracy_score
print(confusion_matrix(y_train, y_train_pred)) # Para entrenamiento
print(confusion_matrix(y_test, y_test_pred))    # Para prueba
# Precision, Recall y F1-Score
from sklearn.metrics import classification_report
print(classification_report(y_train, y_train_pred)) # Entrenamiento
print(classification_report(y_test, y_test_pred))   # Prueba
# Accuracy
# Porcentaje total de predicciones correctas

print("Train Accuracy:", accuracy_score(y_train, y_train_pred)) # Entrenamiento
print("Test Accuracy:", accuracy_score(y_test, y_test_pred))    # Prueba
# AUC Área bajo la curva ROC

print("Train AUC:", roc_auc_score(y_train, y_train_prob)) # Entrenamiento
print("Test AUC:", roc_auc_score(y_test, y_test_prob))    # Prueba
```

Figura 3-18: Captura de la evaluación del modelo XGBoost

Fuente: *Elaboración propia, 2025*

Una vez entrenado el modelo predictivo XGBoost, su desempeño fue evaluado utilizando métricas de clasificación empleando funciones de la librería scikit-learn. A partir de las predicciones generadas para los conjuntos de entrenamiento y prueba (`y_train_pred` y `y_test_pred`), se calculó la matriz de confusión con el fin de identificar los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.

Posteriormente, se obtuvieron las métricas de precisión, recall y F1-score mediante la función `classification_report` en Python, permitiendo evaluar la confiabilidad de las predicciones y la capacidad del modelo para identificar postulantes en riesgo académico. La exactitud (accuracy) fue calculada como el porcentaje total de predicciones correctas realizadas por el modelo, mientras que el AUC-ROC se obtuvo a partir de las probabilidades estimadas (`y_train_prob` y `y_test_prob`), lo que permitió evaluar la capacidad discriminatoria del modelo.

Este procedimiento fue aplicado tanto al conjunto de entrenamiento como al conjunto de prueba, con el objetivo de analizar el comportamiento del modelo frente a datos no utilizados durante el entrenamiento y evaluar su capacidad de generalización.

3.6.1.3. Evaluación del modelo con Random forest

Para el modelo Random forest se evalúa empleando las mismas métricas de evaluación (exactitud (accuracy), precisión (precision), recall, F1-score y el área bajo la curva ROC (AUC-ROC)), con el objetivo de comparar de forma adecuada los algoritmos implementados, la evaluación se realizó utilizando datos que no fueron usados durante el entrenamiento. Esto permitió observar cómo se comporta cada modelo frente a nuevos datos y qué tan bien logra generalizar sus predicciones.

En la Figura 3-19 se muestra el procedimiento utilizado para la obtención de las métricas de evaluación correspondientes al modelo Random forest.

```
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.metrics import accuracy_score, roc_auc_score

# Matriz de confusión
print(confusion_matrix(y_train, y_train_pred_rf)) # Entrenamiento
print(confusion_matrix(y_test, y_test_pred_rf))   # Prueba

# Precision, Recall y F1-score
print(classification_report(y_train, y_train_pred_rf)) # Entrenamiento
print(classification_report(y_test, y_test_pred_rf))   # Prueba

# Accuracy (porcentaje de aciertos)
print("Accuracy TRAIN:", accuracy_score(y_train, y_train_pred_rf)) # Entrenamiento
print("Accuracy TEST:", accuracy_score(y_test, y_test_pred_rf))    # Prueba

# AUC (capacidad de separar las clases)
print("AUC TRAIN:", roc_auc_score(y_train, y_train_prob_rf)) # Entrenamiento
print("AUC TEST:", roc_auc_score(y_test, y_test_prob_rf))    # Prueba
```

Figura 3-19: Captura de la evaluación del modelo Random forest

Fuente: *Elaboración propia, 2025*

Después de entrenar el modelo Random Forest, su desempeño fue evaluado utilizando el mismo conjunto de métricas empleadas para los demás modelos. A partir de las predicciones generadas para los conjuntos de entrenamiento y prueba (`y_train_pred_rf` y `y_test_pred_rf`), se calculó la matriz de confusión para identificar los aciertos y errores del modelo. Posteriormente, se obtuvieron las métricas de precisión, recall y F1-score mediante la función `classification_report` en Python, permitiendo evaluar la confiabilidad de las predicciones y la capacidad del modelo para identificar postulantes en riesgo académico. La exactitud (accuracy) fue calculada como el porcentaje total de predicciones correctas, mientras que el AUC-ROC se obtuvo a partir de las probabilidades estimadas (`y_train_prob_rf` y `y_test_prob_rf`), permitiendo analizar la capacidad del modelo para diferenciar entre postulantes con y sin riesgo académico. Este procedimiento fue aplicado tanto al conjunto de entrenamiento como al conjunto de prueba.

3.6.1.4. Evaluación del modelo con Regresión logística binaria

Finalmente, se evaluó el modelo de Regresión Logística Binaria utilizando el mismo conjunto de métricas de desempeño. Este modelo fue considerado como referencia, permitiendo contrastar el rendimiento de los algoritmos más complejos frente a un modelo lineal ampliamente utilizado en problemas de clasificación.

La Figura 3-20 presenta el código y los resultados obtenidos durante la evaluación del modelo de regresión logística.

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, roc_auc_score

# Matriz de confusión
print(confusion_matrix(y_train, y_train_pred_lr)) # Entrenamiento
print(confusion_matrix(y_test, y_test_pred_lr))   # Prueba

# Precision, Recall y F1-score
print(classification_report(y_train, y_train_pred_lr)) # Entrenamiento
print(classification_report(y_test, y_test_pred_lr))   # Prueba

# Accuracy
print("Train Accuracy:", accuracy_score(y_train, y_train_pred_lr)) # Entrenamiento
print("Test Accuracy:", accuracy_score(y_test, y_test_pred_lr))    # Prueba

# AUC (Área bajo la curva ROC)
print("Train AUC:", roc_auc_score(y_train, y_train_prob_lr)) # Entrenamiento
print("Test AUC:", roc_auc_score(y_test, y_test_prob_lr))    # Prueba
```

Figura 3-20: Captura de la evaluación del modelo Regresión Logística Binaria.

Fuente: *Elaboración propia, 2025*

Una vez entrenado el modelo de Regresión Logística Binaria, su desempeño fue evaluado utilizando el mismo procedimiento aplicado a los demás modelos predictivos. A partir de las predicciones generadas para los conjuntos de entrenamiento y prueba (`y_train_pred_lr` y `y_test_pred_lr`), se calculó la matriz de confusión, lo que permitió identificar los aciertos y errores del modelo. Luego, se obtuvieron las métricas de precisión, recall y F1-score mediante la función `classification_report` en Python, permitiendo evaluar la capacidad del modelo para identificar postulantes en riesgo académico. La exactitud (accuracy) fue calculada como el porcentaje total de predicciones correctas, mientras que el AUC-ROC se obtuvo a partir de las probabilidades estimadas (`y_train_prob_lr` y `y_test_prob_lr`), permitiendo analizar la capacidad del modelo para diferenciar entre postulantes con y sin riesgo académico. Este procedimiento fue aplicado tanto al conjunto de entrenamiento como al conjunto de prueba.

Los tres modelos muestran coincidencias en la evaluación mediante métricas, ya que se aplicaron las mismas directrices para la construcción de los conjuntos de entrenamiento y prueba.

3.6.2. Comparación de métricas entre modelos

Es necesario comparar el desempeño de los modelos predictivos desarrollados, se calcularon métricas de clasificación sobre el conjunto de prueba, incluyendo precision, recall, F1-score, accuracy y el área bajo la curva ROC (AUC). Estas métricas permiten evaluar de manera integral la capacidad de cada modelo para identificar correctamente a los postulantes con mayor probabilidad de reprobación (ver Figura 3-21).

```
from sklearn.metrics import classification_report, accuracy_score, roc_auc_score

def extraer_metricas(y_true, y_pred, y_prob):
    reporte = classification_report(y_true, y_pred, output_dict=True)

    precision = reporte['1']['precision'] # clase positiva (aprobado)
    recall = reporte['1']['recall']
    f1 = reporte['1']['f1-score']
    accuracy = accuracy_score(y_true, y_pred)
    auc = roc_auc_score(y_true, y_prob)

    return [precision, recall, f1, accuracy, auc]
```

Figura 3-21: Fragmento del código de comparación gráfica de modelos

Fuente: *Elaboración propia, 2025*

A partir del flujograma presentado, se incorporó un procedimiento de extracción de métricas mediante una función programada en Python llamada `extraer_metricas`, la cual recibe como entrada los valores reales del conjunto de prueba, las predicciones generadas por cada modelo y sus probabilidades estimadas. Este componente utiliza el reporte producido por la librería `scikit-learn` para calcular las métricas de precision, recall, F1-score, accuracy y AUC, permitiendo mostrar dichos indicadores en un vector comparable entre los algoritmos evaluados. De esta manera, la función garantiza que la comparación del desempeño se realice bajo los mismos criterios de medición, facilitando el breve análisis estadístico mostrado en la Figura 3-21.

3.7. Desarrollo de la interfaz interactiva

Para la visualización del modelo predictivo, se desarrolló una interfaz interactiva que permite la interacción directa del usuario con el modelo ya entrenado. Diseñada para facilitar el ingreso de variables demográficas y académicas del postulante, así como la visualización del resultado de la predicción.

3.7.1. Instalación de frameworks

Para el correcto funcionamiento del sistema, se instala un conjunto de frameworks y librerías para el procesamiento de datos, aprendizaje automático y desarrollo de interfaces gráficas. Estas dependencias fueron instaladas sobre un entorno de ejecución basado en Python, siendo compatibles entre los distintos componentes del sistema. El listado de las librerías utilizadas y sus respectivas versiones se encuentra documentado en el archivo `requerimientos.txt` (Ver Anexo 12). A continuación, se presenta una captura

de dicho archivo, donde se detallan las principales dependencias necesarias para ver el código de la interfaz (ver Figura 3-22).

```
python -m pip install PyQt6
python -m pip install pandas
python -m pip install numpy==2.0.2
python -m pip install scipy
python -m pip install scikit-learn==1.6.1
python -m pip install imbalanced-learn==0.14.0
python -m pip install --force-reinstall xgboost
python -m pip install matplotlib
python -m pip install openpyxl

python version 3.11.1
```

Figura 3-22: Fragmento del código de inicialización de la interfaz gráfica desarrollada con PyQt6

Fuente: *Elaboración propia, 2025*

Para el desarrollo de la interfaz de usuario se utilizó el lenguaje Python en la versión 3.11.1 con un conjunto de librerías que permiten integrar el procesamiento de datos, la ejecución de modelos entrenados y la visualización de resultados.

La librería PyQt6 fue empleada para la construcción de la interfaz gráfica de usuario, permitiendo desarrollar una aplicación interactiva, intuitiva y multiplataforma. SciPy para la matemáticas y estadísticas necesarias durante el análisis.

Para lectura del modelo predictivo se empleó scikit-learn e imbalanced-learn. La librería XGBoost fue utilizada para leer el algoritmo.

Finalmente, Matplotlib fue empleada para la visualización de resultados, OpenPyXL para la lectura y escritura de archivos en formato Excel.

En conjunto, estas librerías permiten desarrollar una aplicación integrada, funcional y reproducible para la ejecución y evaluación del modelo.

3.7.2. Estructura de archivos de la interfaz

La estructura de archivos del sistema fue organizada de manera modular, separando los componentes de la interfaz gráfica que sería el frontend, el backend y los archivos asociados al modelo entrenado. Esta organización ayuda a la comprensión del proyecto, el mantenimiento del código y la posibilidad de futuras ampliaciones.

A continuación, se presenta la estructura general de carpetas y archivos del sistema implementado, donde se identifican los archivos principales de la aplicación, los scripts de prueba y los recursos necesarios para su ejecución (ver Figura 3-23).

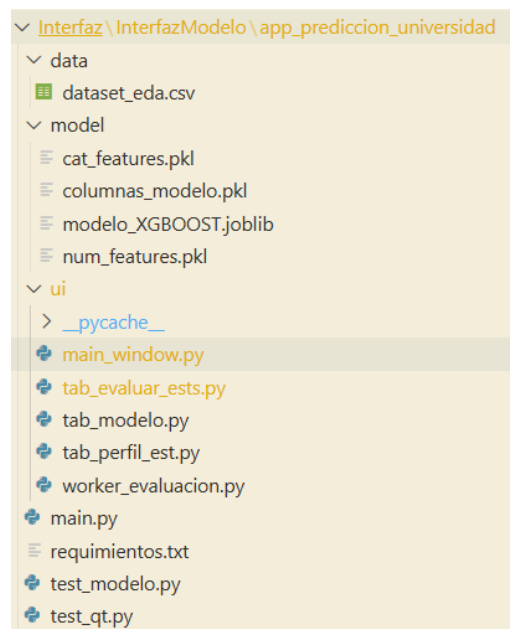


Figura 3-23: Estructura de archivos del sistema implementado

Fuente: *Elaboración propia, 2025*

El sistema se organizó bajo una estructura modular de directorios, con el propósito de separar los datos, los modelos predictivos y la interfaz gráfica, y así ser de fácil mantenimiento y su comprensión. El directorio data contiene el conjunto de datos de referencia utilizado durante el análisis exploratorio, mientras que la carpeta model almacena el modelo XGBoost previamente entrenado junto con lo necesario para su correcta ejecución, teniendo coherencia entre el entrenamiento y el proceso de predicción.

Por su parte, el directorio ui agrupa los archivos de la interfaz gráfica de usuario, incluyendo la ventana principal y las pestañas funcionales del sistema. El archivo main.py actúa como punto de entrada de la aplicación, mientras que los archivos auxiliares, como worker_evaluacion.py, requerimientos.txt y los scripts de prueba, permiten gestionar procesos de evaluación, definir dependencias y validar el correcto funcionamiento del sistema.

3.7.3. Implementación de la Interfaz

En este apartado se aborda el desarrollo de la interfaz desde el frontend y backend.

3.7.3.1. Desarrollo del frontend

La interfaz gráfica fue desarrollada utilizando la librería PyQt6, la cual permite la creación de aplicaciones de escritorio mediante componentes visuales como ventanas, etiquetas, botones y cuadros de selección. El archivo principal main.py contiene la inicialización de la aplicación y la definición de la ventana principal del sistema, donde se integran los elementos visuales necesarios para la interacción con el

usuario. A continuación se muestra un fragmento del código correspondiente a la inicialización de la interfaz gráfica y la carga de la ventana principal, evidenciando el uso del framework PyQt6 para la construcción del entorno visual de la aplicación (ver Figura-3-24).

```
from ui.tab_modelo import TabModelo
from ui.tab_perfil_est import TabPerfilEst
from ui.tab_evaluar_ests import TabEvaluarEsts

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Sistema de Predicción Universitaria")
        self.setGeometry(100, 100, 1200, 750)
        self.tabs = QTabWidget()

        self.tab_modelo = TabModelo()
        self.tab_perfil = TabPerfilEst()
        self.tab_evaluar = TabEvaluarEsts()

        # Agregar pestañas
        self.tabs.addTab(self.tab_modelo, "Modelo Predictivo")
        self.tabs.addTab(self.tab_perfil, "Perfil Estadístico")
        self.tabs.addTab(self.tab_evaluar, "Evaluación Masiva")
```

Figura 3-24: Fragmento del código de inicialización de la interfaz gráfica desarrollada con PyQt6

Fuente: *Elaboración propia, 2025*

La clase `MainWindow` representa la ventana principal del sistema de predicción universitaria, encargada de inicializar y organizar la interfaz gráfica de usuario. Durante su construcción, se define el título de la aplicación y las dimensiones de la ventana, estableciendo un entorno visual adecuado para la interacción con el usuario.

La interfaz se estructura mediante un sistema de pestañas (`QTabWidget`), el cual permite organizar las distintas funcionalidades del sistema de forma clara y accesible. En particular, se inicializan tres pestañas principales: `Modelo Predictivo`, destinada a la ejecución del modelo de predicción; `Perfil Estadístico`, orientada al análisis descriptivo de los datos; y `Evaluación Masiva`, enfocada en la evaluación del desempeño del modelo sobre múltiples registros. Esta organización facilita la navegación del usuario y permite una interacción ordenada con las principales funcionalidades del sistema.

3.7.3.2. Desarrollo del backend

El backend del sistema se encarga de la carga del modelo predictivo previamente entrenado y de la ejecución del proceso de predicción a partir de los datos ingresados en la interfaz gráfica. Para este propósito se utilizó la librería `joblib`, que permite cargar el modelo serializado de manera eficiente.

La correcta carga y funcionamiento del modelo fue verificada mediante un script de prueba independiente, el cual permite comprobar que el modelo puede ser utilizado sin errores dentro del

sistema. A continuación se muestra un fragmento del código correspondiente a la carga del modelo y la ejecución de una predicción de prueba a través del modelo (ver Figura 3-25 y Figura 3-26).

```
# Cargar modelo y datos
def cargar_artifactos(self):
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    data_path = os.path.join(base_dir, "data", "dataset_eda.csv")
    self.df_ref = pd.read_csv(data_path)
    self.tasa_por_colegio = (
        self.df_ref
        .groupby("NOMBRE_COLEGIO")["RESULTADO_FINAL"]
        .apply(lambda x: (x == "APR").mean())
        .to_dict()
    )
    self.modelo = joblib.load("model/modelo_XGBOOST.joblib")
    with open("model/columnas_modelo.pkl", "rb") as f:
        self.columnas_modelo = pickle.load(f)
    with open("model/cat_features.pkl", "rb") as f:
        self.cat_features = pickle.load(f)
    self.tasa_actual = 0.0
```

Figura 3-25: Fragmento del código para la carga del modelo predictivo y del dataset en el backend.

Fuente: *Elaboración propia, 2025*

La función `cargar_artifactos()` inicializa los recursos para el funcionamiento del modelo predictivo dentro de la interfaz. Primero identifica el directorio base del proyecto y se carga el conjunto de datos. Este conjunto de datos se emplea para calcular la tasa de aprobación por colegio. Posteriormente, se carga el modelo XGBoost entrenado junto con lo necesario asegurando el proceso de predicción. Finalmente, se inicializa una variable que permite almacenar la tasa de aprobación actual asociada a un registro evaluado.

```
def predecir(self):
    try:
        datos = {}
        for col, widget in self.inputs.items():
            if widget.currentIndex() == -1:
                raise ValueError(f"Seleccione un valor para {col}")
            if col in ["MAYOR_EDAD", "MIGRA_UNIVERSIDAD"]:
                datos[col] = int(widget.currentData())
            elif col in NUMERICAS_MODELO:
                datos[col] = float(widget.currentText())
            else:
                datos[col] = widget.currentText()
        datos["TASA_APR_COLEGIO"] = float(self.tasa_actual)
        # Construir DataFrame
        df = pd.DataFrame([datos])
        df = df.reindex(columns=self.columnas_modelo, fill_value=0)
        pred = self.modelo.predict(df)[0]
        proba = self.modelo.predict_proba(df)[0][1]
        self.mostrar_resultado(pred, proba)
        if hasattr(self, "tab_perfil") and hasattr(self, "tabs_widget"):
            self.tab_perfil.actualizar_perfil(datos, proba)
            self.tabs_widget.setCurrentIndex(
                self.tabs_widget.indexOf(self.tab_perfil)
            )
    except Exception as e:
        QMessageBox.critical(self, "Error", str(e))
```

Figura 3-26: Fragmento del código para la ejecución del modelo en el backend.

Fuente: *Elaboración propia, 2025*

La función `predecir()` se encarga de ejecutar el proceso completo de predicción a partir de los datos ingresados, esta recopila y valida los valores seleccionados en los distintos campos de entrada según lo definido por el modelo. Posteriormente, se incorpora la tasa de aprobación por ya calculado, y se construye un DataFrame con la estructura esperada por el modelo. A continuación, el modelo XGBoost realiza la predicción de la clase (fuera de riesgo o en riesgo) y estima la probabilidad asociada a la clase positiva, las cuales son mostradas directamente en la interfaz.

Finalmente, la función actualiza de forma automática el perfil estadístico del postulante y redirige al usuario a la pestaña correspondiente, permitiendo una visualización inmediata de los resultados. En caso de que ocurra algún error durante el proceso, se gestiona la excepción mostrando un mensaje informativo, permitiendo una ejecución controlada y robusta del sistema (ver Figura 3-27).

```
def predecir(self, df):
    X = df.reindex(columns=self.columnas_modelo, fill_value=0)

    pred = self.modelo.predict(X)
    proba = self.modelo.predict_proba(X)[:, 1]

    df["PREDICCION"] = np.where(pred == 1, "FUERA DE RIESGO", "EN RIESGO")
    df["PROBABILIDAD"] = proba

    self.df_resultados = df
    self.mostrar_tabla(df)
```

Figura 3-27: Fragmento del código para la ejecución del modelo en el backend.

Fuente: *Elaboración propia, 2025*

Esta función se encarga de realizar la predicción masiva de un conjunto de registros, evaluando simultáneamente a múltiples postulantes. Los datos de entrada se ajustan a la estructura requerida por el modelo. Posteriormente, el modelo genera las predicciones de clase y las probabilidades asociadas para cada registro. A partir de estos resultados, se clasifica a cada postulante como “EN RIESGO” o “FUERA DE RIESGO”, y se incorpora la probabilidad correspondiente a cada predicción. Finalmente, los resultados se almacenan y se presentan en una tabla dentro de la interfaz.

3.7.4. Predicción para identificar si uno o muchos postulantes están en riesgo o no de reprobación del examen de ingreso, en la interfaz

Es necesario hablar sobre cómo se realizó la predicción en la interfaz cuando hay uno o varios postulantes.

3.7.4.1. Evaluación para un postulante

Para la implementación de la interfaz interactiva del modelo predictivo fue necesario desarrollar un procedimiento que genere dinámicamente los controles de entrada de acuerdo con las variables seleccionadas. El método programado permite cargar desde el conjunto de datos los valores categóricos

válidos y asociarlos a componentes tipo ComboBox conectados a eventos, garantizando que las predicciones se ejecuten bajo un proceso estructurado y coherente con el framework utilizado (ver Figura 3-29).

```
def crear_inputs(self):
    combo_colegio = QComboBox()
    colegios = sorted(
        self.df_ref["NOMBRE_COLEGIO"].dropna().astype(str).unique().tolist()
    )
    combo_colegio.addItem(colegios)
    combo_colegio.setCurrentIndex(-1)
    combo_colegio.currentTextChanged.connect(self.actualizar_tasa_colegio)
    self.form.addRow(QLabel("Nombre del colegio"), combo_colegio)
    self.combo_colegio = combo_colegio
    for col in self.columnas_modelo:
        if col in ["RESULTADO_FINAL", "AREA_CARRERA", "TASA_APR_COLEGIO"]:
            continue
        label = ETIQUETAS_COLUMNAS.get(col, col)
        combo = QComboBox()
        if col == "ANTIO_BACHILLERATO":
            combo.addItem([str(a) for a in range(1995, 2011)])
        elif col == "EDAD":
            combo.addItem([str(e) for e in range(15, 43)])
        elif col == "ANIOS_POST_BACH":
            combo.addItem(["0", "1", "2", "3", "4", "5"])
        elif col == "MAYOR_EDAD":
            combo.addItem("No", 0)
            combo.addItem("Sí", 1)
        elif col == "MIGRA_UNIVERSIDAD":
            combo.addItem("No", 0)
            combo.addItem("Sí", 1)
        else:
            valores = (self.df_ref[col].dropna().astype(str).sort_values().unique().tolist())
            combo.addItem(valores)
        combo.setCurrentIndex(-1)
        self.form.addRow(QLabel(label), combo)
        self.inputs[col] = combo
```

Figura 3-28: Fragmento del código para la predicción de un solo postulante

Fuente: *Elaboración propia, 2025*

Como se observa en la Figura 3-2, el método desarrollado registra cada control en un diccionario interno de la aplicación, desde donde los botones de predicción y de visualización del perfil pueden recuperar la información ingresada por el usuario.

Este componente constituye la base funcional de la herramienta, ya que asegura la consistencia de los datos de entrada y facilita el uso del modelo por parte de estudiantes o profesionales sin necesidad de interactuar directamente con el código.

3.7.4.2. Evaluación para varios postulantes

Para la presentación de los resultados del modelo predictivo seleccionado se desarrolló un método encargado de construir y configurar una tabla de visualización dentro de la aplicación.

Este componente habilita el botón de carga de información, organiza las columnas que serán mostradas al usuario y prepara el conjunto de datos mediante el reinicio del índice, asegurando un entorno adecuado para la interacción posterior (ver Figura 3-29).

```

def mostrar_resultados(self, df):
    self.btn_cargar.setEnabled(True)
    self.df_resultados = df.reset_index(drop=True)
    columnas = [
        "SEXO", "EDAD", "NOMBRE_COLEGIO",
        "PREDICCION", "PROBABILIDAD", "PERFIL"]
    self.tabla.setRowCount(len(df))
    self.tabla.setColumnCount(len(columnas))
    self.tabla.setHorizontalHeaderLabels(columnas)
    for i, row in self.df_resultados.iterrows():
        for j, col in enumerate(columnas):
            if col == "PERFIL":
                btn = QPushButton("Ver perfil")
                btn.setStyleSheet("""...""")
                btn.clicked.connect(lambda _, idx=i: self.ver_perfil(idx))
                self.tabla.setCellWidget(i, j, btn)
                self.tabla.setRowHeight(i, 44)
                continue
            valor = row[col]
            if col == "PROBABILIDAD":
                valor = f"{valor:.2%}"
            item = QTableWidgetItem(str(valor))
            item.setTextAlignment(Qt.AlignmentFlag.AlignCenter)
            if col == "PREDICCION": ...
            self.tabla.setItem(i, j, item)
    self.tabla.resizeColumnsToContents()
    col_perfil = columnas.index("PERFIL")
    self.tabla.setColumnWidth(col_perfil, 120)

```

Figura 3-29: Fragmento del código para la predicción de varios postulantes

Fuente: *Elaboración propia, 2025*

Como se observa en la Figura 3-29, el método implementado crea dinámicamente los controles de tipo botón asociados a cada registro de postulante, permitiendo acceder a la opción “Ver perfil” desde la misma tabla. Además, consolida los valores de probabilidad y predicción con un formato homogéneo y define el ancho de la columna PERFIL, facilitando que los usuarios puedan interpretar los hallazgos del estudio sin necesidad de manipular directamente el código fuente. Este componente se constituye así en la base funcional para la explicación de indicadores y para el uso práctico del modelo en predicciones individuales y masivas.

4. Análisis de Resultados y Discusión

4.1. Resultados generales del análisis exploratorio de datos

Se presentan los resultados obtenidos a partir del análisis exploratorio de datos y la estadística descriptiva aplicada a los registros históricos del examen de ingreso de la Facultad de Ciencias y Tecnología de la UMSS, correspondientes a las gestiones 2006–2025.

Este análisis permitió identificar patrones, tendencias temporales y características relevantes del desempeño académico de los postulantes, utilizando visualizaciones desarrolladas mediante dashboards interactivos en Power BI, los cuales facilitan la interpretación de la información (ver Anexo 5 y 6).

4.1.1. Tasa de aprobación en general del examen de ingreso

A continuación, se muestra el dashboard con los análisis estadísticos de los datos (ver Figura 4-1).

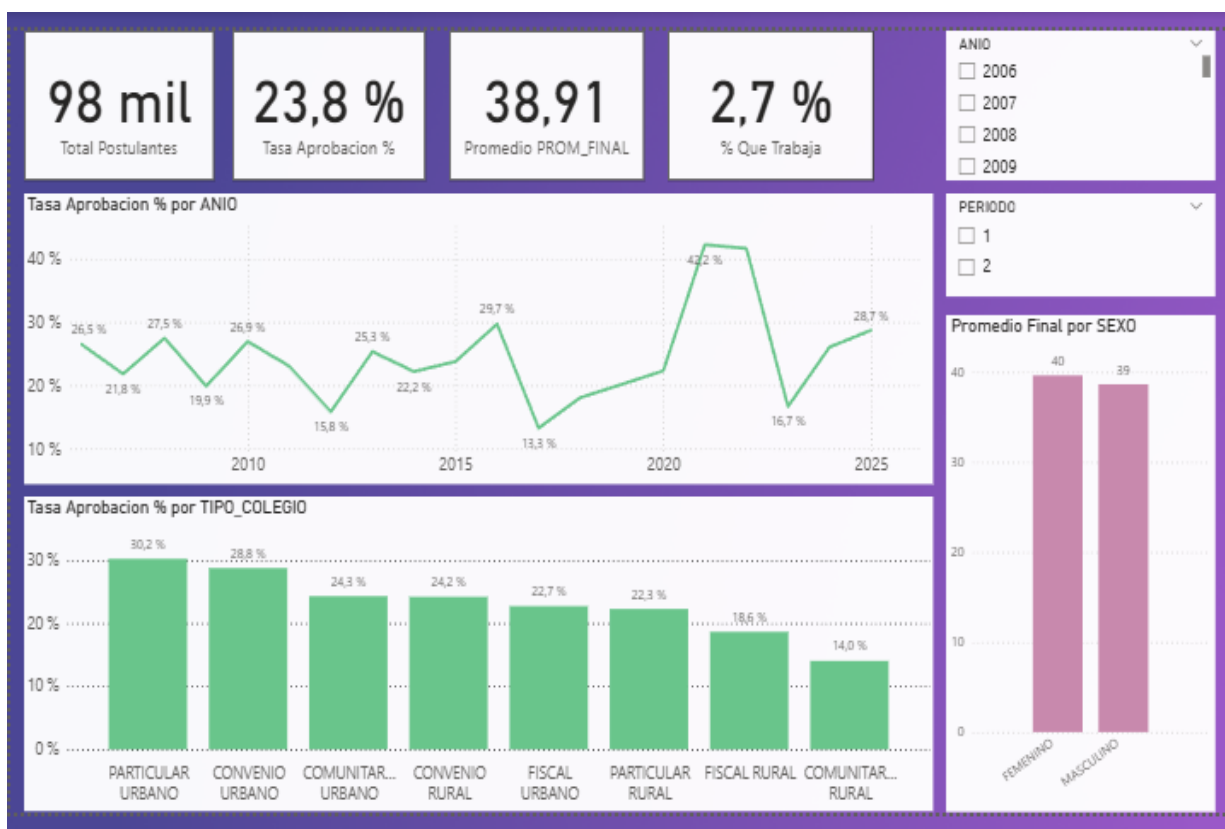


Figura 4-1: Dashboard, un Panorama general

Fuente: *Elaboración propia, 2025*

La Figura 4-1 presenta un resumen de los datos del examen de ingreso de la FCyT–UMSS. En la parte superior se observa que el total de postulantes analizados asciende aproximadamente a 98 mil registros y

una tasa de aprobación global del 23,8 %, mientras que el promedio general del puntaje final (PROM_FINAL) es de 38,91. Asimismo, se muestra que solo un 2,7 % de los postulantes declara encontrarse trabajando y registra condición laboral activa, evidenciando que la mayoría se dedica exclusivamente a sus estudios.

El dashboard incluye cuatro visualizaciones principales: la tasa de aprobación por año, con valores que varían entre más del 40 % y menos del 20 % según el periodo (mejor explicado en la sección 4.1.2.); la tasa de aprobación por tipo de colegio, donde los particulares urbanos alcanzan 30,2 %, los de convenio urbano 28,8 %, los comunitarios urbanos y convenio rural alrededor de 24 %, los fiscales urbanos y particulares rurales cerca de 22 %, y los comunitarios rurales 14 % (mejor explicado en la sección 4.1.3.); el promedio del puntaje por sexo, que muestra valores similares entre mujeres de 40 y varones de 39 notando que en su mayoría las mujeres tienen un promedio un poco más alto que los varones; y, finalmente, los filtros por año y periodo académico, que permiten segmentar el análisis de forma dinámica.

4.1.2. Evolución temporal de la tasa de aprobación por año

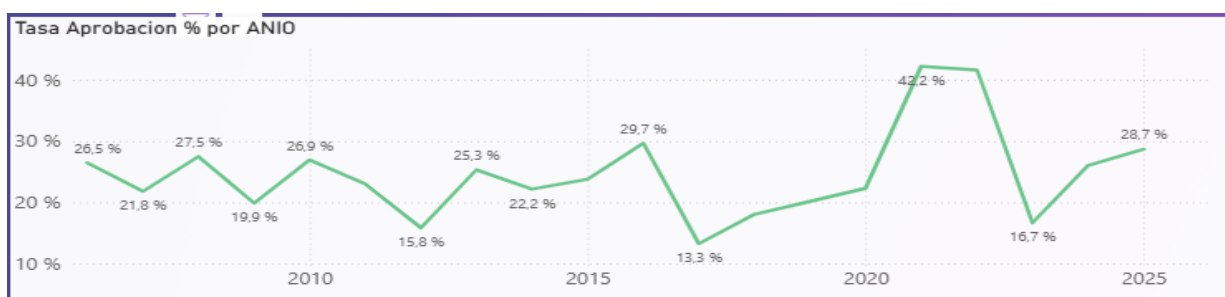


Figura 4-2: Gráfico de líneas de la tasa de aprobación por año

Fuente: *Elaboración propia, 2025*

La figura 4-2 muestra la evolución de la tasa de aprobación porcentual por año, se aprecia un comportamiento que varía a lo largo del periodo analizado. En los primeros años se ven valores cercanos al 25 %, con fluctuaciones moderadas entre incrementos y descensos, destacando algunos años con tasas superiores al 27 % y otros con reducciones por debajo del 22 %. Posteriormente, se identifica una disminución más marcada alrededor de ciertos años, donde la tasa de aprobación alcanza valores cercanos al 15 % o incluso al 13 %, lo que refleja un descenso significativo en el rendimiento de los postulantes.

A partir de los años siguientes por 2021, la tasa muestra una tendencia de recuperación gradual, alcanzando un pico máximo superior al 40 % en un periodo específico, lo que representa el mayor nivel de aprobación registrado en toda la serie. Sin embargo, este incremento no se mantiene de forma sostenida, ya que posteriormente se observa una nueva caída hasta valores cercanos al 17 %, seguida de una recuperación parcial en los años más recientes, donde la tasa vuelve a situarse alrededor del 28 %. En conjunto, esta evolución temporal evidencia que la tasa de aprobación no presenta una tendencia

lineal, sino que está sujeta a variaciones importantes a lo largo del tiempo, lo cual sugiere la influencia de factores académicos, institucionales o contextuales que afectan el desempeño de los postulantes en distintos periodos.

4.1.3. Evolución temporal de la tasa de aprobación

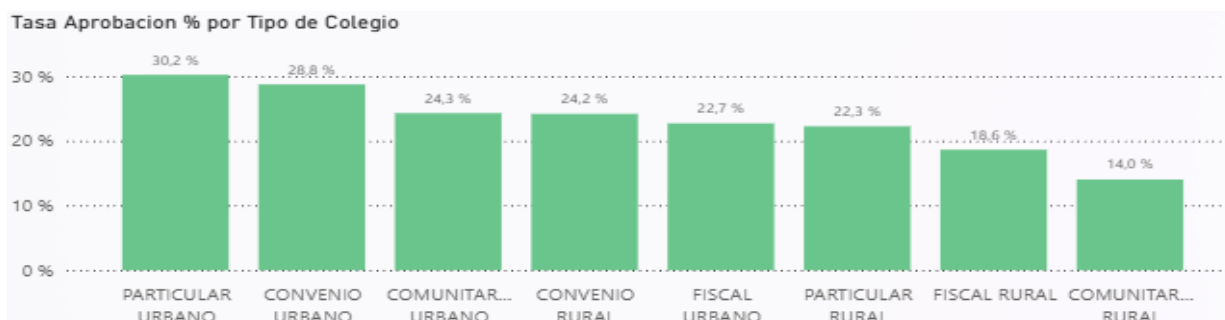


Figura 4-3: Grafico de líneas de la tasa de aprobación por año

Fuente: *Elaboración propia, 2025*

La Figura 4-3 presenta la tasa de aprobación según el tipo de colegio, evidenciando diferencias claras en el desempeño de los postulantes en función del tipo de establecimiento de procedencia. Se observa que los colegios particulares urbanos alcanzan la mayor tasa de aprobación, con un valor aproximado de 30,2 %, seguidos por los colegios de convenio urbano, que registran alrededor del 28,8 %. Estos resultados sugieren que los postulantes provenientes de instituciones urbanas de gestión privada o de convenio presentan mejores condiciones académicas para afrontar el examen de ingreso.

En un nivel intermedio se encuentran los colegios comunitarios urbanos y los colegios de convenio rural, cuyas tasas de aprobación se sitúan alrededor del 24 %, lo que indica un desempeño moderado. Por debajo de estos valores se ubican los colegios fiscales urbanos y los particulares rurales, con tasas cercanas al 22 %, reflejando una menor probabilidad de aprobación. Finalmente, los valores más bajos corresponden a los colegios fiscales rurales y comunitarios rurales, con tasas de aprobación de aproximadamente 18,6 % y 14,0 %, respectivamente. En conjunto, estos resultados permiten concluir que el contexto urbano-rural y el tipo de gestión educativa influyen de manera significativa en el rendimiento académico de los postulantes, evidenciando brechas que podrían ser consideradas en el diseño de estrategias de apoyo académico y políticas educativas orientadas a reducir las desigualdades en el acceso y desempeño en el examen de ingreso.

4.1.4. Perfil general de los postulantes

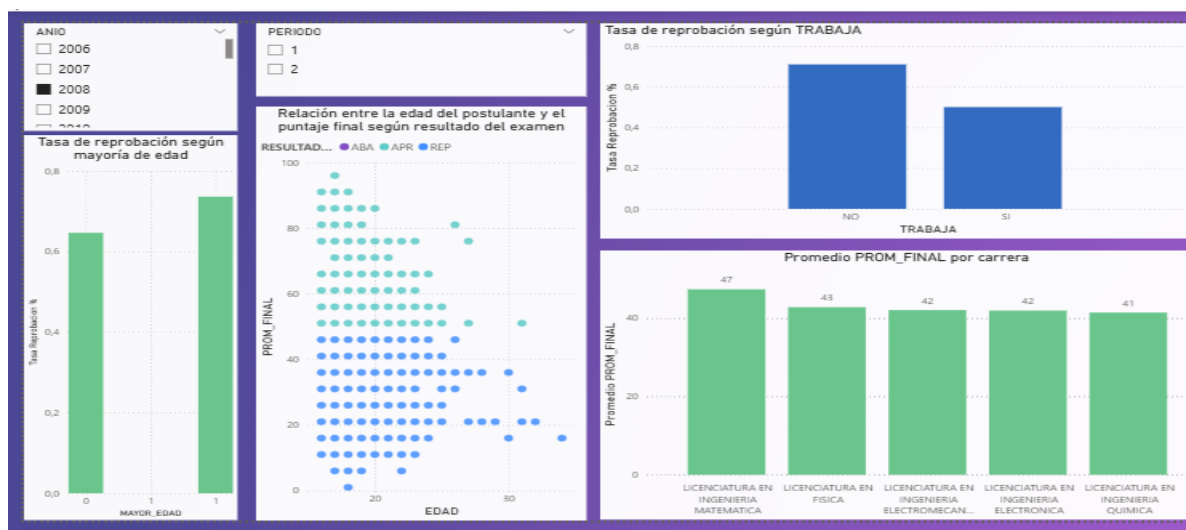


Figura 4-4: Dashboard de perfil general de los postulantes

Fuente: *Elaboración propia, 2025*

En la figura 4-4 el gráfico muestra un dashboard del perfil general de los postulantes donde, el gráfico de barras correspondiente a la tasa de reprobación según mayoría de edad, se observa que los postulantes mayores de edad presentan una tasa de reprobación aproximada del 75 %, mientras que los menores de edad registran una tasa cercana al 65 %. Esta diferencia de alrededor de 10 puntos porcentuales indica que los postulantes de mayor edad tienden a reprobar en mayor proporción el examen de ingreso, lo que sugiere una posible relación entre la edad y el rendimiento académico, posiblemente asociada a una menor continuidad en los estudios formales.

En el diagrama de dispersión que relaciona la edad del postulante con el puntaje final (PROM_FINAL) se observa que la mayor concentración de puntajes aprobatorios se encuentra en el rango de edades entre 17 y 20 años, con valores de puntaje final que superan los 51 puntos, correspondientes a resultados aprobados. En este grupo, los puntos se agrupan principalmente en la zona superior del gráfico, lo que indica un desempeño académico más consistente.

Por el contrario, a partir de los 21 años aumenta la dispersión de los puntajes, observándose una mayor variabilidad en los resultados obtenidos.

En este rango se incrementa la presencia de resultados reprobatorios, con valores frecuentes por debajo de los 40 puntos, y una menor concentración de puntajes altos. Esta tendencia sugiere que, conforme aumenta la edad del postulante, el rendimiento académico tiende a disminuir, posiblemente debido a factores como una menor continuidad en los estudios, mayor tiempo transcurrido desde la finalización del bachillerato o la combinación de responsabilidades adicionales, lo cual podría influir en el desempeño durante el examen de ingreso.

Por otro lado, el gráfico de barras que muestra la tasa de reprobación según la condición laboral indica que los postulantes que no trabajan presentan una tasa de reprobación cercana al 70 %, mientras que aquellos que sí trabajan registran una tasa aproximada del 55 %.

Esta diferencia de alrededor de 15 puntos porcentuales sugiere que los postulantes que trabajan podrían desarrollar habilidades de organización y responsabilidad que influyen positivamente en su desempeño académico, aunque este resultado debe interpretarse considerando posibles variables adicionales.

El dashboard tiene filtros correspondientes al año y al período académico, los cuales permiten segmentar el análisis y observar cómo varían los indicadores y gráficos al seleccionar subconjuntos específicos de los datos. Esta funcionalidad ayuda a tener un análisis dinámico del comportamiento de los postulantes en distintos contextos temporales y académicos.

Finalmente, el gráfico de barras correspondiente al promedio del puntaje final por carrera muestra que la Licenciatura en Ingeniería Matemática presenta el promedio más alto, con un valor aproximado de 47 puntos, seguida por Licenciatura en Física con 43 puntos. Las carreras de Ingeniería Electromecánica e Ingeniería Electrónica registran promedios cercanos a los 42 puntos, mientras que Ingeniería Química presenta el promedio más bajo, con alrededor de 41 puntos. Estas diferencias, aunque moderadas, evidencian variaciones en el rendimiento académico promedio según la carrera elegida.

4.1.5. Origen geográfico y colegio de los postulantes



Figura 4-5: Dashboard del origen y colegio de los postulantes

Fuente: *Elaboración propia, 2025*

La Figura 4-5 es otro dashboard que muestra un análisis del origen geográfico y colegio de los postulantes; el gráfico de barras de color azul que presenta el Top 6 de ciudades de nacimiento por tasa de reprobación, se observa que la ciudad de Beni registra la tasa de reprobación más alta, con un valor aproximado de 80%, seguida por Potosí con 79%. A continuación, se encuentran Pando y Santa Cruz, ambas con tasas cercanas a 78%, mientras que La Paz y Tarija presentan valores ligeramente menores, alrededor de 77%. Estos resultados indican que, independientemente de la ciudad de nacimiento, las tasas de reprobación son elevadas en todos los casos analizados, superando el 75 %, lo que evidencia un alto nivel de dificultad del examen de ingreso y posibles desigualdades en la formación académica previa según el contexto regional.

Por otro lado, el gráfico de barras color verde correspondiente al Top 7 de colegios con mayor nota promedio en el examen muestra que el Colegio La Salle Convenio Tarija alcanza el promedio más alto de puntaje final, cercano a los 100 puntos, seguido por el Colegio Mariscal Braun y el Colegio Pascual Guallart – Licenciado de TII, con promedios aproximados de 95 y 94 puntos, respectivamente. Asimismo, los colegios Unidad Educativa Luis Eduardo López y San Cristóbal presentan promedios cercanos a los 92 puntos, mientras que 10 de Noviembre Unión Campeche y Modelo Nuevo Horizonte registran valores ligeramente inferiores, alrededor de los 88 a 90 puntos. Estos resultados evidencian que ciertos establecimientos educativos concentran postulantes con un desempeño académico notablemente superior al promedio general.

En el gráfico de barras de color lila se muestra el Top 6 de ciudades de procedencia del colegio por tasa de reprobación, se observa nuevamente que Beni presenta la tasa de reprobación más elevada, con un valor cercano a 83%, seguido por Potosí con aproximadamente 80%. Las ciudades de Santa Cruz, La Paz, Oruro y Tarija presentan tasas que oscilan entre 79% y 77%, manteniéndose todas por encima del 75 %. Este comportamiento sugiere que la ciudad donde se ubica el colegio de procedencia también está asociada a diferencias en el rendimiento académico, aunque la reprobación sigue siendo alta de manera generalizada.

Finalmente, el gráfico de barras de color guindo correspondiente a la cantidad de reprobados por nombre de colegio muestra que el Instituto Americano registra el mayor número de postulantes reprobados, con una cifra cercana a los 600 casos, seguido por los colegios Calama y La Salle, con valores aproximados de 580 y 560 reprobados, respectivamente. A continuación, se encuentran los colegios Santo Domingo Savio, 6 de Agosto y Don Bosco, cuyos valores se sitúan entre 520 y 500 reprobados, mientras que Guillermo Daniel Sánchez Bustamante presenta un número ligeramente menor, cercano a los 470 casos.

Estos resultados reflejan que algunos colegios concentran un mayor volumen absoluto de reprobaciones, lo cual puede estar relacionado tanto con el tamaño de la población postulante como con el desempeño académico promedio de sus estudiantes.

4.1.6. Aprobados y reprobados por origen y colegio



Figura 4-6: Dashboard de aprobados y reprobados por origen y colegio

Fuente: *Elaboración propia, 2025*

En el gráfico correspondiente al Top 5 de colegios según la cantidad de aprobados y reprobados, se observa que el colegio San Agustín presenta la mayor proporción de aprobados, alcanzando un 62,85 %, mientras que el 37,15 % de sus postulantes resultan reprobados. En contraste, los colegios Santo Domingo Savio y La Salle muestran una mayor predominancia de reprobación, con porcentajes de 61,65 % y 67,56 %, respectivamente, y tasas de aprobación de 38,35 % y 32,44 %. Asimismo, el Instituto Americano registra una tasa de reprobación del 70,13 %, mientras que Calama presenta el escenario más desfavorable, con un 76,54 % de postulantes reprobados y solo un 23,46 % de aprobados. Estos resultados evidencian que, incluso entre los colegios con mayor volumen de postulantes, la reprobación continúa siendo predominante, con excepción parcial del colegio San Agustín.

Por otro lado, el gráfico que muestra el Top 5 de ciudades de colegio según los resultados del examen refleja que todas las ciudades analizadas presentan altas tasas de reprobación. En Cochabamba, el 75,43 % de los postulantes reprueba, frente a un 24,57 % que aprueba. De postulantes de Potosí, la reprobación alcanza el 81,21 %, siendo una de las más elevadas, con solo 18,79 % de aprobados. De manera similar, La Paz y Santa Cruz registran tasas de reprobación del 78,63 % y 80,92 %, respectivamente, mientras que Beni presenta la tasa más alta, con un 84,27 % de reprobados y apenas un 15,73 % de aprobados. Estos resultados confirman que la reprobación es un fenómeno generalizado a nivel urbano, aunque con variaciones entre ciudades.

Finalmente, el gráfico correspondiente a la cantidad de aprobados y reprobados según el tipo de colegio permite identificar diferencias significativas entre las distintas categorías. Los colegios particulares urbanos presentan la mayor proporción de aprobados, con un 30,60 %, seguidos por los colegios de convenio urbano, con 28,95 %. En contraste, los colegios fiscales rurales y comunitarios rurales exhiben las tasas más bajas de aprobación, con 18,97 % y 14,62 %, respectivamente, y niveles de reprobación que superan el 80 %. Asimismo, los colegios comunitarios urbanos registran una tasa de reprobación del 75,39 %, mientras que los fiscales urbanos alcanzan un 76,97 %. En conjunto, estos resultados evidencian una marcada brecha en los resultados del examen de ingreso asociada tanto al tipo de gestión educativa como al contexto urbano o rural del establecimiento de procedencia.

4.2. Resultados del desarrollo de modelos predictivos

Se desarrollaron modelos predictivos para identificar postulantes con mayor probabilidad de reprobación el examen de ingreso a la Facultad de Ciencias y Tecnología de la UMSS, a partir del uso de datos históricos. Para este fin, se implementaron tres algoritmos de aprendizaje automático utilizados: XGBoost, Random Forest y Regresión Logística Binaria. Estos modelos fueron seleccionados con el objetivo de comparar el desempeño de técnicas de distinta complejidad.

El desarrollo de los modelos se basó en la utilización de variables demográficas y académicas de los postulantes, así como de variables derivadas generadas durante el análisis exploratorio de datos, tales como la edad, el tipo de colegio, la condición laboral, el origen geográfico y la tasa de aprobación del colegio de procedencia. La selección de estas variables se fundamenta en los patrones y asociaciones identificados previamente en el análisis exploratorio, garantizando coherencia entre ambas etapas del estudio.

Previo al entrenamiento, se aplicaron técnicas de procesamiento de datos que incluyeron la codificación de variables categóricas, el escalado de variables numéricas y el tratamiento del desbalance de clases presente en el conjunto de datos, para que después los modelos sean entrenados utilizando una partición de los datos en conjuntos de entrenamiento y prueba, asegurando condiciones iguales para su comparación posterior. Como resultado de este proceso, se obtuvieron modelos predictivos funcionales capaces de generar predicciones sobre la probabilidad de reprobación de los postulantes, los cuales constituyen la base para la evaluación cuantitativa de su desempeño presentada en el siguiente apartado.

4.3. Resultados de la evaluación del desempeño de los modelos predictivos

4.3.1. Resultados del modelo XGBoost

Utilizando la librería de Sklearn se obtienen las métricas de desempeño del modelo, en la Figura 4-7 se muestran los resultados, del conjunto de entrenamiento y de prueba.

```

[[16002 8090]
 [ 1950 5876]]
--
--
precision    recall  f1-score   support

      0       0.89       0.66       0.76      24092
      1       0.42       0.75       0.54       7826

 accuracy          0.69      31918
  macro avg       0.66       0.71       0.65      31918
  weighted avg    0.78       0.69       0.71      31918

Train Accuracy: 0.6854439501221881
Train AUC: 0.78438492752397

```

Figura 4-7: Resultados del conjunto de entrenamiento de XGBOOST

Fuente: *Elaboración propia, 2025*

Para interpretar la Figura 4-7 se consideran dos clases: 0 (reprobados) y 1 (aprobados). La matriz de confusión del entrenamiento muestra que, de 31 918 registros, XGBoost clasificó correctamente 16 002 casos de la clase 0 y 5 876 de la clase 1; se identificaron además 8 090 falsos positivos (clase 1 que eran 0) y 1 950 falsos negativos (clase 1 clasificados como 0). Con ello el modelo reconoce la clase mayoritaria o sea los reprobados aunque sobreestima los aprobados.

En las métricas de entrenamiento se obtuvo accuracy 0.69; para la clase 0, precisión 0.89, recall 0.66 y f1-score de 0.76, indicando detección adecuada con omisiones moderadas. En la clase 1 la precisión fue 0.42, el recall 0.75 y el f1-score de 0.54, logrando identificar la mayoría de positivos, pero con errores relevantes. confirmando desempeño desigual. El AUC 0.78 evidencia buena discriminación del modelo.

Finalmente, en la Figura 4-8 se presenta la evaluación del conjunto de prueba para contrastar estos resultados.

```

[[8268 4706]
 [1341 2873]]
--
--
precision    recall  f1-score   support

      0       0.86       0.64       0.73      12974
      1       0.38       0.68       0.49       4214

 accuracy          0.65      17188
  macro avg       0.62       0.66       0.61      17188
  weighted avg    0.74       0.65       0.67      17188

Test Accuracy: 0.648184780079125
Test AUC: 0.7276282512818708

```

Figura 4-8: Resultados del conjunto de prueba de XGBOOST

Fuente: *Elaboración propia, 2025*

Para el conjunto de prueba, la matriz de confusión muestra que, de 17 188 registros evaluados, el modelo clasificó correctamente 8 268 casos de la clase 0 y 2 873 casos de la clase 1. Sin embargo, se registraron 4 706 falsos positivos y 1 341 falsos negativos. Estos resultados indican una disminución moderada del desempeño respecto al conjunto de entrenamiento, lo cual es esperable al evaluar el modelo sobre datos no utilizados durante el proceso de aprendizaje.

Las métricas calculadas sobre el conjunto de prueba reflejan una exactitud del 0.65, lo que representa una ligera reducción frente al entrenamiento. Para la clase 0, se obtuvo una precisión del 0.86 y un recall del 0.64, mientras que para la clase 1 la precisión fue del 0.38 y el recall del 0.68. Esto confirma que el modelo mantiene una buena sensibilidad para detectar la clase positiva, aunque con una menor precisión, generando un número significativo de falsos positivos. El F1-score alcanzó valores de 0.73 para la clase 0 y 0.49 para la clase 1, mostrando un desempeño aceptable pero limitado en la clase minoritaria. El AUC en el conjunto de prueba fue de 0.73, esto indica que el modelo conserva una capacidad adecuada para diferenciar entre ambas clases al generalizar a nuevos datos.

De forma global, el modelo XGBoost presentó una exactitud del 0.685 en el conjunto de entrenamiento y del 0.648 en el conjunto de prueba, junto con una reducción del AUC de 0.78 a 0.73. Estas diferencias sugieren una generalización razonable del modelo, sin sobreajuste. En conjunto, los resultados indican que el modelo es especialmente efectivo para identificar postulantes en riesgo, aunque existe margen de mejora en la precisión de la clase positiva.

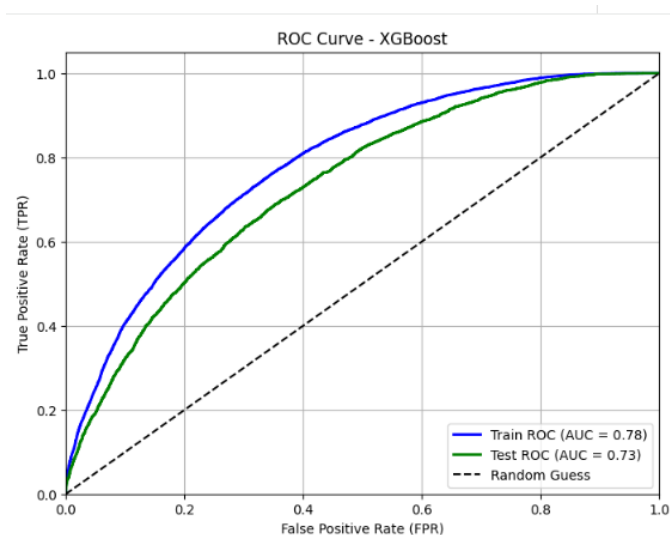


Figura 4-9: Grafica de ROC del modelo XGBoost

Fuente: *Elaboración propia, 2025*

El gráfico de la curva ROC evidencia que tanto el conjunto de entrenamiento (representado por la línea azul) como el conjunto de prueba (línea verde) presentan un comportamiento muy similar y se mantienen claramente por encima de la línea diagonal que representa una clasificación aleatoria. Este

comportamiento indica que el modelo XGBoost posee una capacidad consistente para diferenciar entre postulantes aprobados y reprobados a lo largo de distintos umbrales de decisión, independientemente del conjunto de datos evaluado.

El valor del área bajo la curva (AUC) alcanza aproximadamente 0.78 en el conjunto de entrenamiento y 0.73 en el conjunto de prueba, lo que refleja una capacidad de discriminación adecuada del modelo. Estos valores indican que, en promedio, el modelo tiene una probabilidad significativamente mayor de asignar una puntuación más alta a un postulante aprobado que a uno reprobado. La diferencia moderada entre ambos valores sugiere que el modelo ha logrado aprender patrones relevantes sin ajustarse de manera excesiva a los datos de entrenamiento, manteniendo así una buena capacidad de generalización cuando se enfrenta a datos no vistos. En conjunto, el análisis de la curva ROC confirma que el modelo XGBoost ofrece un desempeño estable y confiable como herramienta predictiva dentro del contexto del estudio.

4.3.2. Resultados del modelo Random forest

Utilizando la librería scikit-learn se obtienen las métricas de desempeño del modelo, para evaluar la capacidad predictiva. En la Figura 4-10 se presentan los resultados correspondientes tanto al conjunto de entrenamiento como al conjunto de prueba, haciendo comparación del comportamiento del modelo en ambas fases. Estas métricas incluyen indicadores clave como exactitud, precisión, recall, F1-score y AUC-ROC.

Para entender la Figura 4-10 se tiene 2 clases: clase 0 que representa a los reprobados y la clase 1 que representa a los aprobados.

```
[[14837  9255]
 [ 3179 4647]]
-1      -1-
precision  recall  f1-score  support

      0      0.82      0.62      0.70      24092
      1      0.33      0.59      0.43      7826

accuracy          0.61      31918
macro avg          0.58      0.60      0.57      31918
weighted avg       0.70      0.61      0.64      31918

Accuracy TRAIN: 0.6104392505796102
AUC TRAIN: 0.6539258938571748
```

Figura 4-10: Resultados del conjunto de entrenamiento de Random Forest

Fuente: *Elaboración propia, 2025*

En el conjunto de entrenamiento, la matriz de confusión muestra que, de un total de 31 918 registros, el modelo clasificó correctamente 14 837 observaciones de la clase 0 y 4 647 de la clase 1. Sin embargo, se produjeron 9 255 falsos positivos, correspondientes a casos de la clase 0 clasificados erróneamente como clase 1, y 3 179 falsos negativos, es decir, registros de la clase 1 que el modelo no logró identificar

correctamente. Estos resultados evidencian que el modelo presenta mayores dificultades para discriminar con precisión la clase positiva.

Las métricas de evaluación en el conjunto de entrenamiento reflejan una exactitud del 0.61, lo que indica que poco más de seis de cada diez predicciones fueron correctas. Para la clase 0 se obtuvo una precisión del 0.82 y un recall del 0.62, lo que sugiere un buen nivel de confiabilidad cuando el modelo predice esta clase, aunque con una proporción moderada de casos no detectados. En contraste, la clase 1 presentó una precisión del 0.33 y un recall del 0.59, lo que indica que, si bien el modelo identifica más de la mitad de los casos positivos, lo hace con un número elevado de errores. El F1-score fue de 0.70 para la clase 0 y de 0.43 para la clase 1, confirmando un desempeño inferior en la clase minoritaria. El valor del AUC en entrenamiento fue de 0.65, lo que evidencia una capacidad de discriminación moderada.

Siguiendo con los resultados, se observa la evaluación del conjunto de prueba (ver Figura 4-11).

```

[[7973 5001]
 [1739 2475]]

              precision    recall  f1-score   support

     0       0.82         0.61         0.70      12974
     1       0.33         0.59         0.42       4214

 accuracy                   0.61      17188
 macro avg              0.58         0.60         0.56      17188
 weighted avg           0.70         0.61         0.63      17188

Accuracy TEST: 0.6078659529904584
AUC TEST: 0.6406422881907072

```

Figura 4-11: Resultados del conjunto de prueba de Random Forest

Fuente: *Elaboración propia, 2025*

En el conjunto de prueba, la matriz de confusión indica que, de 17 188 registros evaluados, el modelo clasificó correctamente 7 973 casos de la clase 0 y 2 475 de la clase 1. No obstante, se registraron 5 001 falsos positivos y 1 739 falsos negativos, lo que confirma un comportamiento similar al observado durante el entrenamiento, con una reducción leve en el desempeño al aplicarse sobre datos no utilizados en el proceso de aprendizaje.

Las métricas del conjunto de prueba muestran una exactitud del , valor muy cercano al obtenido en el entrenamiento, lo que sugiere una generalización estable del modelo. Para la clase 0, la precisión se mantuvo en 82% y el recall en 61%, mientras que para la clase 1 la precisión fue del 33% y el recall del 59%. El F1-score alcanzó 0.70 para la clase 0 y 0.42 para la clase 1, reflejando consistencia en los resultados, aunque con limitaciones claras en la identificación precisa de la clase positiva. El AUC en el

conjunto de prueba fue de 0.64, indicando que el modelo conserva una capacidad de discriminación moderada frente a datos nuevos.

De manera global, el modelo presentó valores de accuracy del 0.61 en entrenamiento y del 0.60 en prueba, junto con valores de AUC de 0.65 y 0.64, respectivamente. La similitud entre las métricas de ambos conjuntos sugiere que el modelo no presenta sobreajuste significativo; sin embargo, el desempeño general es moderado. En particular, los resultados muestran que el modelo es más confiable al identificar la clase negativa, mientras que la baja precisión en la clase positiva evidencia la necesidad de ajustes adicionales para mejorar su capacidad predictiva (ver Figura 4-12).

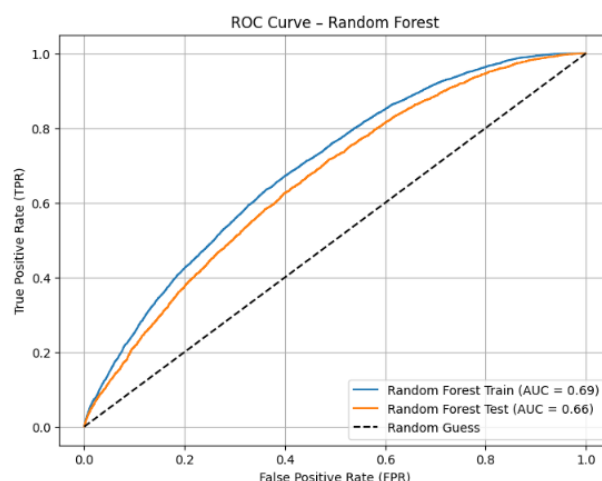


Figura 4-12: Grafica de curva de ROC del modelo Random Forest

Fuente: *Elaboración propia, 2025*

Se muestra la curva ROC correspondiente al modelo Random Forest para los conjuntos de entrenamiento y prueba. Se observa que ambas curvas se sitúan por encima de la línea de clasificación aleatoria, lo que indica que el modelo presenta una capacidad de discriminación superior al azar para diferenciar entre postulantes aprobados y reprobados. El valor del área bajo la curva (AUC) alcanza aproximadamente 0.69 en el conjunto de entrenamiento y 0.66 en el conjunto de prueba, reflejando un desempeño moderado del modelo. La diferencia reducida entre ambos valores sugiere una adecuada capacidad de generalización, aunque con un nivel de discriminación inferior en comparación con modelos más complejos. En general, el modelo Random Forest muestra un comportamiento estable, pero con una capacidad predictiva limitada para separar de forma más precisa las clases evaluadas.

4.3.3. Resultados del modelo Regresión logística binaria

Utilizando la librería scikit-learn se obtienen las métricas de desempeño del modelo, para evaluar la capacidad predictiva. En la Figura 4-10 se presentan los resultados correspondientes tanto al conjunto

de entrenamiento como al conjunto de prueba, haciendo comparación del comportamiento del modelo en ambas fases. Estas métricas incluyen indicadores clave como exactitud, precisión, recall, F1-score y AUC-ROC.

Para entender la Figura 4-13 se tiene 2 clases: clase 0 que representa a los reprobados y la clase 1 que representa a los aprobados.

```

[[15410  8682]
 [ 3154 4672]]
-      --
      precision    recall  f1-score   support

      0        0.83        0.64        0.72    24092
      1        0.35        0.60        0.44     7826

 accuracy          0.63    31918
 macro avg         0.59        0.62        0.58    31918
 weighted avg      0.71        0.63        0.65    31918

Train Accuracy: 0.6291747603233285
Train AUC: 0.6744513794955609

```

Figura 4-13: Resultados del conjunto de entrenamiento de Regresión Logística Binaria

Fuente: *Elaboración propia, 2025*

En el conjunto de entrenamiento, la matriz de confusión indica que, de un total de 31 918 registros, el modelo clasificó correctamente 15 410 observaciones de la clase 0 y 4 672 de la clase 1. No obstante, se registraron 8 682 falsos positivos, correspondientes a casos de la clase 0 clasificados erróneamente como clase 1, y 3 154 falsos negativos, es decir, casos de la clase 1 que no fueron detectados correctamente. Estos valores muestran que el modelo logra identificar una parte importante de ambas clases, aunque presenta dificultades para reducir los errores asociados a la clase positiva.

Las métricas del conjunto de entrenamiento reflejan una exactitud del 0.629, lo que implica que aproximadamente seis de cada diez predicciones fueron correctas. Para la clase 0, se obtuvo una precisión del 0.83 y un recall del 0.64, evidenciando un buen desempeño al predecir correctamente esta clase, aunque con una fracción de casos reales no identificados. En la clase 1, la precisión fue del 0.35 y el recall del 0.60, lo que indica que el modelo detecta más de la mitad de los casos positivos, pero con un nivel considerable de predicciones incorrectas. El F1-score fue de 0.72 para la clase 0 y 0.44 para la clase 1, confirmando un rendimiento inferior en la clase minoritaria. El AUC en entrenamiento fue de 0.67, lo que sugiere una capacidad de discriminación moderada.

Siguiendo con los resultados, se observa la evaluación del conjunto de prueba (ver Figura 4-14).

```

[[8308 4666]
 [1762 2452]]

      precision    recall  f1-score   support

     0       0.83       0.64       0.72      12974
     1       0.34       0.58       0.43       4214

 accuracy          0.63      17188
 macro avg          0.58       0.61       0.58      17188
 weighted avg       0.71       0.63       0.65      17188

Test Accuracy: 0.6260181521992088
Test AUC: 0.6622260255606682

```

Figura 4-14: Resultados del conjunto de entrenamiento de Regresión Logística Binaria

Fuente: *Elaboración propia, 2025*

La matriz de confusión indica, de 17 188 registros, el modelo clasificó correctamente 8 308 casos de la clase 0 y 2 452 de la clase 1, mientras que se identificaron 4 666 falsos positivos y 1 762 falsos negativos, comportamiento consistente con el entrenamiento y con la variación esperable en datos no vistos. Las métricas de prueba presentan exactitud 0.626, con precisión 0.83 y recall 0.64 para la clase 0, y precisión 0.34 junto a recall 0.58 para la clase 1; el F1-score alcanzó 0.72 en la clase 0 y 0.43 en la clase 1, reflejando generalización cercana al entrenamiento, aunque con limitaciones en la identificación de la clase positiva. Finalmente, el AUC fue 0.67, confirmando una capacidad de discriminación moderada del modelo.

En general, el modelo presentó valores de accuracy cercanos al 0.63 tanto en entrenamiento como en prueba, junto con valores de AUC alrededor de 0.67, lo que sugiere que no existe un sobreajuste significativo. Sin embargo, los resultados muestran que el modelo es claramente más confiable para identificar la clase negativa, mientras que la baja precisión en la clase positiva indica la necesidad de ajustes adicionales, como la optimización del umbral de decisión o el uso de estrategias de balanceo más avanzadas para mejorar su desempeño predictivo(ver Figura 3-15).

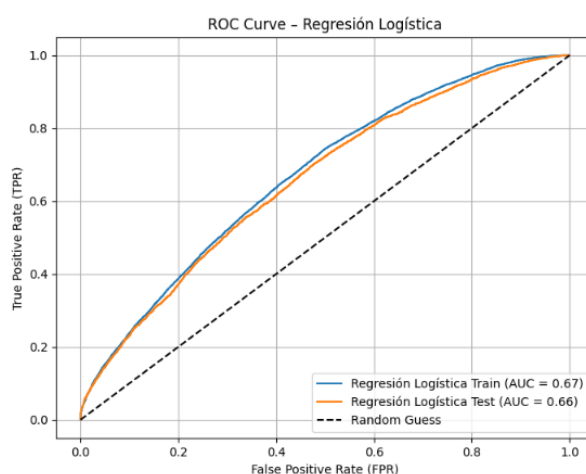


Figura 4-15: Grafica de curva de ROC del modelo de Regresión logística binaria

Fuente: *Elaboración propia, 2025*

Se presenta la curva ROC del modelo de Regresión Logística para los conjuntos de entrenamiento y prueba. Se observa que ambas curvas se sitúan por encima de la línea de clasificación aleatoria, lo que indica que el modelo posee una capacidad de discriminación superior al azar para diferenciar entre postulantes aprobados y reprobados. El valor del área bajo la curva (AUC) alcanza aproximadamente 0.67 en el conjunto de entrenamiento y 0.66 en el conjunto de prueba, evidenciando un desempeño moderado del modelo. La similitud entre ambos valores sugiere una adecuada capacidad de generalización, aunque con una capacidad discriminativa limitada en comparación con modelos más complejos. En conjunto, la Regresión Logística muestra un comportamiento estable, pero con un menor poder predictivo para capturar relaciones no lineales presentes en los datos.

4.4. Resultados de la Comparación y Selección de modelos desarrollados

En la Figura 4-12 se presenta la comparación de las métricas de desempeño obtenidas por los modelos de Regresión Logística, Random Forest y XGBoost sobre el conjunto de prueba. Los valores de las métricas se encuentran en un rango entre 0 y 1, donde valores más cercanos a 1 indican un mejor desempeño del modelo, reflejando una mayor capacidad predictiva, mientras que valores próximos a 0 representan un desempeño bajo, asociado a una menor efectividad en la clasificación de los postulantes. Esta comparación ayuda a analizar el comportamiento relativo de cada modelo en términos de precisión, recall, F1-score, exactitud y área bajo la curva ROC, con el fin de identificar el modelo con mejor desempeño para la identificación de postulantes en riesgo académico.

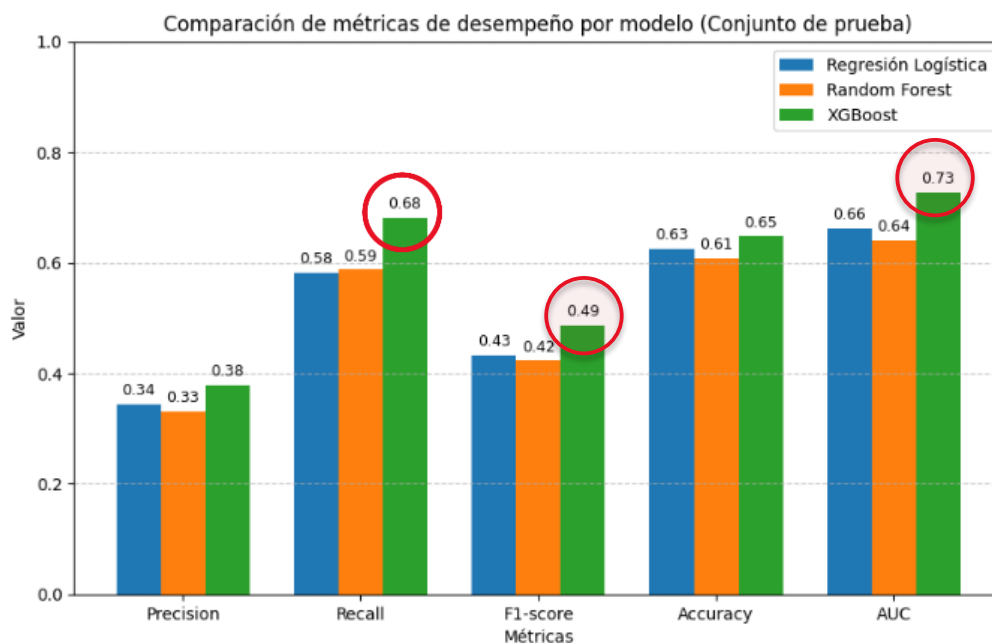


Figura 4-16: Comparación de métricas de desempeño de los modelos

Fuente: *Elaboración propia, 2025*

En cuanto a precisión, el modelo XGBoost alcanza el valor más alto (0.38), superando a Random Forest (0.33) y a la Regresión Logística (0.34), lo que indica una mayor confiabilidad en las predicciones positivas realizadas por este modelo. Respecto al recall, XGBoost muestra un desempeño notablemente superior (0.68), en comparación con la Regresión Logística (0.58) y Random Forest (0.59), evidenciando una mejor capacidad para identificar correctamente a los postulantes en riesgo académico, aspecto especialmente relevante dado que el objetivo principal del estudio es minimizar la omisión de casos de riesgo.

En cuanto al F1-score, que refleja el equilibrio entre precisión y recall, el modelo XGBoost vuelve a presentar el mejor resultado (0.49), superando a la Regresión Logística (0.43) y Random Forest (0.42).

Asimismo, al analizar la exactitud (accuracy), los tres modelos presentan valores similares, aunque XGBoost obtiene el valor más alto (0.65), seguido de Random Forest (0.63) y Regresión Logística (0.61), lo que indica un desempeño global ligeramente superior.

Finalmente, el AUC-ROC confirma la mayor capacidad discriminatoria del modelo XGBoost, con un valor de 0.73, frente a 0.64 obtenido por Random Forest y Regresión Logística (0.63). En conjunto, estos resultados muestran que, aunque todos los modelos presentan un desempeño aceptable, XGBoost sobresale de manera consistente en las métricas más relevantes, lo que respalda su selección como el modelo predictivo con mejor desempeño para el problema abordado.

En la Tabla 4-1 se muestra la tabla comparativa de las métricas de desempeño de los tres modelos de aprendizaje automático.

Tabla 4-1: Tabla de comparación de las métricas de los modelos evaluados.

Indicador	XGBOOST	Random Forest	Regresión Logística B.
Precisión	0.38	0.33	0.34
Recall	0.68	0.59	0.58
F1-score	0.49	0.42	0.43
Accuracy	0.65	0.61	0.63
AUC	0.73	0.64	0.66

Fuente: *Elaboración propia, 2025*

De acuerdo con los resultados presentados en la Tabla 4-7, el modelo XGBoost ha obtenido la mayor precisión, con un valor de 0.38, lo que indica que, entre todos los casos clasificados como positivos por el modelo, el 38% corresponde efectivamente a postulantes aprobados. Este resultado evidencia que XGBoost presenta una mejor capacidad para realizar predicciones acertadas cuando identifica casos positivos, reduciendo la proporción de falsos positivos en comparación con los otros modelos evaluados.

En términos de recall, el modelo XGBoost también alcanza el valor más alto, con 0.68, lo que significa que logra identificar correctamente el 69% de los casos positivos reales. Esto indica que el modelo

presenta una mayor capacidad para detectar a los postulantes aprobados, disminuyendo la cantidad de falsos negativos, lo cual resulta relevante en el contexto del análisis del rendimiento académico.

Respecto al F1-score, que representa el equilibrio entre precisión y recall, el modelo XGBoost obtiene el mejor desempeño con un valor de 0.49, superando a los modelos de Regresión Logística y Random Forest, ambos con un valor de 0.43 y 0.42 respectivamente. Este resultado sugiere que XGBoost mantiene un mejor balance entre la correcta identificación de casos positivos y la reducción de errores de clasificación, siendo especialmente adecuado para escenarios con desbalance de clases.

En cuanto a la exactitud (accuracy), el modelo XGBoost presenta el valor más alto con 0.65, lo que indica que aproximadamente el 65% de las predicciones totales realizadas por el modelo son correctas. Aunque los modelos de Random Forest y Regresión Logística presentan valores cercanos, XGBoost logra una mayor proporción de aciertos generales.

Finalmente, al analizar el Área Bajo la Curva ROC (AUC), el modelo XGBoost alcanza un valor de 0.73, superior al de los modelos de Random Forest y Regresión Logística, ambos con 0.64 y 0.66 respectivamente. Este resultado evidencia que XGBoost tiene una mayor capacidad discriminativa para diferenciar entre postulantes aprobados y reprobados, independientemente del umbral de decisión utilizado.

En conjunto, los resultados obtenidos muestran que el modelo XGBoost presenta el mejor desempeño, destacando especialmente en las métricas de F1-score y AUC, las cuales son particularmente relevantes en problemas de clasificación con desbalance de clases.

Por tanto, XGBoost es seleccionado como el modelo final del estudio, por ofrecer un mejor equilibrio entre precisión, capacidad de detección y poder discriminativo, aunque los valores obtenidos indican que el modelo debe ser interpretado como una herramienta de apoyo a la identificación de perfiles con probabilidad de aprobación en el examen de ingreso, y no como un sistema de predicción absoluta, debido a las limitaciones inherentes a los datos y a la complejidad del fenómeno analizado.

Si bien el modelo XGBoost presenta el mejor desempeño global en comparación con la Regresión Logística y el Random Forest, los valores obtenidos en las métricas evaluadas indican que el desempeño predictivo aún es moderado. Esto sugiere que, a pesar de la capacidad del modelo para capturar relaciones no lineales y manejar el desbalance de clases, la información contenida en las variables utilizadas no aporta un peso suficientemente significativo para lograr una separación más precisa entre las clases. En particular, muchas de las variables disponibles corresponden a características demográficas y académicas que, de manera individual o conjunta, presentan una capacidad limitada para explicar el resultado del examen. Como consecuencia, el modelo alcanza un desempeño aceptable pero no óptimo, lo que evidencia que los resultados obtenidos están condicionados principalmente por la calidad y el poder explicativo de las variables, más que por la complejidad del algoritmo empleado.

Tabla 4-2: Modelo seleccionado.

Indicador	XGBOOST
Precisión	0.39
Recall	0.69
F1-score	0.50
Accuracy	0.66
AUC	0.73

Fuente: *Elaboración propia, 2025*

4.5. Discusión de Resultados

Antes de presentar la comparación de resultados, se pone en contexto el estudio de referencia utilizado. El trabajo desarrollado por (Contreras y otros, 2020) tiene como objetivo la predicción del éxito o fracaso académico de estudiantes universitarios mediante técnicas de aprendizaje automático, empleando diversos algoritmos de clasificación y un conjunto amplio de variables académicas. Dicho estudio sirve como antecedente para el presente proyecto, permitiendo contrastar metodologías, métricas de evaluación y niveles de desempeño obtenidos en contextos educativos distintos.

Los resultados obtenidos por (Contreras y otros, 2020) demuestran que es posible predecir el éxito o fracaso académico de estudiantes universitarios mediante la aplicación de técnicas de aprendizaje automático, utilizando algoritmos de clasificación como Árboles de Decisión, K-Nearest Neighbors y Perceptrón. En dicho estudio, el autor reporta un desempeño adecuado de los modelos evaluados, respaldado por el análisis de matrices de confusión y porcentajes de clasificación correcta, los cuales se presentan en la Figura 4-9.

Tabla 4-3: Resultados del modelo predictivo de éxito/fracaso académico en Contreras (2020).

Algoritmo	Exactitud con todas las características	Exactitud con características seleccionadas	Precisión	Recall	F1
SVN	0.54	0.66	0.61	0.66	0.54
KNN	0.54	0.64	0.55	0.64	0.57
Perceptron	0.53	0.6624	0.61	0.66	0.54
Arbol de Decision	0.55	0.65	0.55	0.64	0.57

Fuente: *Contreras, 2020*

A continuación, se revisa el desempeño de los modelos predictivos desarrollados en el presente trabajo, se evaluaron los algoritmos XGBoost, Random Forest y Regresión Logística Binaria utilizando el conjunto de datos correspondiente al examen de ingreso universitario. La evaluación se realizó empleando métricas estándar de clasificación, tales como precisión, recall, F1-score, exactitud (accuracy) y área bajo la curva ROC (AUC), las cuales permiten medir de forma integral la capacidad predictiva de cada modelo y facilitar su comparación con estudios previos.

Tabla 4-4: Resultados comparativos de los modelos aplicados

Indicador	XGBoost	Random Forest	Regresión Logística B.
Precisión	0.38	0.33	0.34
Recall	0.68	0.59	0.58
F1-score	0.49	0.42	0.43
Accuracy	0.65	0.61	0.63
AUC	0.73	0.64	0.66

Fuente: *Elaboración propia, 2025*

El análisis de los resultados presentados en la Tabla 4-11 evidencia que el modelo XGBoost obtiene el mejor desempeño relativo entre los algoritmos evaluados que destaca por alcanzar el valor más alto de recall (0,68), lo que indica una mayor capacidad para identificar correctamente a los postulantes en riesgo académico, aspecto prioritario dentro de los objetivos del estudio. Asimismo, el F1-score (0,49) obtenido por XGBoost refleja un mejor equilibrio entre precisión y capacidad de detección en comparación con Random Forest y Regresión Logística, cuyos valores se sitúan en 0,43.

En términos de exactitud (accuracy), los tres modelos presentan valores similares, aunque XGBoost mantiene una ligera ventaja con 0,66, seguido por Random Forest (0,63) y Regresión Logística (0,62), lo que sugiere un desempeño global comparable. Finalmente, el AUC-ROC confirma la mayor capacidad discriminatoria del modelo XGBoost, alcanzando un valor de 0,73, frente a 0,66 obtenido por los otros dos modelos. En conjunto, estos resultados respaldan la selección de XGBoost como el modelo más adecuado dentro de las limitaciones del conjunto de datos analizado, ya que ofrece un mejor balance entre detección de casos de riesgo y estabilidad en las predicciones.

4.5.1. Análisis comparativo y discusión entre este proyecto y el de referencia

Al comparar los resultados obtenidos en el presente estudio con los reportados por (Contreras y otros, 2020), se observa que ambos trabajos presentan un comportamiento similar en cuanto al uso de modelos de clasificación supervisada para la predicción del rendimiento académico; sin embargo, existen diferencias en los niveles de desempeño alcanzados. En el estudio de (Contreras y otros, 2020), los algoritmos evaluados alcanzan valores de exactitud con características seleccionadas entre 0,64 y 0,66, así como valores de recall cercanos a 0,64–0,66 y F1-score entre 0,54 y 0,57, lo que evidencia una mayor capacidad para clasificar correctamente a los estudiantes en las categorías de éxito y fracaso académico.

En contraste, los modelos desarrollados en el presente estudio presentan valores de desempeño más moderados. Si bien el modelo XGBoost alcanza una exactitud de 0,66, comparable con los valores reportados por (Contreras y otros, 2020), los valores de F1-score (0,49) son ligeramente inferiores. No obstante, se destaca que XGBoost obtiene un recall de 0,68, valor que resulta incluso superior a los

reportados en el estudio de referencia, lo que indica una mayor capacidad para identificar correctamente a los postulantes en riesgo académico, aspecto prioritario dentro de los objetivos de esta investigación.

Estas diferencias pueden explicarse principalmente por la disponibilidad y calidad de las variables predictoras. Mientras que el estudio de referencia dispone de un conjunto de datos más amplio y detallado, con variables académicas de alto poder explicativo, el presente estudio se encuentra limitado por la información disponible. Esta restricción reduce la capacidad predictiva de los modelos, lo que evidencia que el desempeño alcanzado depende no solo del algoritmo utilizado, sino principalmente de la información contenida en los datos analizados.

Desde una perspectiva metodológica, los resultados obtenidos confirman que el desempeño de un modelo predictivo no depende únicamente del algoritmo utilizado, sino fundamentalmente de la calidad, relevancia y representatividad de los datos disponibles. A pesar de estas limitaciones, el modelo XGBoost presenta el mejor equilibrio entre capacidad de detección (recall), estabilidad del desempeño (F1-score) y poder discriminativo ($AUC = 0,73$), superando a Random Forest y Regresión Logística en las métricas más relevantes para el problema abordado.

En consecuencia, y considerando tanto la comparación con estudios previos como las particularidades del conjunto de datos analizado, se selecciona el modelo XGBoost como el más adecuado para el presente estudio. Si bien sus resultados no superan completamente los valores reportados por el estudio de referencia, el desempeño alcanzado es coherente con las características de los datos disponibles y permite cumplir con el objetivo principal de identificar postulantes con mayor probabilidad de fracaso académico, proporcionando una herramienta de apoyo para el análisis y la toma de decisiones académicas.

4.6. Resultados del desarrollo de la interfaz interactiva

Como resultado del desarrollo, se implementó un prototipo funcional que integra el análisis estadístico, el modelo predictivo y una interfaz gráfica de usuario, con el objetivo de facilitar la aplicación práctica de los resultados obtenidos. El prototipo permite evaluar de manera individual y masiva a postulantes, proporcionando información sobre la probabilidad de aprobación o reprobación en el examen de ingreso a la FCyT–UMSS.

4.6.1. Interfaz del modelo predictivo

La Figura 4-13 muestra la interfaz principal del prototipo correspondiente al módulo *Modelo Predictivo*. En esta vista, el usuario puede ingresar las variables sociodemográficas y académicas del postulante, tales como el nombre del colegio, período académico, género, edad, opción de ingreso, procedencia geográfica, tipo de colegio, condición laboral y migración universitaria previa. Una vez ingresados los datos, el sistema permite ejecutar la predicción mediante el botón *Predecir Resultado*, mostrando de forma inmediata el resultado estimado (aprobado o no aprobado) junto con la probabilidad asociada generada

por el modelo XGBoost. Adicionalmente, se presenta la tasa histórica de aprobación del colegio de procedencia, lo que brinda un contexto adicional para la interpretación del resultado.

Variable	Valor
Nombre del colegio	ADELA HOYOS MORALES
Período de la gestión académica	1
Género del postulante	MASCULINO
Edad del postulante en el examen	17
Opción de ingreso	1
Ciudad del colegio	COCHABAMBA
Provincia del colegio	ALONSO DE IBAÑES
Año de egreso de bachillerato	2004
Municipio de residencia	BETANZOS
Nacionalidad	BOLIVIA
Estado civil	SOLTERO(A)
Años posteriores al bachillerato	1
Tipo de colegio	NO PARTICULAR URBANO
Mayor de edad	Sí
Migración universitaria previa	No

Tasa de aprobación del colegio: 66.67%

Predecir Resultado

Complete los datos y presione el botón

Figura 4-17: Interfaz del módulo de predicción individual del prototipo desarrollado.

Fuente: *Elaboración propia, 2025*

La imagen presenta la interfaz del módulo de Modelo Predictivo del sistema desarrollado, el cual permite realizar la evaluación individual de un postulante a partir de la información ingresada por el usuario. La interfaz se organiza mediante un formulario estructurado que recopila variables demográficas, académicas y contextuales relevantes, tales como el nombre y tipo de colegio, período de gestión académica, género, edad del postulante al momento del examen, opción de ingreso, ciudad y provincia del colegio, año de egreso de bachillerato, municipio de residencia, nacionalidad, estado civil y años posteriores al bachillerato, entre otras. Esta disposición facilita una entrada de datos ordenada y coherente con las variables utilizadas durante el entrenamiento del modelo predictivo.

Asimismo, el sistema incorpora variables derivadas y de apoyo al análisis, como la condición de mayoría de edad, la existencia de migración universitaria previa y la tasa de aprobación histórica del colegio, la cual se calcula automáticamente y se muestra como información contextual adicional. Una vez completados los datos, el usuario puede ejecutar la predicción mediante el botón “Predecir Resultado”. El sistema reenvía de forma inmediata a la pantalla de Perfil estadístico, en el cual está el resultado de la clasificación, indicando si el postulante se encuentra aprobado o en riesgo, junto con la probabilidad estimada asociada a dicha predicción. Esta presentación clara y directa de los resultados permite una

interpretación rápida por parte del usuario, consolidando la interfaz como una herramienta práctica de apoyo para la evaluación académica individual.

4.6.2. Perfil estadístico del postulante

Con el fin de complementar el resultado del modelo predictivo, el prototipo incorpora un módulo de perfil estadístico, el cual se presenta en la Figura 4-14. Este módulo permite comparar las características del postulante con valores históricos del conjunto de datos, incluyendo el percentil de la tasa histórica de aprobación del colegio, la comparación de tasas de aprobación, los años posteriores al bachillerato y un nivel de riesgo estimado para rendir el examen.

Esta funcionalidad aporta una interpretación adicional al resultado del modelo, permitiendo contextualizar la predicción dentro del comportamiento estadístico histórico de los postulantes.



Figura 4-18: Interfaz del módulo de predicción individual del prototipo desarrollado.

Fuente: *Elaboración propia, 2025*

La Figura 4-14 corresponde al Perfil Estadístico del Postulante, el cual presenta un resumen que complementa el resultado del modelo predictivo. Esta sección permite contextualizar la predicción realizada dando una visión comparativa entre las características del postulante evaluado y los valores históricos del conjunto de datos utilizado en el estudio.

Se muestra el percentil de la tasa histórica de aprobación del colegio para ubicar al postulante dentro de la distribución general de desempeño entre las instituciones educativas. Asimismo, se presenta una

comparación directa entre la tasa de aprobación del postulante y el promedio histórico para estimar si el postulante se encuentra por encima o por debajo del comportamiento promedio. De manera similar, se incluye una comparación de los años posteriores al bachillerato, contrastando el valor individual con el promedio histórico, aportando información adicional sobre la trayectoria académica del postulante. Finalmente, el módulo muestra un nivel de riesgo estimado para rendir el examen, expresado de forma visual mediante una categoría cualitativa y un indicador gráfico.

4.6.3. Evaluación masiva de postulantes

El prototipo también incorpora una funcionalidad de evaluación masiva, orientada al análisis simultáneo de múltiples postulantes. En la Figura 4-15 se observa la interfaz correspondiente a este módulo, donde el usuario puede cargar un archivo en formato Excel que contiene registros de postulantes.

Tras la carga del archivo, el sistema procesa automáticamente cada registro y genera una tabla de resultados que incluye información como nombre del colegio, género, edad, predicción y probabilidad asociada. Esta funcionalidad permite analizar grandes volúmenes de datos de forma eficiente, facilitando el uso del modelo como herramienta de apoyo al análisis institucional.

Evaluación Masiva de Postulantes						
Cargar archivo Excel (.xlsx)						
	SEXO	EDAD	NOMBRE_COLEGIO	PREDICCIÓN	PROBABILIDAD	PERFIL
1	MASCULINO	20	GERARDO VARGAS VIDAL	EN RIESGO	5.04%	Ver perfil
2	FEMENINO	25	IRLANDES	FUERA DE RIESGO	74.64%	Ver perfil
3	MASCULINO	18	MARIANO RICARDO TERRAZAS	EN RIESGO	6.95%	Ver perfil
4	MASCULINO	19	GERONIMO DE OSORIO	FUERA DE RIESGO	78.14%	Ver perfil
5	FEMENINO	21	DIURNO SACABA	EN RIESGO	23.10%	Ver perfil
6	MASCULINO	20	SAMUEL FINA SAVIO	EN RIESGO	33.61%	Ver perfil
7	MASCULINO	18	CALAMA	EN RIESGO	5.68%	Ver perfil
8	MASCULINO	19	8 DE MAYO RENE BARRIENTOS ORTUÑO	EN RIESGO	9.03%	Ver perfil
9	MASCULINO	19	JESUS FE Y ALEGRIA	EN RIESGO	3.54%	Ver perfil
10	MASCULINO	21	MISIONERAS DE CRISTO JESUS	FUERA DE RIESGO	59.38%	Ver perfil

Figura 4-19: Interfaz del módulo de evaluación masiva de postulantes mediante carga de archivos

Fuente: *Elaboración propia, 2025*

La imagen muestra el módulo de Evaluación Masiva, diseñado para permitir la aplicación del modelo predictivo a un conjunto amplio de postulantes de manera simultánea. Esta sección permite la carga de

información mediante un archivo en formato Excel (.xlsx), lo que permite procesar grandes volúmenes de datos de forma eficiente y sin la necesidad de ingresar la información de manera individual.

Una vez cargado el archivo, el sistema ejecuta automáticamente el preprocesamiento de los datos y aplica el modelo predictivo a cada registro, generando una tabla de resultados que presenta, para cada postulante, información relevante como el nombre del colegio, sexo, edad, resultado de la predicción y la probabilidad asociada a dicha predicción(ver Figura 4-15). La presentación tabular permite ordenar y revisar los resultados de forma clara, facilitando la identificación de postulantes con mayor riesgo académico.

4.6.4. Perfil de postulantes en la evaluación masiva

Dentro de la ventana “Evaluación masiva”, se encuentra el botón “Ver perfil” el cual va a abrir una adicinal en cual vamos ubicar los datos del postulante ver en la Figura 4-16.

The screenshot shows a window titled "Perfil del Postulante" with a close button (X). At the top, a green banner displays a checkmark icon, the text "FUERA DE RIESGO", and "Probabilidad estimada de aprobación: 78.14%". Below this, the data is organized into two sections: "Datos académicos" and "Datos personales". Each section contains a list of attributes with their corresponding values and a "Ver perfil" button to the right of each value.

Datos académicos		
Período académico	1	
Opción de ingreso	2	PERFIL
Nombre del colegio	GERONIMO DE OSORIO	Ver perfil
Ciudad del colegio	COCHABAMBA	Ver perfil
Provincia del colegio	COCHABAMBA (CERCADO)	Ver perfil
Año de bachillerato	2020	Ver perfil
Tipo de colegio	NO_PARTICULAR URBANO	Ver perfil
Datos personales		
Género	MASCULINO	Ver perfil
Edad en el examen	19	Ver perfil
Estado civil	SOLTERO(A)	Ver perfil
Nacionalidad	BOLIVIA	Ver perfil
Mayor de edad	Si	Ver perfil
Migración universitaria	No	Ver perfil
Años post bachillerato	1	

Figura 4-20: Interfaz del módulo de evaluación masiva de postulantes mediante carga de archivos

Fuente: *Elaboración propia, 2025*

El botón “Ver perfil” permite visualizar de manera individual y detallada la información académica y personal de cada postulante evaluado en la predicción masiva. Al hacer clic, se despliega una ventana con el perfil del postulante, que resume su nivel de riesgo académico, la probabilidad estimada de aprobación y sus principales características académicas y demográficas, organizadas de forma clara para facilitar el análisis.

5. Conclusiones

El análisis exploratorio y estadístico de los datos históricos del examen de ingreso a la FCyT-UMSS, fue de 98 mil postulantes aproximadamente, permitió identificar patrones en el desempeño académico. Se evidenció una alta tasa de reprobación superior al 75 %, así como diferencias significativas asociadas al tipo de colegio, donde los colegios particulares urbanos alcanzan tasas de aprobación cercanas al 30 %, mientras que los colegios comunitarios y fiscales rurales presentan tasas inferiores al 20%. Asimismo, se observó que variables como la edad y el sexo no existe impacto relevante en el resultado del examen, con diferencias mínimas en los promedios de puntaje final, lo que permitió caracterizar el contexto académico general de los postulantes y definir las bases para el desarrollo de modelos predictivos.

Se desarrollo modelos predictivos a partir de las variables seleccionadas permitiendo estimar la probabilidad de reprobación en el examen de ingreso, utilizando información demográfica, académica y variables derivadas del análisis exploratorio. La implementación de los algoritmos XGBoost con accuracy de 65% y AUC de 0.73; Random Forest con accuracy de 61% y AUC de 0.64.

La evaluación del desempeño de los modelos predictivos muestra que el algoritmo XGBoost presentó el mejor rendimiento, destacando especialmente en métricas relevantes para problemas con desbalance de clases. XGBoost alcanzó una precision de 38%, un recall de 68%, un F1-score de 49%, un accuracy de 65% y AUC de 0.73; superando a Random Forest y Regresión Logística, cuyos valores de recall se situaron alrededor de 0.59–0,58 y un AUC de 0.64 y 0.66. Estos resultados determinan que el modelo es capaz de identificar de manera consistente a los postulantes con mayor probabilidad de reprobación.

Cabe recalcar que, si bien se obtuvieron esas métricas, el desempeño de los modelos es moderado, esto se da por la limitada capacidad explicativa de las variables disponibles. La mayoría de las variables utilizadas corresponden a características demográficas y académicas, las cuales, de acuerdo con el análisis exploratorio realizado, no permiten una diferenciación clara entre postulantes aprobados y reprobados. Esta limitada capacidad de separación se refleja en los valores obtenidos por las métricas de desempeño. A pesar de esta restricción, los resultados alcanzados son coherentes con la estructura del conjunto de datos analizado

La implementación de la interfaz interactiva que ejecuta el modelo predictivo seleccionado permite tener una herramienta en forma de aplicación que disponible de manera local para quien lo ejecute. Esta misma permite realizar predicciones individuales y masivas, así como la visualización de indicadores estadísticos que contextualizan los resultados obtenidos como se mostró en la sección 4.3.1.

En síntesis, el estudio muestra la utilidad del análisis de datos y de los modelos predictivos como herramientas de apoyo en el ámbito académico. A pesar de las limitaciones asociadas a la disponibilidad de variables, los resultados obtenidos permiten entender mejor la reprobación en el examen de ingreso a

la FCyT–UMSS y constituyen una base para futuras investigaciones. Asimismo, este trabajo resalta la importancia de la analítica de datos en la toma de decisiones educativas orientadas a la detección temprana de postulantes en riesgo.

6. Recomendaciones

Se recomienda considerar el uso de modelos predictivos como una herramienta de apoyo complementaria al proceso de admisión de la Facultad de Ciencias y Tecnología de la Universidad Mayor de San Simón, ya que permite identificar tendencias generales y perfiles de riesgo académico, aportando información adicional para la toma de decisiones, sin sustituir los mecanismos tradicionales de evaluación, sino reforzándolos con un componente analítico basado en datos.

Se recomienda que, para un mejor desarrollo de modelos predictivos futuros, se disponga de un conjunto de variables más amplio y con mayor contenido académico, como antecedentes de calificaciones de colegio en áreas específicas como Matemáticas, Física o Química, brindar más información cuantitativa en las variables para no tener limitaciones en los resultados.

Cabe destacar que la mayoría de los trabajos relacionados con la predicción del rendimiento académico reportados en la literatura cuentan con variables académicas más detalladas, lo cual ha permitido alcanzar un mejor desempeño de los modelos predictivos desarrollados en dichos estudios.

Los resultados obtenidos son producto del análisis de datos históricos específicos. Por lo tanto, las predicciones generadas no deben considerarse definitivas, sino que requieren procesos periódicos de actualización, validación y recalibración conforme se incorporen nuevos datos a lo largo del tiempo, garantizando así la vigencia y confiabilidad del modelo.

Adicionalmente, se recomienda realizar evaluaciones longitudinales que permitan analizar el comportamiento real de los postulantes a lo largo de su trayectoria académica, con el fin de contrastar las predicciones generadas con los resultados efectivos obtenidos en semestres posteriores para validar el impacto real del modelo en la identificación temprana del riesgo académico.

Finalmente, se sugiere continuar con la mejora del modelo desarrollado y la interfaz, incorporando nuevas funcionalidades y visualizaciones interactivas que faciliten su uso a nivel institucional. Entre estas mejoras se incluye la integración con sistemas académicos existentes o la generación automática de reportes lo que permitiría que la herramienta sea un apoyo efectivo y sostenible en la identificación temprana de postulantes con mayor riesgo de reprobación.

Referencia Bibliográfica

- FCyT. (2020-2022). Guía estudiantil. *Modalidades de Admisión*, 5-6. <http://ode.fcyt.umss.edu.bo/wp-content/uploads/2022/07/Guia-Estudiantil-FCyT.pdf>
- Anguiano Hernández, E. (2009). Naive Bayes Multinomial para Clasificación de Texto Usando un Esquema de Pesado por Clases. 3.
- Avendaño, C. A., Gutiérrez, K. A., Salgado, C. F., & Alonso Dos Santos, M. (junio de 2016). *Rendimiento Académico en Estudiantes de Ingeniería Comercial: Modelo por Competencias y Factores de Influencia*. <http://dx.doi.org/10.4067/S0718-50062016000300002>
- Baker, R. S., & Inventado, P. R. (2014). Educational Data Mining and Learning Analytics. En K. Sawyer, *Cambridge Handbook of the Learning Sciences* (págs. 589-610). Cambridge University Press.
- Barreno Freire, S., Haro Jácome, O., Martínez Benítez, J., & Borja Naranjo, G. (2022). Análisis de factores determinantes en el rendimiento académico del estudiantado de la Facultad de Filosofía-Universidad Central del Ecuador. *Cátedra*, 75-97.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32. <https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>
- Burgueño, F. (2017). *Modelo predictivo para la selección de postulantes destacados a una institución de educación superior*. <https://repositorio.uchile.cl/handle/2250/147565>
- Carneiro, T., Medeiros da Nóbrega, R. V., Nepomuceno, T., Bian, G. B., De Albuquerque, V. C., & Reboucas Filho, P. P. (2018). Performance Analysis of Google Colaboratory as a Tool for Accelerating Deep Learning Applications. 6. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8485684>
- Castillo Requiz, B. J., Tarazona Silva, J. D., Tarazona Silva, C. E., Hurtado Enriquez, C., & Cornelio Orbegoso, F. A. (2023). Automatización del análisis exploratorio de datos y procesamiento geoquímico univariado empleando Python. *Rev. Inst. investig. Fac. minas metal. cienc geogr.*, 26(51), 14.
- Chaparro, J. A., Giraldo, B., & Rodón, S. (2013). Evaluación del clasificador Naïve Bayes como herramienta . *Investigación científica y tecnológica terminada.*, 89.
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. CRISP-DM Consortium (patrocinado por SPSS, NCR, Daimler-Chrysler).
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. <https://arxiv.org/abs/1603.02754>
- Contreras, L. E., Fuentes, H. J., & Rodríguez, J. I. (2020). Predicción del rendimiento académico como indicador de éxito/fracaso de los estudiantes de ingeniería, mediante aprendizaje automático.

- Predicción del rendimiento*, 13(5), 240-245. <https://www.scielo.cl/pdf/formuniv/v13n5/0718-5006-formuniv-13-05-233.pdf>
- Cover, T., & Hart, P. E. (1967). Clasificación de patrones de vecinos más cercanos. *IEEE Transactions on Information Theory*, 21-27.
- Eckert, K. B., & Suénaga, R. (2015). Análisis de deserción-permanencia de estudiantes universitarios utilizando técnica de clasificación en minería de datos. *Formación Universitaria*, 3-12.
- García Ortiz, Y., López de Castro Machado, D., & Rivero Frutos, O. (2014). Estudiantes universitarios con bajo rendimiento académico, ¿qué hacer? . *EDUMECENTRO*, 272.
- García Peñalvo, F. J., & Cruz Benito, J. (2019). *Learning analytics and academic analytics: An overview*. <https://educationaltechnologyjournal.springeropen.com/articles/10.1186/s41239-019-0155-0>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Aprendizaje profundo*. Prensa del MIT.
- Hand, D., & Till, R. (2001). A simple generalisation of the area under the ROC curve for multiple class classification problems. *Machine Learning*, 45(2), 171-186. <https://link.springer.com/content/pdf/10.1023/A:1010920819831.pdf>
- Herrero, V., Merlino, A., Ayllon, S., & Escanes, G. (2013). Aplicación de un modelo de duración en programas de prevención de deserción universitaria. *Revista Electrónica de Investigación Educativa*, 38-52.
- Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression*. Wiley. http://books.google.es/books?hl=es&lr=&id=bRoxQBIZRd4C&oi=fnd&pg=PR13&dq=Applied+logistic+regression&ots=kM6OxsdRia&sig=3vHKkQ8RVyhOQIveaaLe_LUs-F0#v=onepage&q=Applied%20logistic%20regression&f=false
- Kabakchieva, D. (2013). Predicting student performance by using data mining methods for classification. *Cybernetics and Information Technologies*, 13(1), 61-72. <https://doi.org/https://doi.org/10.2478/cait-2013-0006>
- Kotsiantis, S. (2007). Supervised machine learning: A review of classification techniques. *Informatica*, 31(3), 249-268. https://books.google.es/books?hl=es&lr=&id=vLiTXDHR_sYC&oi=fnd&pg=PA3&dq=Supervised+machine+learning:+A+review+of+classification+techniques.&ots=C_qvzA-Gno&sig=KrOalP6A2h6_6N2CO8oUfPfwWs#v=onepage&q=Supervised%20machine%20learning%3A%20A%20review%20of%20
- Lamba, M., & Madhusudhan, M. (2022). *Minería de texto para profesionales de la información*. Springer.
- Larose, D. T., & Larose, C. D. (2004). *Discovering Knowledge in Data: An Introduction to Data Mining*. . Wiley. https://doc.lagout.org/Others/Data%20Mining/Discovering%20Knowledge%20in%20Data_%20An%20Introduction%20to%20Data%20Mining%20%282nd%20ed.%29%20%5BLarose%20%26%20Larose%202014-06-30%5D.pdf
- López Altamirano, D. A., Gómez Morales, M., Mayorga Alvarado, F., Paredes Ojeda, M. E., & Martínez Pérez, S. G. (2020). La puntuación del examen Ser Bachiller como predictor del rendimiento académico universitario.

- Menacho Chiok, C. (2020). Predicción del rendimiento académico aplicando técnicas de minería de datos. *Dialnet.*, 27. <https://dialnet.unirioja.es/>
- Microsoft Power Platform. (2020). <https://powerbi.microsoft.com/es-es/what-is-power-bi/>
- Ministerio de Educación. (2019). *Ministerio de Educación del Estado Plurinacional de Bolivia*. Estadísticas de la educación en Bolivia. <https://www.minedu.gob.bo/>
- Molina Félix, L. C., & Sangüesa i Solé, R. (2010). *Evaluación de Modelos*. <https://openaccess.uoc.edu/server/api/core/bitstreams/8f717da2-0d0a-49b5-a46e-22f7127422ce/content>
- Montano, M. (2020).
- Moré, J. (2019). *Evaluación de la calidad de los sistemas de reconocimiento de sentimientos*. Barcelona: FUOC.
- Moreno Seco, F. (2004). Clasificadores eficaces basados en algoritmos rápidos de búsqueda del vecino más cercano. 56-89.
- Padilla Magaña, R. A. (2007). El sentido del examen en la educación superior ¿Reproducción o demostración de lo aprendido? *Universidad Autónoma Metropolitana Unidad Xochimilco Reencuentro*, 27-33. <https://www.redalyc.org/pdf/340/34004804.pdf>
- Perez Martinez, P., & Peredo, M. (2021). *jdshfdf*. Bogota.
- Perez, P. (2024). *ddgh*.
- Rodriguez Villanueva, A. A., Sánchez Adauto, E. L., & Valverde Rojo, L. M. (2023). Aplicación de técnicas de Machine Learning para predecir el número de ventas de créditos en el sector bancario. *Repositorio Institucional Universidad ESAN*, 27. <https://repositorio.esan.edu.pe/server/api/core/bitstreams/1bbdba28-32d1-4881-842f-1cf9cbede622/content>
- Romero, C., & Ventura, S. (2020). *Educational Data mining and Learning Analytics: An updated survey*. <https://arxiv.org/pdf/2402.07956>
- Sadiku, M. N., Ajayi, S. A., & Sadiku, J. O. (2025). Machine Learning: An Overview. *International Journal of Trend in Scientific Research and Development*, 9(5), 1121-1131. <https://www.ijtsrd.com/papers/ijtsrd97693.pdf>
- Sandoval, L. J. (2018). ALGORITMOS DE APRENDIZAJE AUTOMÁTICO PARA ANÁLISIS Y. *REVISTA TECNOLÓGICA*, 37-38.
- Santiesteban López, N., & López Malo, A. (2008). Descripción e importancia de algunos modelos predictivos utilizados como herramienta para la conservación de alimentos. *Temas Selectos de Ingeniería de Alimentos*, 14-26.
- Sutton, R. S., & Barto, A. G. (2018). *Aprendizaje por refuerzo: una introducción*. Prensa del MIT.
- Tamayo Guajala, L. P., Tinitana Ordoñez, A. G., Apolo Castillo, J. E., Martínez Avelino, E. I., & Zambrano Pérez, V. L. (2021). Examen para el ingreso a la Educación Superior. *Sociedad & Tecnología*, 439. <https://institutojubones.edu.ec/ojs/index.php/societec/article/view/162/450>
- Timón, C. E. (16 de Enero de 2017). *Análisis predictivo: técnicas y modelos utilizados y aplicaciones del mismo- herramientas Open Source que permitan su uso*. Análisis predictivo: técnicas y modelos utilizados y aplicaciones del mismo- herramientas Open Source que permitan su uso.

<https://openaccess.uoc.edu/server/api/core/bitstreams/91aa36a3-f15c-40f3-bab9-a156722e81de/content>

- Torres López, C. A., Pachas Huaytan, J. V., López Gómez, H. E., Sánchez Soto, J. M., & Ortiz Fernández, J. H. (2024). *Predicción del rendimiento académico mediante modelos de regresión logística y el análisis discriminante*. <https://biblioteca.ciencialatina.org/wp-content/uploads/2024/04/Prediccion-del-rendimiento-academico-mediante-modelos-de-regresion-logistica-y-el-analisis-discriminante.pdf>
- Universidad Mayor San Simon Facultad de Ciencias y Tecnologia . (01 de Enero de 2017). FCyT datos y cifras 2017. *FCyT datos y cifras N° 2017*. Cochabamba, Cercado, Bolivia.
- Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the Fourth International Conference on the Practical Application of Knowledge Discovery and Data Mining*, 29-39. <https://doi.org/Manchester, United Kingdom>

Anexos

Anexo 1. Dataset de los datos de los postulantes

Datos demográficos y académicos de los postulantes al examen de ingreso entre los años 2006 a 2025.

RABAJ	BACHILL	NOMBRE COLEGIO	PROVINCIA COLEGIO	CIUDAD COLEGIO	TIPO D. COLE	OM. FIN. TADO
NO	0	FRANCISCO VIGNAUD	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL URBANO	ABA
NO	0	MEJILLONES	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	21 REP
NO	0	INCOS	ANTONIO VACA DIEZ	BENI	2 FISCAL U	31 REP
NO	0	FE Y ALEGRIA	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	11 REP
NO	0	NORDICO BOLIVIANO	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	21 REP
NO	0	FRANCISCO GONZALES PRADA	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	31 REP
NO	0	MARIA AUXILIADORA	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	51 APR
NO	0	DE MAYO RENE BARRIENTOS ORTUÑO	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	11 REP
NO	0	ANTONIO QUIJARRO	TOMAS FRIAS	POTOSI	2 FISCAL U	51 APR
NO	0	KHOCHAPAMPA	CHAPARE	COCHABAMBA	2 PARTICU	26 REP
NO	0	SAN MARCOS	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	26 REP
NO	0	COCHABAMBA	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	21 REP
NO	0	PUERTO DE MEJILLONES (TECNOLOGICO)	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	26 REP
NO	0	EUCALIPTUS	TOMAS BARRON	ORURO	1 FISCAL F	26 REP
NO	2005	COPACABANA	MURILLO	LAPAZ	2 FISCAL U	6 REP
NO	0	ALCIDES ARGUEDAS	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	21 REP
NO	2005	MARISCAL SANTA CRUZ	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICULAR URB	ABA
NO	0	CEA - TERESA DE CANCUTA	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL URBANO	ABA
NO	2006	PEDRO POVEDA	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	31 REP
NO	2005	SACABA DIURNO	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	26 REP
NO	0	SAN FRANCISCO XAVIER	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	16 REP
NO	0	SANTA CLARA DE ASIS	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	6 REP
NO	0	INSTITUTO AMERICANO	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	16 REP
NO	0	GERONIMO DE OSORIO	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	51 APR
NO	0	DANIEL SANCHEZ BUSTAMANTE	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	51 APR
NO	0	SANTA LUISA DE MARILLAC	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	16 REP
NO	0	VISION MUNDIAL	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	56 APR
NO	0	MARIANO MELGAREJO (GENERAL)	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	51 APR
NO	0	SAN MIGUEL	QUILLACOLLO	COCHABAMBA	1 FISCAL F	51 APR
NO	2005	BOLIVIANO ARGENTINO	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	31 REP
NO	0	SAINT GERMAIN	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	51 APR
NO	2006	JESUS FE Y ALEGRIA	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	21 REP
NO	2003	EDUARDO AVAROA	CERCADO	BENI	2 FISCAL U	21 REP
NO	0	LICEO GRAN CHACO	TARIJA (CERCADO)	TARIJA	2 FISCAL U	26 REP
NO	0	MARTIN CARDENAS	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	31 REP
NO	0	CALAMA	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	21 REP
NO	0	MANUEL ASCENCIO VILLARROEL	PUNATA	COCHABAMBA	1 FISCAL F	21 REP
NO	2005	SAINT ANDREWS	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	16 REP
NO	0	DON BOSCO	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	31 REP
NO	0	ILAT	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	31 REP
NO	0	COMPAÑIA DE JESUS	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	51 APR
NO	2005	MARYKNOLL	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	6 REP
NO	0	PUERTO DE MEJILLONES (TECNOLOGICO)	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	51 APR
NO	0	INSTITUTO AMERICANO	COCHABAMBA (CERCADO)	COCHABAMBA	2 PARTICU	51 APR
NO	0	ELENA ARZE DE ARZE	COCHABAMBA (CERCADO)	COCHABAMBA	2 FISCAL U	51 APR

Fuente: Elaborado por FCyT-UMSS, 2025

Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_1_Datos_proporcionados.png

Ubicación CD: \Modelo PredictivoFCyT_UMSS\Anexos\Anexo_1_Datos_proporcionados

Anexo 2. Creación de las variables

Para un mejor modelo predictivo, se crean variables que ayudaran en el proceso.

Columna EDAD

```
df_postulantes['FECHA_NAC'] = pd.to_datetime(df_postulantes['FECHA_NAC'], errors='coerce', dayfirst=True)

df_postulantes['EDAD'] = df_postulantes.apply(
    lambda fila: fila['ANIO'] - fila['FECHA_NAC'].year
    - ((1, 1) < (fila['FECHA_NAC'].month, fila['FECHA_NAC'].day)), axis=1)
```

Columna ANIOS_POST_BACH

```
df_postulantes["ANIOS_POST_BACH"] = (df_postulantes["ANIO"] - df_postulantes["ANIO_BACHILLERATO"])
    .where(df_postulantes["ANIO_BACHILLERATO"] != 0, 0)

df_postulantes["ANIOS_POST_BACH"] = df_postulantes["ANIOS_POST_BACH"].astype(int)
```

Columna TRABAJO_COLEGIO

```
df_postulantes["TRABAJO_COLEGIO"] = (df_postulantes["TRABAJO"].astype(str) + "_"
    + df_postulantes["TIPO_COLEGIO"].astype(str))
```

Columna MIGRA_UNIVERSIDAD

```
df_postulantes["MIGRA_UNIVERSIDAD"] = ((df_postulantes["CIUDAD_COLEGIO"] != "COCHABAMBA" |
    (df_postulantes["PROVINCIA_COLEGIO"] != "COCHABAMBA (CERCADO)")).astype(int))
```

Columna MAYOR_EDAD

```
df_postulantes["MAYOR_EDAD"] = (df_postulantes["EDAD"] >= 18).astype(int)
```

Columna TASA_APR_COLEGIO

```
tasa_col = (df_postulantes.groupby("NOMBRE_COLEGIO")["RESULTADO_FINAL"]
    .apply(lambda x: (x == "APR").mean()))

df_postulantes["TASA_APR_COLEGIO"] = df_postulantes["NOMBRE_COLEGIO"].map(tasa_col)
```

Fuente: Elaborado propia, 2025

Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT-UMSS/blob/main/Anexos/Anexo_2_Creacion_de_las_variables.JPG

Ubicación CD: \Modelo PredictivoFCyT-UMSS\Anexos\Anexo_2_Creacion_de_las_variables

Anexo 3. Filtrado y eliminación de datos para el modelo Predictivo

Código de la filtración y eliminación de las filas que contienen datos fuera del rango o nulos, en variables específicas.

```
# Filtro de edad (15 a 43 años)
df_postulantes = df_modelo[
    (df_modelo["EDAD"] >= 15) &
    (df_modelo["EDAD"] <= 43)
]
#Eliminacion de la columna EDAD
df_modelo = df_modelo.drop(columns=["EDAD"])
# Filtrar valores inválidos de ANIO_BACHILLERATO solo para el modelo
df_modelo = df_modelo[df_modelo["ANIO_BACHILLERATO"] > 0]

df_modelo = df_modelo.dropna(subset=['MUNICIPIO'])
```

Fuente: Elaborado propia, 2025

Ubicación Github:

[https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_3_Filtrado y eliminacion de datos.png](https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_3_Filtrado_y Eliminacion de datos.png)

Ubicación CD: \Modelo PredictivoFCyT_UMSS\Anexos\Anexo_3_Filtrado_y eliminacion de datos

Anexo 4. Asignación de variables categóricas y numéricas

Código que asigna las variables escogidas para el modelo en categóricas y numéricas para su tratamiento.

```
# Identificar tipos de variables
cat_features = X.select_dtypes(include=["object"]).columns
num_features = X.select_dtypes(include=["int64", "float64"]).columns
print("Categóricas:", cat_features)
print("Numéricas:", num_features)

# Preprocesador
preprocesador = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_features),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_features)
    ]
)
```

Fuente: Elaborado propia, 2025

Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_4_Asignacion categorica numerica%20.JPG

Ubicación CD: Modelo PredictivoFCyT_UMSS\Anexos\Anexo_4_Asignacion_categorica_numerica

Anexo 5. Código de implementación de XGBoost

Este bloque identifica las variables categóricas y numéricas para aplicar un preprocesamiento adecuado a cada tipo de dato. Posteriormente, se integra el modelo dentro de un pipeline que es entrenado y utilizado para generar predicciones y probabilidades sobre los conjuntos de entrenamiento y prueba.

```
# Identificar tipos de variables
cat_features = X.select_dtypes(include=["object"]).columns
num_features = X.select_dtypes(include=["int64", "float64"]).columns
print("Categóricas:", cat_features)
print("Numéricas:", num_features)
# Preprocesador
preprocesador = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_features),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_features)])
# Modelo XGBoost
xgb_modelo = XGBClassifier(
    n_estimators=800, max_depth=5, learning_rate=0.05, subsample=0.7,
    colsample_bytree=0.7, eval_metric="logloss", random_state=42, reg_alpha=2, reg_lambda=3
)
# Undersampling
undersampler = RandomUnderSampler(
    sampling_strategy=1.0, random_state=42
)
pipeline = ImbPipeline(steps=[
    ("preproceso", preprocesador),
    ("undersample", undersampler),
    ("modelo", xgb_modelo)
])
# Entrenamiento del modelo
pipeline.fit(X_train, y_train)
# Predicciones
y_train_pred = pipeline.predict(X_train)
y_train_prob = pipeline.predict_proba(X_train)[: , 1]
y_test_pred = pipeline.predict(X_test)
y_test_prob = pipeline.predict_proba(X_test)[: , 1]
```

Fuente: Elaborado propia, 2025

Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_5_XGB_OOST.png

Ubicación CD: Modelo PredictivoFCyT_UMSS\Anexos\Anexo_5_XGBOOST

Anexo 6. Código de implementación de Random Forest

En este bloque se configura el modelo dentro de un pipeline. Finalmente, el pipeline es entrenado y utilizado para generar predicciones y probabilidades sobre los conjuntos de entrenamiento y prueba.

```
rf_modelo = RandomForestClassifier(
    n_estimators=800,
    max_depth=5,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)
# Undersampling
undersampler = RandomUnderSampler(
    sampling_strategy=1.0, # Balancea 1:1
    random_state=42
)
#Preprocesamiento y Pipeline
pipeline_rf = Pipeline(steps=[
    ("preprocesamiento", preprocesador),
    ("undersampling", undersampler),
    ("modelo", rf_modelo)
])
pipeline_rf.fit(X_train, y_train)
y_train_pred_rf = pipeline_rf.predict(X_train)
y_train_prob_rf = pipeline_rf.predict_proba(X_train)[:, 1]

y_test_pred_rf = pipeline_rf.predict(X_test)
y_test_prob_rf = pipeline_rf.predict_proba(X_test)[:, 1]
```

Fuente: Elaborado propia, 2025

Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_6_Random_Forest.png

Ubicación CD: Modelo PredictivoFCyT_UMSS\Anexos\Anexo_6_Random_Forest

Anexo 7. Código de Implementación de Regresión logística binaria

En este bloque se implementa el modelo y se incorpora una técnica de balanceo. El preprocesamiento, el balanceo y el modelo se integran dentro de un pipeline para ser entrenado y utilizado para generar predicciones y probabilidades en los conjuntos de entrenamiento y prueba.

```
logreg_modelo = LogisticRegression(
    penalty="l2",
    solver="liblinear",
    max_iter=500,
    random_state=42
)
# Undersampling
undersampler = RandomUnderSampler(
    sampling_strategy=1.0, # Balancea 1:1
    random_state=42
)
pipeline_logreg = Pipeline(steps=[
    ("preprocesamiento", preprocesador),
    ("undersampling", undersampler),
    ("modelo", logreg_modelo)
])
pipeline_logreg.fit(X_train, y_train)
y_train_pred_lr = pipeline_logreg.predict(X_train)
y_train_prob_lr = pipeline_logreg.predict_proba(X_train)[:, 1]

y_test_pred_lr = pipeline_logreg.predict(X_test)
y_test_prob_lr = pipeline_logreg.predict_proba(X_test)[:, 1]
```

Fuente: Elaborado propia, 2025

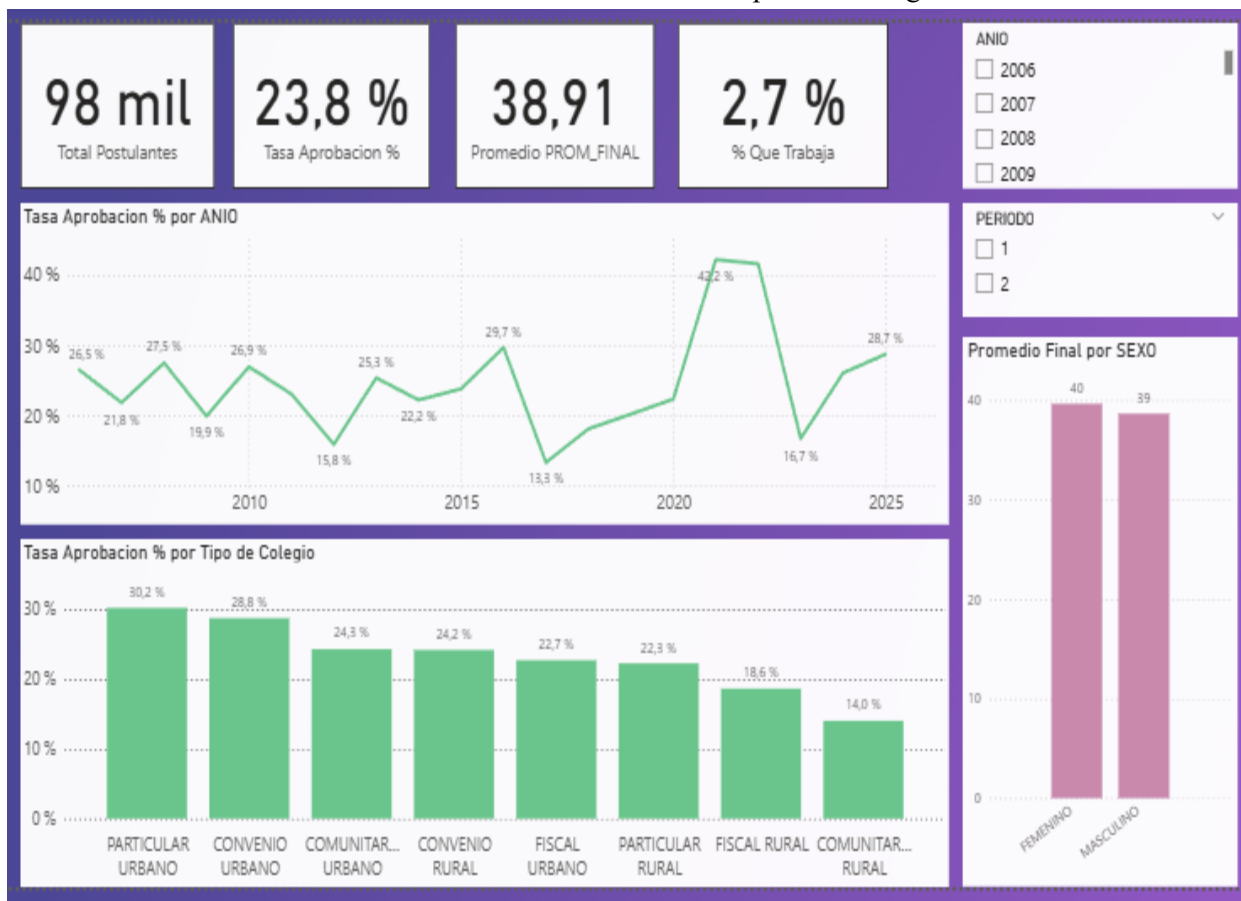
Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_7_Regresion_Logistica_Binaria.png

Ubicación CD: Modelo PredictivoFCyT_UMSS\Anexos\Anexo_7_Regresion_Logistica_Binaria

Anexo 8. Dashboards implementado Panorama General

Los Dashboards son muy importantes a nivel visual para comprender el comportamiento de los datos. A continuación se muestra el dashboard con un panorama general de los datos.



Fuente: Elaborado propia, 2025

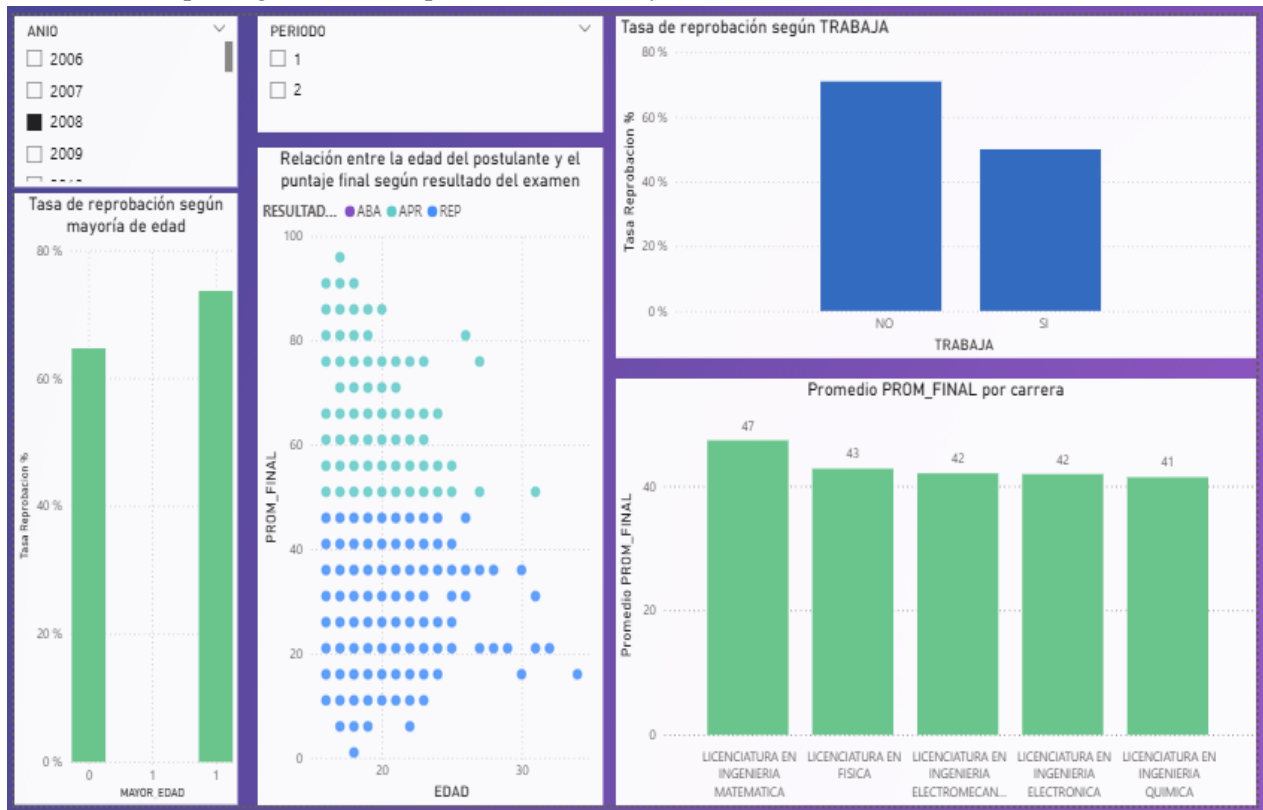
Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_8_Panorama_General.png

Ubicación CD: Modelo PredictivoFCyT_UMSS\Anexos\Anexo_8_Panorama_General

Anexo 9. Dashboards implementado Perfil de los estudiantes

Se visualiza el perfil general de los postulantes a la FCyT.



Fuente: Elaborado propia, 2025

Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_9_Perfil_Estudiantes.png

Ubicación CD: Modelo PredictivoFCyT_UMSS\Anexos\Anexo_8_Perfil_Estudiantes

Anexo 10. Dashboards implementado Origen y Colegio de los postulantes

A continuación, se muestra el dashboard donde se aprecia mediante graficas de barra, las 6 ciudades con mayor tasa de reprobación, los 7 colegios con mayor nota de examen, las ciudades cuyo colegio tiene mayor tasa de reprobación y la cantidad de reprobados.



Fuente: Elaborado propia, 2025

Ubicación Github:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_diez_Origen_y_colegios.png

Ubicación CD: Modelo PredictivoFCyT_UMSS\Anexos\Anexo_diez_Origen_y_colegios

Anexo 11. Dashboards implementado Aprobados y reprobados

El siguiente dashboard muestra los aprobados y reprobados por el origen y su colegio, añadiendo filtros para una visualización interactiva.



Fuente: Elaborado propia, 2025

Ubicación Github:

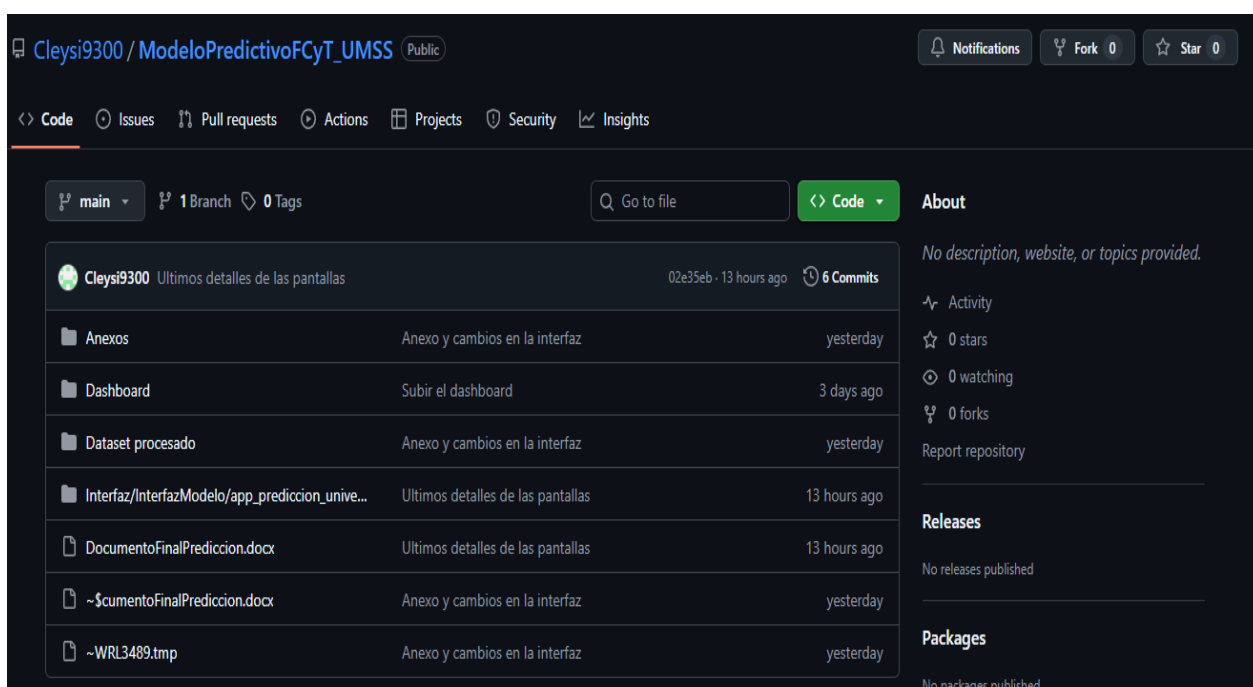
https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS/blob/main/Anexos/Anexo_once_Aprobados_Reprobados.png

Ubicación CD: Modelo PredictivoFCyT_UMSS\Anexos\Anexo_once_Aprobados_Reprobados

Anexo 12. Repositorio de Github

En el repositorio se encuentran todos los archivos utilizados en el proyecto de Analisis estadístico y modelos predictivos, en el siguiente link:

https://github.com/Cleysi9300/ModeloPredictivoFCyT_UMSS



Cleysi9300 / ModeloPredictivoFCyT_UMSS (Public)

Notifications Fork 0 Star 0

Code Issues Pull requests Actions Projects Security Insights

main 1 Branch 0 Tags Go to file Code

File/Folder	Commit Message	Commit Time
Anexos	Anexo y cambios en la interfaz	yesterday
Dashboard	Subir el dashboard	3 days ago
Dataset procesado	Anexo y cambios en la interfaz	yesterday
Interfaz/InterfazModelo/app_prediccion_unive...	Ultimos detalles de las pantallas	13 hours ago
DocumentoFinalPrediccion.docx	Ultimos detalles de las pantallas	13 hours ago
~DocumentoFinalPrediccion.docx	Anexo y cambios en la interfaz	yesterday
~WRL3489.tmp	Anexo y cambios en la interfaz	yesterday

About: No description, website, or topics provided.

Activity: 0 stars, 0 watching, 0 forks

Releases: No releases published

Packages: No packages published

Adjunto qr para el repositorio:



Anexo 13. CD con el contenido del proyecto

Se detalla los directorios y archivos que contiene el CD del proyecto.

Revisar el CD adjunto o visitar el enlace